prpy: Probabilistic Robot Localization Python Library

Release 0.1

Pere Ridao

Oct 22, 2024

CONTENTS

Probabilistic Robot Localization is Python Library containing the main algorithms explained in the **Probabilisitic Robot Localization** Book used in the **Probabilisitic Robotics** and the **Hands-on Localization** Courses of the **Intelligent Field Robotic Systems (IFRoS)** European Erasmus Mundus Master.

Note: This documentation is still under construction.

CHAPTER

ONE

API

1.1 Pose Representation

1.1.1 Pose

class Pose.**Pose**

Bases: ndarray

Definition of a robot pose interface from where all the particular poses of different DOF inherit. This class defines a robot pose AxB as the pose of the B-Frame expressed in the A-Frame coordinates.

oplus(*BxC*)

Given a Pose object *AxB* (the self object) and a Pose object *BxC*, it returns the compounded Pose object *AxC*.

The operation is defined as:

$$^{\mathbf{A}}\mathbf{x_C} = {}^{\mathbf{A}}\mathbf{x_B} \oplus {}^{\mathbf{B}}\mathbf{x_C} \tag{1.1}$$

This is a pure virtual method that must be implemented by a child class.

Parameters
BxC – C-Frame pose expressed in B-Frame coordinates

Returns
C-Frame pose expressed in A-Frame coordinates

J_1oplus(*BxC*)

Jacobian of the pose compounding operation (eq. (1.1)) with respect to the first pose:

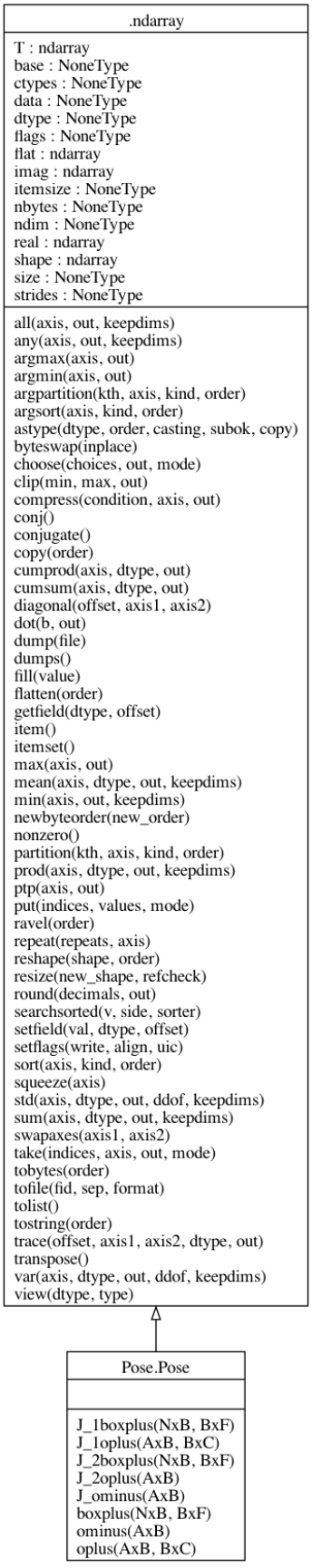
$$J_{1\oplus} = \frac{\partial^A x_B \oplus^B x_C}{\partial^A x_B} \tag{1.2}$$

The method returns a numerical matrix containing the evaluation of the Jacobian for the pose *AxB* (the self object) and the 2nd pose *BxC*.

This is a pure virtual method that must be implemented by a child class.

Parameters
BxC – 2nd pose

Returns
Evaluation of the $J_{1\oplus}$ Jacobian of the pose compounding operation with respect to the first pose (eq. (1.2))


.ndarray
T : ndarray
base : NoneType
ctypes : NoneType
data : NoneType
dtype : NoneType
flags : NoneType
flat : ndarray
imag : ndarray
itemsize : NoneType
nbytes : NoneType
ndim : NoneType
real : ndarray
shape : ndarray
size : NoneType
strides : NoneType
all(axis, out, keepdims)
any(axis, out, keepdims)
argmax(axis, out)
argmin(axis, out)
argpartition(kth, axis, kind, order)
argsort(axis, kind, order)
astype(dtype, order, casting, subok, copy)
byteswap(inplace)
choose(choices, out, mode)
clip(min, max, out)
compress(condition, axis, out)
conj()
conjugate()
copy(order)
cumprod(axis, dtype, out)
cumsum(axis, dtype, out)
diagonal(offset, axis1, axis2)
dot(b, out)
dump(file)
dumps()
fill(value)
flatten(order)
getfield(dtype, offset)
item()
itemset()
max(axis, out)
mean(axis, dtype, out, keepdims)
min(axis, out, keepdims)
newbyteorder(new_order)
nonzero()
partition(kth, axis, kind, order)
prod(axis, dtype, out, keepdims)
ptp(axis, out)
put(indices, values, mode)
ravel(order)
repeat(repeats, axis)
reshape(shape, order)
resize(new_shape, refcheck)
round(decimals, out)
searchsorted(v, side, sorter)
setfield(val, dtype, offset)
setflags(write, align, uic)
sort(axis, kind, order)
squeeze(axis)
std(axis, dtype, out, ddof, keepdims)
sum(axis, dtype, out, keepdims)
swapaxes(axis1, axis2)
take(indices, axis, out, mode)
tobytes(order)
tofile(fid, sep, format)
tolist()
tostring(order)
trace(offset, axis1, axis2, dtype, out)
transpose()
var(axis, dtype, out, ddof, keepdims)
view(dtype, type)
Pose.Pose
J_1boxplus(NxB, BxF)
J_1oplus(AxB, BxC)
J_2boxplus(NxB, BxF)
J_2oplus(AxB)
J_ominus(AxB)
boxplus(NxB, BxF)
ominus(AxB)
oplus(AxB, BxC)

J_2oplus()

Jacobian of the pose compounding operation ((1.1)) with respect to the second pose:

$$J_{2\oplus} = \frac{\partial^A x_B \oplus^B x_C}{\partial^B x_C} \tag{1.3}$$

The method returns a numerical matrix containing the evaluation of the Jacobian for the 1st pose *AxB* (the self object).

This is a pure virtual method that must be implemented by a child class.

Returns
Evaluation of the $J_{2\oplus}$ Jacobian of the pose compounding operation with respect to the second pose (eq. (1.3))

ominus()

Inverse pose compounding of the *AxB* pose (the self object):

$${}^B x_A = \ominus^A x_B \tag{1.4}$$

This is a pure virtual method that must be implemented by a child class.

Returns
A-Frame pose expressed in B-Frame coordinates (eq. (1.4))

J_ominus()

Jacobian of the inverse pose compounding operation ((1.1)) with respect the pose *AxB* (the self object):

$$J_{\ominus} = \frac{\partial \ominus^A x_B}{\partial^A x_B} \tag{1.5}$$

Returns the numerical matrix containing the evaluation of the Jacobian for the pose *AxB* (the self object).
This is a pure virtual method that must be implemented by a child class.

Returns
Evaluation of the $J_{\ominus}$ Jacobian of the inverse pose compounding operation with respect to the pose (eq. (1.12))

boxplus(*BxF*)

Given a Pose object *NxB* (the self object) and a Feature object *BxF*, it returns the Feature object *NxF* providing the same feature but now expresend in the N-Frame.

Parameters
BxF – Feature object expressed in the B-Frame

Returns
NxF Feature object expressed in the N-Frame

J_1boxplus(*BxF*)

Jacobian of the pose-feature compounding operation (eq. (1.18)) with respect to the robot pose:

$$J_{1\boxplus} = \frac{\partial^N x_B \boxplus^B x_F}{\partial^N x_B} \tag{1.6}$$

Parameters
BxF – Feature object expressed in the B-Frame

Returns
$J_{1\boxplus}$ Jacobian of the feature compounding operation with respect to the robot pose (eq. (1.6))

J_2boxplus(*BxF*)

Jacobian of the pose-feature compounding operation (eq. (1.18)) with respect to the feature:

$$J_{2\boxplus} = \frac{\partial^N x_B \boxplus^B x_F}{\partial^B x_F} \tag{1.7}$$

Parameters
BxF – Feature object expressed in the B-Frame

Returns
$J_{2\boxplus}$ Jacobian of the feature compounding operation with respect to the feature (eq. (1.7))

1.1.2 Pose 3DOF

class Pose.**Pose3D**(*input_array=array([[0.], [0.], [0.]])*)

Bases: *Pose*

Definition of a robot pose in 3 DOF (x, y, yaw). The class inherits from a ndarray. This class extends the ndarray with the $oplus$ and $ominus$ operators and the corresponding Jacobians.

__init__(*input_array=array([[0.], [0.], [0.]])*)

oplus(*BxC*)

Given a Pose3D object *AxB* (the self object) and a Pose3D object *BxC*, it returns the Pose3D object *AxC*.

$$\mathbf{^A_{x_B}} = \begin{bmatrix} {}^A x_B & {}^A y_B & {}^A \psi_B \end{bmatrix}^T$$
$$\mathbf{^B_{x_C}} = \begin{bmatrix} {}^B x_C & {}^B y_C & {}^B \psi_C \end{bmatrix}^T$$

The operation is defined as:

$$\mathbf{^A_{x_C}} = \mathbf{^A_{x_B}} \oplus \mathbf{^B_{x_C}} = \begin{bmatrix} {}^A x_B +^B x_C \cos({}^A \psi_B) -^B y_C \sin({}^A \psi_B) \\ {}^A y_B +^B x_C \sin({}^A \psi_B) +^B y_C \cos({}^A \psi_B) \\ {}^A \psi_B +^B \psi_C \end{bmatrix} \tag{1.8}$$

Parameters
BxC – C-Frame pose expressed in B-Frame coordinates

Returns
C-Frame pose expressed in A-Frame coordinates

J_1oplus(*BxC*)

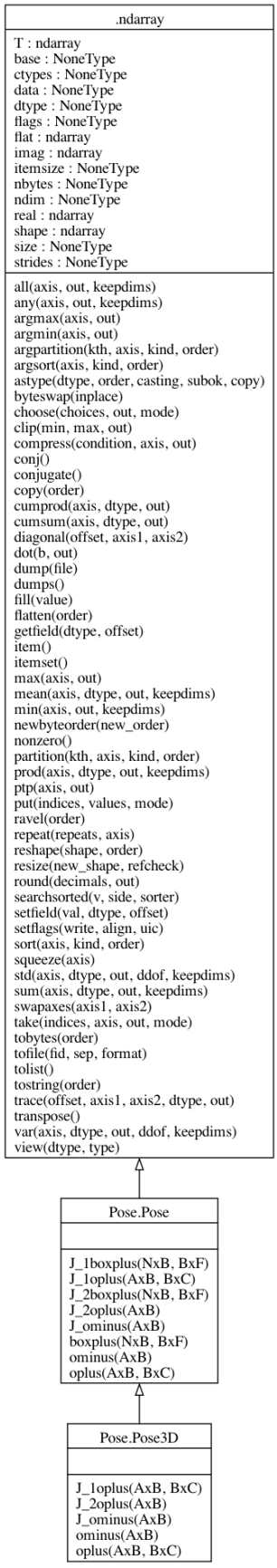
Jacobian of the pose compounding operation (eq. (1.8)) with respect to the first pose:

$$J_{1\oplus} = \frac{\partial^A x_B \oplus^B x_C}{\partial^A x_B} = \begin{bmatrix} 1 & 0 & -^B x_C \sin({}^A \psi_B) -^B y_C \cos({}^A \psi_B) \\ 0 & 1 & {}^B x_C \cos({}^A \psi_B) -^B y_C \sin({}^A \psi_B) \\ 0 & 0 & 1 \end{bmatrix} \tag{1.9}$$

The method returns a numerical matrix containing the evaluation of the Jacobian for the pose *AxB* (the self object) and the 2nd posepose *BxC*.

Parameters
BxC – 2nd pose

Returns
Evaluation of the $J_{1\oplus}$ Jacobian of the pose compounding operation with respect to the first pose (eq. (1.9))


.ndarray
T : ndarray
base : NoneType
ctypes : NoneType
data : NoneType
dtype : NoneType
flags : NoneType
flat : ndarray
imag : ndarray
itemsize : NoneType
nbytes : NoneType
ndim : NoneType
real : ndarray
shape : ndarray
size : NoneType
strides : NoneType
all(axis, out, keepdims)
any(axis, out, keepdims)
argmax(axis, out)
argmin(axis, out)
argpartition(kth, axis, kind, order)
argsort(axis, kind, order)
astype(dtype, order, casting, subok, copy)
byteswap(inplace)
choose(choices, out, mode)
clip(min, max, out)
compress(condition, axis, out)
conj()
conjugate()
copy(order)
cumprod(axis, dtype, out)
cumsum(axis, dtype, out)
diagonal(offset, axis1, axis2)
dot(b, out)
dump(file)
dumps()
fill(value)
flatten(order)
getfield(dtype, offset)
item()
itemset()
max(axis, out)
mean(axis, dtype, out, keepdims)
min(axis, out, keepdims)
newbyteorder(new_order)
nonzero()
partition(kth, axis, kind, order)
prod(axis, dtype, out, keepdims)
ptp(axis, out)
put(indices, values, mode)
ravel(order)
repeat(repeats, axis)
reshape(shape, order)
resize(new_shape, refcheck)
round(decimals, out)
searchsorted(v, side, sorter)
setfield(val, dtype, offset)
setflags(write, align, uic)
sort(axis, kind, order)
squeeze(axis)
std(axis, dtype, out, ddof, keepdims)
sum(axis, dtype, out, keepdims)
swapaxes(axis1, axis2)
take(indices, axis, out, mode)
tobytes(order)
tofile(fid, sep, format)
tolist()
tostring(order)
trace(offset, axis1, axis2, dtype, out)
transpose()
var(axis, dtype, out, ddof, keepdims)
view(dtype, type)
Pose.Pose
J_1boxplus(NxB, BxF)
J_1oplus(AxB, BxC)
J_2boxplus(NxB, BxF)
J_2oplus(AxB)
J_ominus(AxB)
boxplus(NxB, BxF)
ominus(AxB)
oplus(AxB, BxC)
Pose.Pose3D
J_1oplus(AxB, BxC)
J_2oplus(AxB)
J_ominus(AxB)
ominus(AxB)
oplus(AxB, BxC)

J_2oplus()

Jacobian of the pose compounding operation ((1.8)) with respect to the second pose:

$$J_{2\oplus} = \frac{\partial {}^{A}x_B \oplus {}^{B}x_C}{\partial {}^{B}x_C} = \begin{bmatrix} \cos({}^{A}\psi_B) & -\sin({}^{A}\psi_B) & 0 \\ \sin({}^{A}\psi_B) & \cos({}^{A}\psi_B) & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{1.10}$$

The method returns a numerical matrix containing the evaluation of the Jacobian for the :math:`1st posepose *AxB* (the self object).

Returns

Evaluation of the $J_{2\oplus}$ Jacobian of the pose compounding operation with respect to the second pose (eq. (1.10))

ominus()

Inverse pose compounding of the *AxB* pose (the self objetc):

$${}^{B}x_A = \ominus {}^{A}x_B = \begin{bmatrix} -{}^{A}x_B \cos({}^{A}\psi_B) - {}^{A}y_B \sin({}^{A}\psi_B) \\ {}^{A}x_B \sin({}^{A}\psi_B) - {}^{A}y_B \cos({}^{A}\psi_B) \\ -{}^{A}\psi_B \end{bmatrix} \tag{1.11}$$

Returns

A-Frame pose expressed in B-Frame coordinates (eq. (1.11))

J_ominus()

Jacobian of the inverse pose compounding operation ((1.8)) with respect the pose *AxB* (the self object):

$$J_{\ominus} = \frac{\partial \ominus {}^{A}x_B}{\partial {}^{A}x_B} = \begin{bmatrix} -\cos({}^{A}\psi_B) & -\sin({}^{A}\psi_B) & {}^{A}x_B \sin({}^{A}\psi_B) - {}^{A}y_B \cos({}^{A}\psi_B) \\ \sin({}^{A}\psi_B) & -\cos({}^{A}\psi_B) & {}^{A}x_B \cos({}^{A}\psi_B) + {}^{A}y_B \sin({}^{A}\psi_B) \\ 0 & 0 & -1 \end{bmatrix} \tag{1.12}$$

Returns the numerical matrix containing the evaluation of the Jacobian for the pose *AxB* (the self object).

Returns

Evaluation of the $J_{\ominus}$ Jacobian of the inverse pose compounding operation with respect to the pose (eq. (1.12))

1.1.3 Pose 4DOF

class Pose.**Pose4D**(*input_array=array([[0.], [0.], [0.], [0.]])*)

Bases: *Pose*

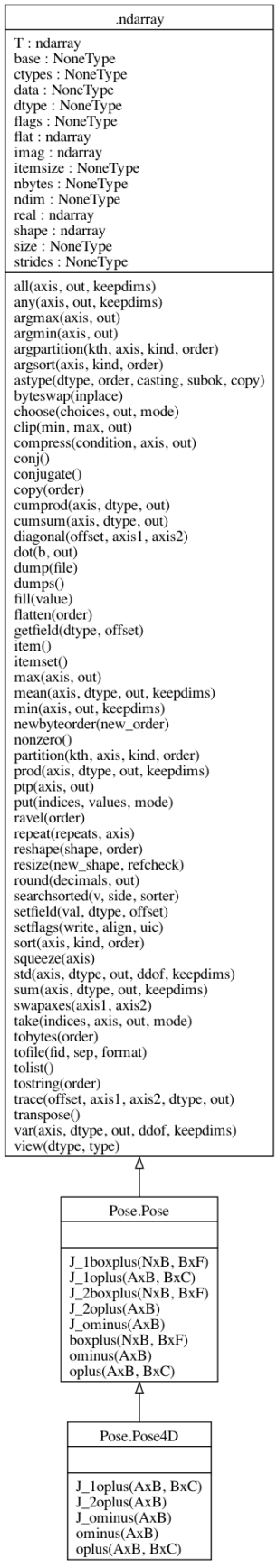
Definition of a robot pose in 4 DOF (x, y, yaw). The class inherits from a ndarray. This class extends the ndarray with the *oplus* and :math:`ominus operators and the corresponding Jacobians.

__init__(*input_array=array([[0.], [0.], [0.], [0.]])*)

oplus(*BxC*)

Given a Pose3D object *AxB* (the self object) and a Pose3D object *BxC*, it returns the Pose4D object *AxC*.

$$\begin{aligned} {}^{A}x_B &= \begin{bmatrix} {}^{A}x_B & {}^{A}y_B & {}^{A}z_B & {}^{A}\psi_B \end{bmatrix}^T \\ {}^{B}x_C &= \begin{bmatrix} {}^{B}x_C & {}^{B}y_C & {}^{B}z_C & {}^{B}\psi_C \end{bmatrix}^T \\ {}^{A}x_C &= {}^{A}x_B \oplus {}^{B}x_C = \begin{bmatrix} {}^{A}x_B + {}^{B}x_C \cos({}^{A}\psi_B) - {}^{B}y_C \sin({}^{A}\psi_B) \\ {}^{A}y_B + {}^{B}x_C \sin({}^{A}\psi_B) + {}^{B}y_C \cos({}^{A}\psi_B) \\ {}^{A}z_B + {}^{B}z_C \\ {}^{A}\psi_B + {}^{B}\psi_C \end{bmatrix} \end{aligned} \tag{1.13}$$


.ndarray
T : ndarray
base : NoneType
ctypes : NoneType
data : NoneType
dtype : NoneType
flags : NoneType
flat : ndarray
imag : ndarray
itemsize : NoneType
nbytes : NoneType
ndim : NoneType
real : ndarray
shape : ndarray
size : NoneType
strides : NoneType
all(axis, out, keepdims)
any(axis, out, keepdims)
argmax(axis, out)
argmin(axis, out)
argpartition(kth, axis, kind, order)
argsort(axis, kind, order)
astype(dtype, order, casting, subok, copy)
byteswap(inplace)
choose(choices, out, mode)
clip(min, max, out)
compress(condition, axis, out)
conj()
conjugate()
copy(order)
cumprod(axis, dtype, out)
cumsum(axis, dtype, out)
diagonal(offset, axis1, axis2)
dot(b, out)
dump(file)
dumps()
fill(value)
flatten(order)
getfield(dtype, offset)
item()
itemset()
max(axis, out)
mean(axis, dtype, out, keepdims)
min(axis, out, keepdims)
newbyteorder(new_order)
nonzero()
partition(kth, axis, kind, order)
prod(axis, dtype, out, keepdims)
ptp(axis, out)
put(indices, values, mode)
ravel(order)
repeat(repeats, axis)
reshape(shape, order)
resize(new_shape, refcheck)
round(decimals, out)
searchsorted(v, side, sorter)
setfield(val, dtype, offset)
setflags(write, align, uic)
sort(axis, kind, order)
squeeze(axis)
std(axis, dtype, out, ddof, keepdims)
sum(axis, dtype, out, keepdims)
swapaxes(axis1, axis2)
take(indices, axis, out, mode)
tobytes(order)
tofile(fid, sep, format)
tolist()
tostring(order)
trace(offset, axis1, axis2, dtype, out)
transpose()
var(axis, dtype, out, ddof, keepdims)
view(dtype, type)
Pose.Pose
J_1boxplus(NxB, BxF)
J_1oplus(AxB, BxC)
J_2boxplus(NxB, BxF)
J_2oplus(AxB)
J_ominus(AxB)
boxplus(NxB, BxF)
ominus(AxB)
oplus(AxB, BxC)
Pose.Pose4D
J_1oplus(AxB, BxC)
J_2oplus(AxB)
J_ominus(AxB)
ominus(AxB)
oplus(AxB, BxC)

Parameters
BxC – C-Frame pose expressed in B-Frame coordinates

Returns
C-Frame pose expressed in A-Frame coordinates

`J_1oplus`(*BxC*)

Jacobian of the pose compounding operation (eq. (1.13)) with respect to the first pose:

$$J_{1\oplus} = \frac{\partial {}^A x_B \oplus {}^B x_C}{\partial {}^A x_B} = \begin{bmatrix} 1 & 0 & 0 & -{}^B x_C \sin({}^A\psi_B) - {}^B y_C \cos({}^A\psi_B) \\ 0 & 1 & 0 & {}^B x_C \cos({}^A\psi_B) - {}^B y_C \sin({}^A\psi_B) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.14}$$

Parameters

- **AxB** – first pose
- **BxC** – 2nd pose

Returns
$J_{1\oplus}$ Jacobian of the pose compounding operation with respect to the first pose (eq. (1.14))

`J_2oplus`()

Jacobian of the pose compounding operation ((1.13)) with respect to the second pose:

$$J_{2\oplus} = \frac{\partial {}^A x_B \oplus {}^B x_C}{\partial {}^B x_C} = \begin{bmatrix} \cos({}^A\psi_B) & -\sin({}^A\psi_B) & 0 & 0 \\ \sin({}^A\psi_B) & \cos({}^A\psi_B) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{1.15}$$

Parameters
AxB – first pose

Returns
$J_{2\oplus}$ Jacobian of the pose compounding operation with respect to the second pose (eq. (1.15))

`ominus`()

Inverse pose compounding of the *AxB* pose (the self object):

$${}^B x_A = \ominus {}^A x_B = \begin{bmatrix} -{}^A x_B \cos({}^A\psi_B) - {}^A y_B \sin({}^A\psi_B) \\ {}^A x_B \sin({}^A\psi_B) - {}^A y_B \cos({}^A\psi_B) \\ -{}^A z_B \\ -{}^A\psi_B \end{bmatrix} \tag{1.16}$$

Parameters
AxB – B-Frame pose expressed in A-Frame coordinates

Returns
A-Frame pose expressed in B-Frame coordinates (eq. (1.16)

`J_ominus`()

Jacobian of the inverse pose compounding operation ((1.13)) with respect the pose *AxB* (the self object):

$$J_{\ominus} = \frac{\partial \ominus {}^A x_B}{\partial {}^A x_B} = \begin{bmatrix} -\cos({}^A\psi_B) & -\sin({}^A\psi_B) & 0 & {}^A x_B \sin({}^A\psi_B) - {}^A y_B \cos({}^A\psi_B) \\ \sin({}^A\psi_B) & -\cos({}^A\psi_B) & 0 & {}^A x_B \cos({}^A\psi_B) + {}^A y_B \sin({}^A\psi_B) \\ 0 & 0 & -1 & {}^A z_B \\ 0 & 0 & 0 & -1 \end{bmatrix} \tag{1.17}$$

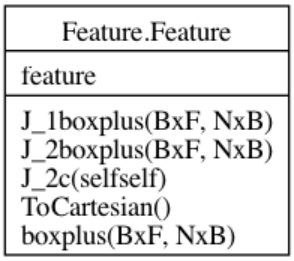
Parameters
AxB – B-Fram pose expressed in A-Frame coordinates

Returns

$J_{\ominus}$ Jacobian of the inverse pose compounding operation with respect to the pose (eq. (1.17))

1.2 Feature Representation

1.2.1 Feature



class Feature.**Feature**(*feature*)

Bases: object

This class implements the **interface of the pose-feature compounding operation**. This class provides the interface to implement the compounding operation between the robot pose (represented in the N-Frame) and the feature pose (represented in the B-Frame) obtaining the feature representation in the N-Frame. The class also provides the interface to implement the Jacobians of the pose-feature compounding operation.

__init__(*feature*)

boxplus(*NxB*)

Pose-Feature compounding operation:

$$^{N}x_{F} = ^{N}x_{B} \boxplus ^{B}x_{F} \tag{1.18}$$

which computes the pose of a feature in the N-Frame given the pose of the robot in the N-Frame and the pose of the feature in the B-Frame. **This is a pure virtual method that must be overwritten by the child class**.

Parameters

- **NxB** – Robot pose in the N-Frame ($^{N}x_{B}$)
- **BxF** – Feature pose in the B-Frame ($^{B}x_{F}$)

Returns

Feature pose in the N-Frame ($^{N}x_{F}$)

J_1boxplus(*NxB*)

Jacobian of the Pose-Feature compounding operation (eq. (1.18)) with respect to the first argument $^{N}x_{B}$.

$$J_{1\boxplus} = \frac{\partial ^{N}x_{B} \boxplus ^{B}x_{F}}{\partial ^{N}x_{B}}. \tag{1.19}$$

To be overriden by the child class.

Parameters

- **NxB** – Robot pose in the N-Frame ($^{N}x_{B}$)

- **BxF** – Feature pose in the B-Frame (${}^{B}x_{F}$)

Returns
Jacobian matrix $J_{1\boxplus}$

`J_2boxplus`(*NxB*)

Jacobian of the Pose-Feature compounding operation (eq. (1.18)) with respect to the second argument ${}^{B}x_{F}$.

$$J_{2\boxplus} = \frac{\partial^{N} x_{B} \boxplus^{B} x_{F}}{\partial^{B} x_{F}}. \tag{1.20}$$

To be overriden by the child class.

Parameters
NxB – Robot pose in the N-Frame (${}^{N}x_{B}$)

Returns
Jacobian matrix $J_{2\boxplus}$

`ToCartesian`()

Translates from its internal representation to the representation used for plotting. **To be overriden by the child class**.

Returns
Feature in Cartesian Coordinates

`J_2c`()

Jacobian of the ToCartesian method. Required for plotting non Cartesian features. **To be overriden by the child class**.

Returns
Jacobian of the transformation

1.2.2 Cartesian Feature

`class` Feature.`CartesianFeature`(*input_array*)

Bases: *Feature*, ndarray

Cartesian feature class. The class inherits from the *Feature* class providing an implementation of its interface for a Cartesian Feature, by implementing the $\boxplus$ operator as well as its Jacobians. The class also inherits from the ndarray numpy class allowing to be operated as a numpy ndarray.

`boxplus`(*NxB*)

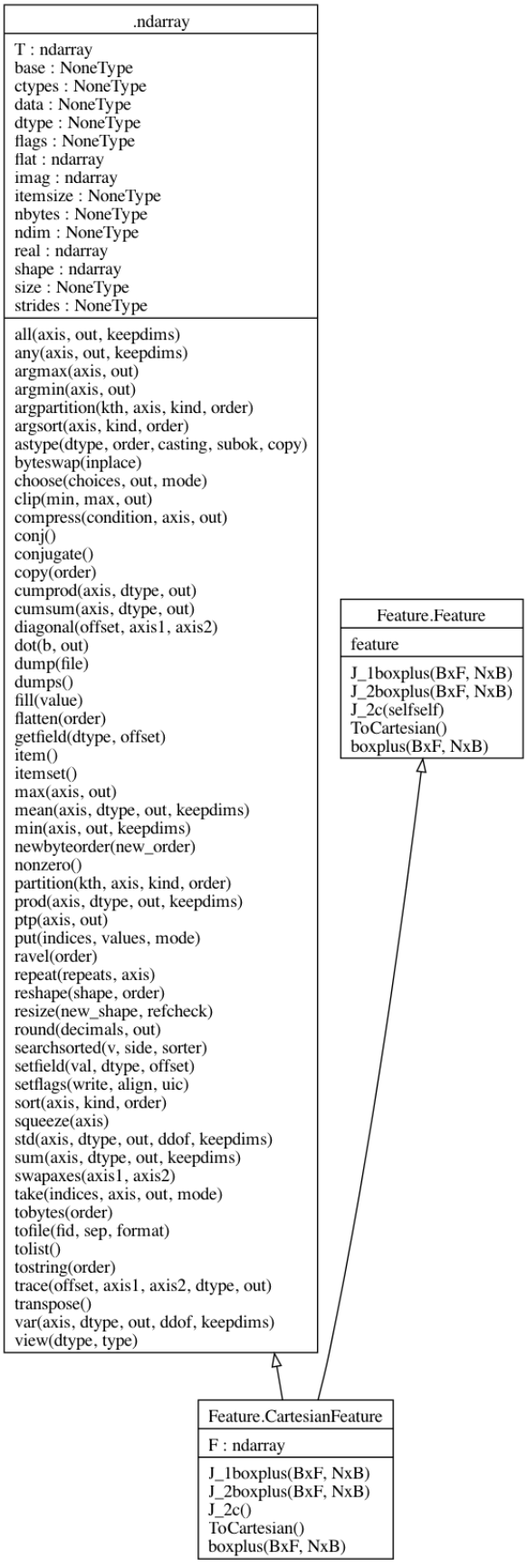
Pose-Cartesian Feature compounding operation:

$$\begin{aligned} F &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\ {}^{N}x_{F} &= {}^{N} x_{B} \boxplus^{B} x_{F} = F({}^{N}x_{B} \oplus^{B} x_{F}) \end{aligned} \tag{1.21}$$

which computes the Cartesian position of a feature in the N-Frame given the pose of the robot in the N-Frame and the Cartesian position of the feature in the B-Frame.

Parameters

- **NxB** – Robot pose in the N-Frame (${}^{N}x_{B}$)
- **BxF** – Cartesian feature pose in the B-Frame (${}^{B}x_{F}$)


.ndarray
T : ndarray
base : NoneType
ctypes : NoneType
data : NoneType
dtype : NoneType
flags : NoneType
flat : ndarray
imag : ndarray
itemsize : NoneType
nbytes : NoneType
ndim : NoneType
real : ndarray
shape : ndarray
size : NoneType
strides : NoneType
all(axis, out, keepdims)
any(axis, out, keepdims)
argmax(axis, out)
argmin(axis, out)
argpartition(kth, axis, kind, order)
argsort(axis, kind, order)
astype(dtype, order, casting, subok, copy)
byteswap(inplace)
choose(choices, out, mode)
clip(min, max, out)
compress(condition, axis, out)
conj()
conjugate()
copy(order)
cumprod(axis, dtype, out)
cumsum(axis, dtype, out)
diagonal(offset, axis1, axis2)
dot(b, out)
dump(file)
dumps()
fill(value)
flatten(order)
getfield(dtype, offset)
item()
itemset()
max(axis, out)
mean(axis, dtype, out, keepdims)
min(axis, out, keepdims)
newbyteorder(new_order)
nonzero()
partition(kth, axis, kind, order)
prod(axis, dtype, out, keepdims)
ptp(axis, out)
put(indices, values, mode)
ravel(order)
repeat(repeats, axis)
reshape(shape, order)
resize(new_shape, refcheck)
round(decimals, out)
searchsorted(v, side, sorter)
setfield(val, dtype, offset)
setflags(write, align, uic)
sort(axis, kind, order)
squeeze(axis)
std(axis, dtype, out, ddof, keepdims)
sum(axis, dtype, out, keepdims)
swapaxes(axis1, axis2)
take(indices, axis, out, mode)
tobytes(order)
tofile(fid, sep, format)
tolist()
tostring(order)
trace(offset, axis1, axis2, dtype, out)
transpose()
var(axis, dtype, out, ddof, keepdims)
view(dtype, type)
Feature.Feature
feature
J_1boxplus(BxF, NxB)
J_2boxplus(BxF, NxB)
J_2c(selfself)
ToCartesian()
boxplus(BxF, NxB)
Feature.CartesianFeature
F : ndarray
J_1boxplus(BxF, NxB)
J_2boxplus(BxF, NxB)
J_2c()
ToCartesian()
boxplus(BxF, NxB)

Returns
Feature pose in the N-Frame (${}^N x_F$)

`J_1boxplus`(*NxB*)

Jacobian of the Pose-Cartesian Feature compounding operation with respect to the robot pose:

$$J_{1\boxplus} = F J_{1\oplus} \tag{1.22}$$

Parameters

- **NxB** – robot pose represented in the N-Frame (${}^N x_B$)
- **BxF** – Cartesian feature pose represented in the B-Frame (${}^B x_F$)

Returns
Jacobian matrix $J_{boxplus}$ (eq. (1.22)) (eq. (1.22))

`J_2boxplus`(*NxB*)

Jacobian of the Pose-Cartesian Feature compounding operation with respect to the feature position:

$$J_{2\boxplus} = F J_{2oplus} \tag{1.23}$$

Parameters

- **NxB** – robot pose represented in the N-Frame (${}^N x_B$)
- **BxF** – Cartesian feature pose represented in the B-Frame (${}^B x_F$)

Returns
Jacobian matrix $J_{1\boxplus}$ (eq. (1.23))

`ToCartesian`()

Translates from its internal representation to the representation used for plotting.

Returns
Feature in Cartesian Coordinates

`J_2c`()

Jacobian of the ToCartesian method. Required for plotting non Cartesian features. Since the feature is in the Cartesian representation, the Jacobian is the identity matrix.

Returns
Jacobian of the transformation

1.2.3 Polar Feature

`class` Feature.`PolarFeature`(*input_array*, **args*)

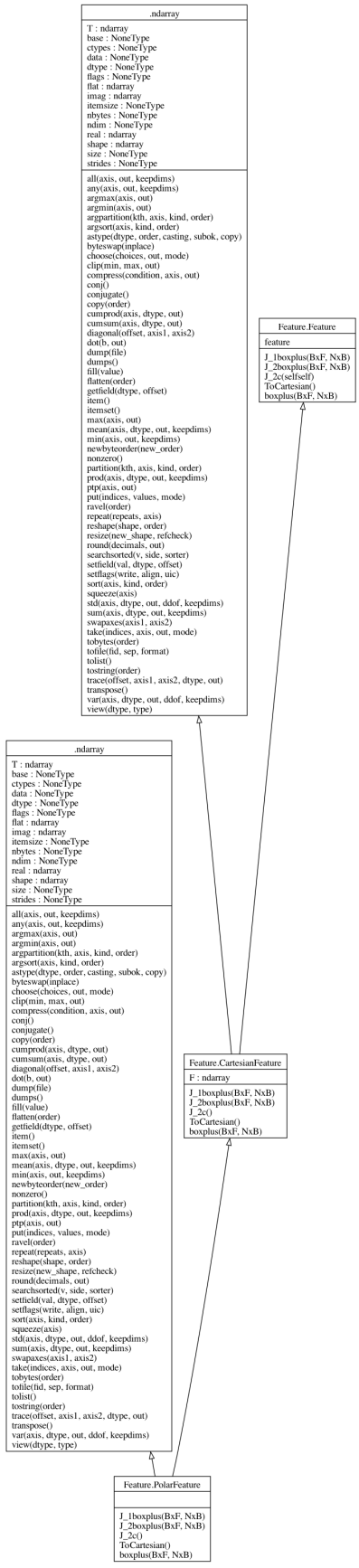
Bases: *CartesianFeature*, ndarray

Polar feature class. The class inherits from the *Feature* class providing an implementation of its interface for a Polar Feature, by implementing the $\boxplus$ operator as well as its Jacobians. The class also inherits from the ndarray numpy class allowing to be operated as a numpy ndarray.

`boxplus`(*NxB*)

Pose-Polar Feature compounding operation:

$$\begin{aligned} F &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\ {}^N x_F &= {}^N x_B \boxplus^B x_F = F({}^N x_B \oplus^B x_F) \end{aligned} \tag{1.24}$$


.ndarray
T : ndarray
base : NoneType
ctypes : NoneType
data : NoneType
dtype : NoneType
flags : NoneType
flat : ndarray
imag : ndarray
itemsize : NoneType
nbytes : NoneType
ndim : NoneType
real : ndarray
shape : ndarray
size : NoneType
strides : NoneType
all(axis, out, keepdims)
any(axis, out, keepdims)
argmax(axis, out)
argmin(axis, out)
argpartition(kth, axis, kind, order)
argsort(axis, kind, order)
astype(dtype, order, casting, subok, copy)
byteswap(inplace)
choose(choices, out, mode)
clip(min, max, out)
compress(condition, axis, out)
conj()
conjugate()
copy(order)
cumprod(axis, dtype, out)
cumsum(axis, dtype, out)
diagonal(offset, axis1, axis2)
dot(b, out)
dump(file)
dumps()
fill(value)
flatten(order)
getfield(dtype, offset)
item()
itemset()
max(axis, out)
mean(axis, dtype, out, keepdims)
min(axis, out, keepdims)
newbyteorder(new_order)
nonzero()
partition(kth, axis, kind, order)
prod(axis, dtype, out, keepdims)
ptp(axis, out)
put(indices, values, mode)
ravel(order)
repeat(repeats, axis)
reshape(shape, order)
resize(new_shape, refcheck)
round(decimals, out)
searchsorted(v, side, sorter)
setfield(val, dtype, offset)
setflags(write, align, uic)
sort(axis, kind, order)
squeeze(axis)
std(axis, dtype, out, ddof, keepdims)
sum(axis, dtype, out, keepdims)
swapaxes(axis1, axis2)
take(indices, axis, out, mode)
tobytes(order)
tofile(fid, sep, format)
tolist()
tostring(order)
trace(offset, axis1, axis2, dtype, out)
transpose()
var(axis, dtype, out, ddof, keepdims)
view(dtype, type)
Feature.Feature
feature
J_1boxplus(BxF, NxB)
J_2boxplus(BxF, NxB)
J_2c(selfself)
ToCartesian()
boxplus(BxF, NxB)
.ndarray
T : ndarray
base : NoneType
ctypes : NoneType
data : NoneType
dtype : NoneType
flags : NoneType
flat : ndarray
imag : ndarray
itemsize : NoneType
nbytes : NoneType
ndim : NoneType
real : ndarray
shape : ndarray
size : NoneType
strides : NoneType
all(axis, out, keepdims)
any(axis, out, keepdims)
argmax(axis, out)
argmin(axis, out)
argpartition(kth, axis, kind, order)
argsort(axis, kind, order)
astype(dtype, order, casting, subok, copy)
byteswap(inplace)
choose(choices, out, mode)
clip(min, max, out)
compress(condition, axis, out)
conj()
conjugate()
copy(order)
cumprod(axis, dtype, out)
cumsum(axis, dtype, out)
diagonal(offset, axis1, axis2)
dot(b, out)
dump(file)
dumps()
fill(value)
flatten(order)
getfield(dtype, offset)
item()
itemset()
max(axis, out)
mean(axis, dtype, out, keepdims)
min(axis, out, keepdims)
newbyteorder(new_order)
nonzero()
partition(kth, axis, kind, order)
prod(axis, dtype, out, keepdims)
ptp(axis, out)
put(indices, values, mode)
ravel(order)
repeat(repeats, axis)
reshape(shape, order)
resize(new_shape, refcheck)
round(decimals, out)
searchsorted(v, side, sorter)
setfield(val, dtype, offset)
setflags(write, align, uic)
sort(axis, kind, order)
squeeze(axis)
std(axis, dtype, out, ddof, keepdims)
sum(axis, dtype, out, keepdims)
swapaxes(axis1, axis2)
take(indices, axis, out, mode)
tobytes(order)
tofile(fid, sep, format)
tolist()
tostring(order)
trace(offset, axis1, axis2, dtype, out)
transpose()
var(axis, dtype, out, ddof, keepdims)
view(dtype, type)
Feature.CartesianFeature
F : ndarray
J_1boxplus(BxF, NxB)
J_2boxplus(BxF, NxB)
J_2c()
ToCartesian()
boxplus(BxF, NxB)
Feature.PolarFeature
J_1boxplus(BxF, NxB)
J_2boxplus(BxF, NxB)
J_2c()
ToCartesian()
boxplus(BxF, NxB)

which computes the Polar pose of a feature in the N-Frame given the pose of the robot in the N-Frame and the Polar pose of the feature in the B-Frame.

Parameters

- **NxB** – Robot pose in the N-Frame (${}^{N}x_{B}$)
- **BxF** – Polar feature pose in the B-Frame (${}^{B}x_{F}$)

Returns

Polar feature pose in the N-Frame (${}^{N}x_{F}$)

J_1boxplus(*NxB*)

Jacobian of the Pose-Polar Feature compounding operation with respect to the robot pose:

$$J_{1\boxplus} = FJ_{1\oplus} \tag{1.25}$$

Parameters

- **NxB** – robot pose represented in the N-Frame (${}^{N}x_{B}$)
- **BxF** – Polar eature pose represented in the B-Frame (${}^{B}x_{F}$)

Returns

Jacobian matrix $J_{boxplus}$ (eq. (1.25))

J_2boxplus(*NxB*)

Jacobian of the Pose-Polar Feature compounding operation with respect to the feature pose:

$$J_{2\boxplus} = FJ_{2oplus} \tag{1.26}$$

Parameters

- **NxB** – Robot pose represented in the N-Frame (${}^{N}x_{B}$)
- **BxF** – Polar feature pose represented in the B-Frame (${}^{B}x_{F}$)

Returns

Jacobian matrix $J_{1\boxplus}$ (eq. (1.26))

ToCartesian()

Translates from its internal representation to the representation used for plotting.

Returns

Feature in Cartesian Coordinates

J_2c()

Jacobian of the ToCartesian method. Required for plotting non Cartesian features. Since the FEature is in the Polar representation, the Jacobian is the Jacobian of the polar to cartesian conversion.

Returns

Jacobian of the transformation

1.3 Coordinate Conversion Functions

This functions are used to convert between different coordinate systems (e.g. from cartesian to polar, from polar to cartesian, etc.). The functions are implemented in a way that they can be used with numpy arrays. or each coordinate conversion its Jacobian is also implemented. This functions are required to convert from the observation space (the coordinates in which the features are observed) to the storage space (the coordinates in which the features are stored in the map). For instance, if the features are observed in polar coordinates, but stored in cartesian coordinates, the conversion from polar to cartesian is required.

1.3.1 Polar To Cartesian

`conversions.p2c(`p`)`

Converts from a 2D Polar coordinate to its corresponding 2D Cartesian coordinate:

$$\begin{aligned} p &= \begin{bmatrix} \rho \\ \theta \end{bmatrix} \\ c &= p2c\left(\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \rho\cos(\theta) \\ \rho\sin(\theta) \end{bmatrix}\right) \end{aligned} \tag{1.27}$$

Parameters
p – point in polar coordinates

Returns
point in cartesian coordinates

`conversions.J_p2c(`p`)`

Jacobian of the 2D Polar to cartesian conversion:

$$J_{p2c} = \begin{bmatrix} \frac{\partial x}{\partial \rho} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial \rho} & \frac{\partial y}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\rho\sin(\theta) \\ \sin(\theta) & \rho\cos(\theta) \end{bmatrix} \tag{1.28}$$

Parameters
p – linearization point in polar coordinates

Returns
Jacobian matrix J_{p2c} (eq. (1.28))

1.3.2 Cartesian To Polar

`conversions.c2p(`c`)`

2D Cartesian to polar conversion:

$$\begin{aligned} c &= \begin{bmatrix} x \\ y \end{bmatrix} \\ p &= c2p\left(\begin{bmatrix} \rho \\ \theta \end{bmatrix} = \begin{bmatrix} \sqrt{x^2+y^2} \\ atan2(y,x) \end{bmatrix}\right) \end{aligned} \tag{1.29}$$

Parameters
c – point in cartesian coordinates

Returns
point in polar coordinates

conversions.**J_c2p**(c)

Jacobian of the 2D Cartesian to polar conversion:

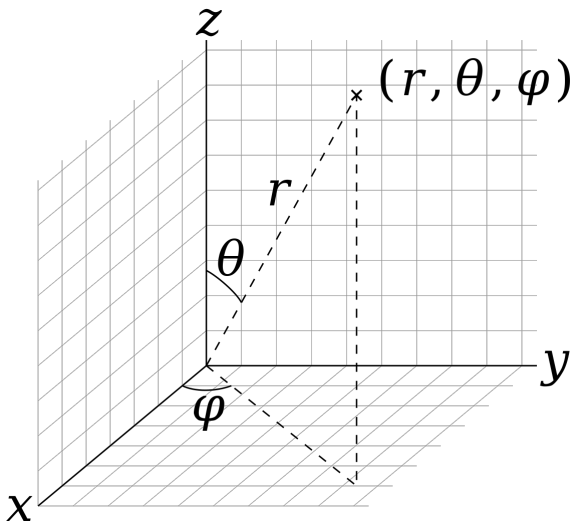
$$J_{c2p} = \begin{bmatrix} \frac{\partial \rho}{\partial x} & \frac{\partial \rho}{\partial y} \\ \frac{\partial \theta}{\partial x} & \frac{\partial \theta}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & \frac{y}{\sqrt{x^2+y^2}} \\ -\frac{y}{x^2+y^2} & \frac{x}{x^2+y^2} \end{bmatrix} \tag{1.30}$$

Parameters
c – point in cartesian coordinates

Returns
Jacobian matrix J_{c2p} (eq. (1.30))

1.3.3 Spherical To Cartesian

conversions.**s2c**(s)



3D Spherical to cartesian conversion:

$$\begin{aligned} s &= \begin{bmatrix} \rho \\ \theta \\ \varphi \end{bmatrix} \\ c &= s2c\left(\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \rho \sin(\theta)\cos(\varphi) \\ \rho \sin(\theta)\sin(\varphi) \\ \rho \cos(\theta) \end{bmatrix} \right) \end{aligned} \tag{1.31}$$

Parameters
s – point in spherical coordinates

Returns
point in cartesian coordinates

conversions.**J_s2c**(s)

Jacobian of the 3D Spherical to cartesian conversion:

$$J_{s2c} = \begin{bmatrix} \frac{\partial x}{\partial \rho} & \frac{\partial x}{\partial \theta} & \frac{\partial x}{\partial \varphi} \\ \frac{\partial y}{\partial \rho} & \frac{\partial y}{\partial \theta} & \frac{\partial y}{\partial \varphi} \\ \frac{\partial z}{\partial \rho} & \frac{\partial z}{\partial \theta} & \frac{\partial z}{\partial \varphi} \end{bmatrix} = \begin{bmatrix} \sin(\theta)\cos(\varphi) & \rho\cos(\theta)\cos(\varphi) & -\rho\sin(\theta)\sin(\varphi) \\ \sin(\theta)\sin(\varphi) & \rho\cos(\theta)\sin(\varphi) & \rho\sin(\theta)\cos(\varphi) \\ \cos(\theta) & -\rho\sin(\theta) & 0 \end{bmatrix} \tag{1.32}$$

Parameters
s – linearization point in spherical coordinates

Returns
Jacobian matrix J_{s2c} (eq. (1.32))

1.3.4 Cartesian To Spherical

`conversions.c2s(c)`

3D Cartesian to spherical conversion:

$$
\begin{aligned}
c &= \begin{bmatrix} x \\ y \\ z \end{bmatrix} \\
s = c2s\left(\begin{bmatrix} \rho \\ \theta \\ \varphi \end{bmatrix} &= \begin{bmatrix} \sqrt{x^2+y^2+z^2} \\ atan2(\sqrt{x^2+y^2}, z) \\ atan2(y, x) \end{bmatrix}\right)
\end{aligned}
\tag{1.33}
$$

Parameters
c – point in cartesian coordinates

Returns
point in spherical coordinates

`conversions.J_c2s(c)`

Jacobian of the 3D Cartesian to spherical conversion:

$$
J_{c2s} = \begin{bmatrix} \frac{\partial\rho}{\partial x} & \frac{\partial\rho}{\partial y} & \frac{\partial\rho}{\partial z} \\ \frac{\partial\theta}{\partial x} & \frac{\partial\theta}{\partial y} & \frac{\partial\theta}{\partial z} \\ \frac{\partial\varphi}{\partial x} & \frac{\partial\varphi}{\partial y} & \frac{\partial\varphi}{\partial z} \end{bmatrix} = \begin{bmatrix} \frac{x}{\sqrt{x^2+y^2+z^2}} & \frac{y}{\sqrt{x^2+y^2+z^2}} & \frac{z}{\sqrt{x^2+y^2+z^2}} \\ \frac{y}{x^2+y^2} & \frac{x}{x^2+y^2} & 0 \\ \frac{-xz}{(x^2+y^2)\sqrt{x^2+y^2}} & \frac{-yz}{(x^2+y^2)\sqrt{x^2+y^2}} & \frac{\sqrt{x^2+y^2}}{x^2+y^2} \end{bmatrix} \tag{1.34}
$$

Parameters
c – linearization point in cartesian coordinates

Returns
Jacobian matrix J_{c2s} (eq. (1.34))

1.3.5 Identity Conversion

`conversions.v2v(v)`

Identity transformation. Returns the same vector.

Parameters
v – input vector

Returns
output vector

`conversions.J_v2v(v)`

Jacobian of the identity transformation. Returns the identity matrix of the same dimensionality as the input vector.

Parameters
v – input vector

Returns
Identity matrix of the same dimensionality as the input vector.

1.4 Robot Simulation

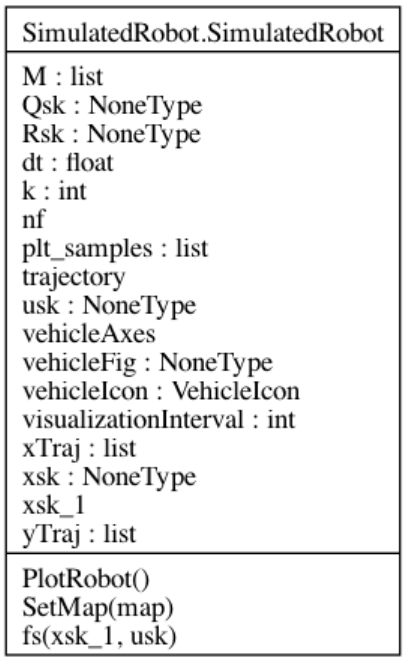


Fig. 1: SimulatedRobot Class Diagram.

class SimulatedRobot.**SimulatedRobot**(*xs0*, *map=[]*, **args*)

Bases: object

This is the base class to simulate a robot. There are two operative frames: the world N-Frame (North East Down oriented) and the robot body frame body B-Frame. Each robot has a motion model and a measurement model. The motion model is used to simulate the robot motion and the measurement model is used to simulate the robot measurements.

All Robot simulation classes must derive from this class .

dt = 0.1

class attribute containing sample time of the simulation

__init__(*xs0*, *map=[]*, **args*)

Parameters

- **xs0** – initial simulated robot state x_{s_0} used to initialize the the motion model
- **map** – feature map of the environment $M = [^N x_{F_1}^T, ..., ^N x_{F_{nf}}^T]^T$

Constructor. First, it initializes the robot simulation defining the following attributes:

- **k** : time step
- **Qsk** : **To be defined in the derived classes**. Object attribute containing Covariance of the simulation motion model noise

- **usk** : **To be defined in the derived classes**. Object attribute contining the simulated input to the motion model
- **xsk** : **To be defined in the derived classes**. Object attribute contining the current simulated robot state
- **zsk** : **To be defined in the derived classes**. Object attribute contining the current simulated robot measurement
- **Rsk** : **To be defined in the derived classes**. Object attribute contining the observation noise covariance matrix
- **xsk** : current pose is the initial state
- **xsk_1** : previouse state is the initial robot state
- **M** : position of the features in the N-Frame
- **nf** : number of features

Then, the robot animation is initialized defining the following attributes:

- **vehicleIcon** : Path file of the image of the robot to be used in the animation
- **vehicleFig** : Figure of the robot to be used in the animation
- **vehicleAxes** : Axes of the robot to be used in the animation
- **xTraj** : list containing the x coordinates of the robot trajectory
- **yTraj** : list containing the y coordinates of the robot trajectory
- **visualizationInterval** : time-steps interval between two consecutive frames of the animation

`PlotRobot()`

Updates the plot of the robot at the current pose

`fs`(*xsk_1*, *usk*)

Motion model used to simulate the robot motion. Computes the current robot state x_k given the previous robot state x_{k-1} and the input u_k. It also updates the object attributes xsk, xsk_1 and usk to be made them available for plotting purposes. *To be overriden in child class*.

Parameters

- **xsk_1** – previous robot state x_{k-1}
- **usk** – model input u_{s_k}

Returns

current robot state x_k

`SetMap`(*map*)

Initializes the map of the environment.

1.4.1 3 DOF Diferential Drive Robot Simulation

class DifferentialDriveSimulatedRobot.**DifferentialDriveSimulatedRobot**(*xs0*, *map=[]*, **args*)

Bases: *SimulatedRobot*

This class implements a simulated differential drive robot. It inherits from the `SimulatedRobot` class and overrides some of its methods to define the differential drive robot motion model.

__init__(*xs0*, *map=[]*, **args*)

Parameters

- **xs0** – initial simulated robot state $\mathbf{x_{s_0}} = [{}^N x_{s_0}\ {}^N y_{s_0}\ {}^N \psi_{s_0}]^T$ used to initialize the motion model
- **map** – feature map of the environment $M = [{}^N x_{F_1}, ..., {}^N x_{F_{nf}}]$

Initializes the simulated differential drive robot. Overrides some of the object attributes of the parent class `SimulatedRobot` to define the differential drive robot motion model:

- **Qsk** : Object attribute containing Covariance of the simulation motion model noise.

$$Q_k = \begin{bmatrix} \sigma_u^2 & 0 & 0 \\ 0 & \sigma_v^2 & 0 \\ 0 & 0 & \sigma_r^2 \end{bmatrix} \tag{1.35}$$

- **usk** : Object attribute containing the simulated input to the motion model containing the forward velocity u_k and the angular velocity r_k

$$\mathbf{u_k} = \begin{bmatrix} u_k & r_k \end{bmatrix}^{\mathbf{T}} \tag{1.36}$$

- **xsk** : Object attribute containing the current simulated robot state

$$x_k = \begin{bmatrix} {}^N x_k & {}^N y_k & {}^N \theta_k & {}^B u_k & {}^B v_k & {}^B r_k \end{bmatrix}^T \tag{1.37}$$

where ${}^N x_k$, ${}^N y_k$ and ${}^N \theta_k$ are the robot position and orientation in the world N-Frame, and ${}^B u_k$, ${}^B v_k$ and ${}^B r_k$ are the robot linear and angular velocities in the robot B-Frame.

- **zsk** : Object attribute containing $z_{s_k} = [n_L\ n_R]^T$ observation vector containing number of pulses read from the left and right wheel encoders.
- **Rsk** : Object attribute containing $R_{s_k} = diag(\sigma_L^2, \sigma_R^2)$ covariance matrix of the noise of the read pulses`.
- **wheelBase** : Object attribute containing the distance between the wheels of the robot ($w = 0.5$ m)
- **wheelRadius** : Object attribute containing the radius of the wheels of the robot ($R = 0.1$ m)
- **pulses_x_wheelTurn** : Object attribute containing the number of pulses per wheel turn ($pulseXwheelTurn = 1024$ pulses)
- **Polar2D_max_range** : Object attribute containing the maximum Polar2D range ($Polar2D_max_range = 50$ m) at which the robot can detect features.
- **Polar2D_feature_reading_frequency** : Object attribute containing the frequency of Polar2D feature readings (50 tics -sample times-)
- **Rfp** : Object attribute containing the covariance of the simulated Polar2D feature noise ($R_{fp} = diag(\sigma_\rho^2, \sigma_\phi^2)$)

Check the parent class `prpy.SimulatedRobot` to know the rest of the object attributes.

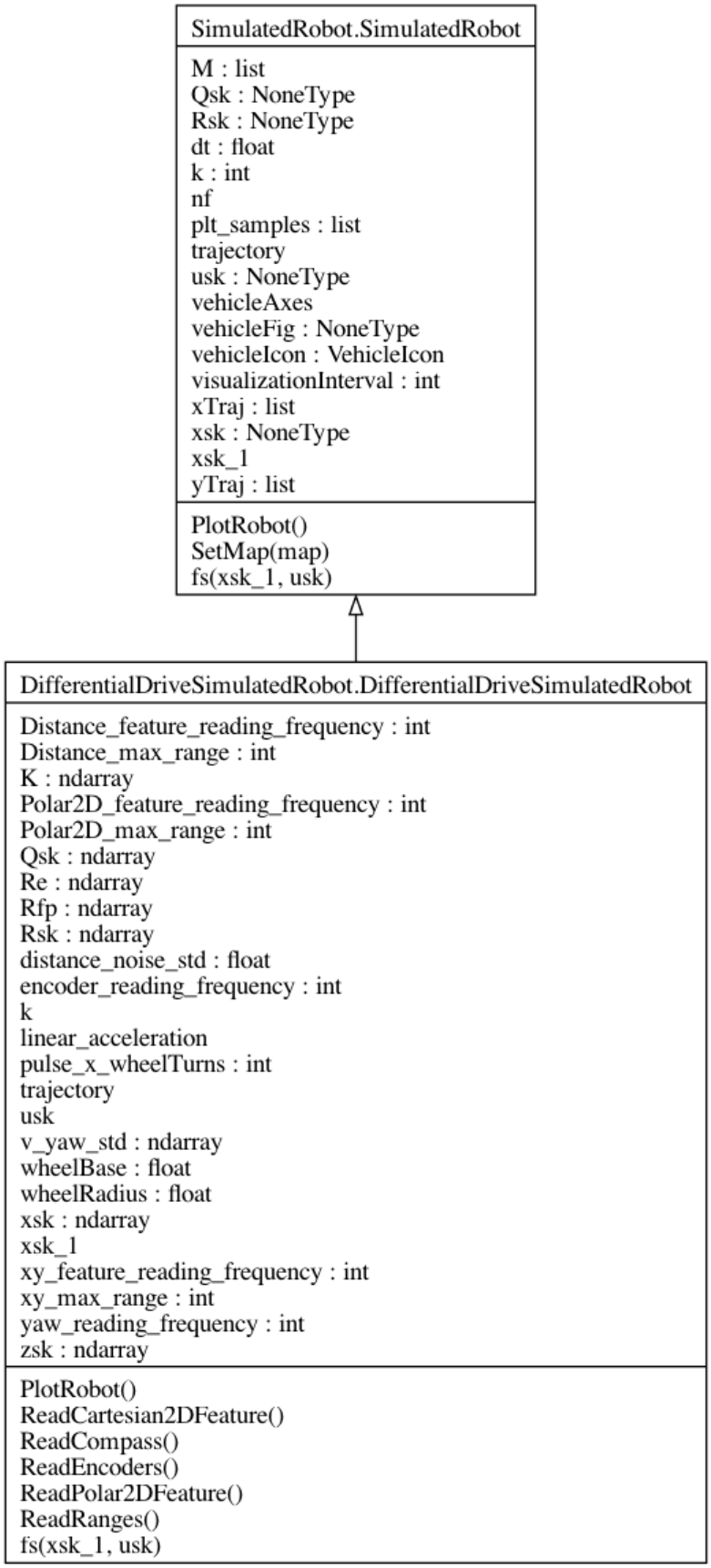


Fig. 2: DifferentialDriveSimulatedRobot Class Diagram.

fs(*xsk_1*, *usk*)

Motion model used to simulate the robot motion. Computes the current robot state x_k given the previous robot state x_{k-1} and the input u_k:

$$\begin{aligned}
\eta_{s_{k-1}} &= \begin{bmatrix} x_{s_{k-1}} & y_{s_{k-1}} & \theta_{s_{k-1}} \end{bmatrix}^T \\
\nu_{s_{k-1}} &= \begin{bmatrix} u_{s_{k-1}} & v_{s_{k-1}} & r_{s_{k-1}} \end{bmatrix}^T \\
x_{s_{k-1}} &= \begin{bmatrix} \eta_{s_{k-1}}^T & \nu_{s_{k-1}}^T \end{bmatrix}^T \\
u_{s_k} &= \nu_d = \begin{bmatrix} u_d & r_d \end{bmatrix}^T \\
w_{s_k} &= \dot{\nu}_{s_k} \\
x_{s_k} &= f_s(x_{s_{k-1}}, u_{s_k}, w_{s_k}) \\
&= \begin{bmatrix} \eta_{s_{k-1}} \oplus (\nu_{s_{k-1}} \Delta t + \frac{1}{2} w_{s_k} \Delta t^2) \\ \nu_{s_{k-1}} + K(\nu_d - \nu_{s_{k-1}}) + w_{s_k} \Delta t \end{bmatrix} \quad ; \quad K = diag(k_1, k_2, k_3) \quad k_i > 0
\end{aligned} \tag{1.38}$$

Where $\eta_{s_{k-1}}$ is the previous 3 DOF robot pose (x,y,yaw) and $\nu_{s_{k-1}}$ is the previous robot velocity (velocity in the direction of x and y B-Frame axis of the robot and the angular velocity). u_{s_k} is the input to the motion model contaning the desired robot velocity in the x direction (u_d) and the desired angular velocity around the z axis (r_d). w_{s_k} is the motion model noise representing an acceleration perturbation in the robot axis. The w_{s_k} acceleration is the responsible for the slight velocity variation in the simulated robot motion. K is a diagonal matrix containing the gains used to drive the simulated velocity towards the desired input velocity.

Finally, the class updates the object attributes xsk, xsk_1 and usk to made them available for plotting purposes.

To be completed by the student.

Parameters

- **xsk_1** – previous robot state $x_{s_{k-1}} = \begin{bmatrix} \eta_{s_{k-1}}^T & \nu_{s_{k-1}}^T \end{bmatrix}^T$
- **usk** – model input $u_{s_k} = \nu_d = \begin{bmatrix} u_d & r_d \end{bmatrix}^T$

Returns
current robot state x_{s_k}

ReadWheelAngularVelocity()

ReadEncoders()

Simulates the robot measurements of the left and right wheel encoders.

To be completed by the student.

Return zsk,Rsk
$zk = [\Delta n_L \ \Delta n_R]^T$ observation vector containing number of pulses read from the left and right wheel encoders during the last differential motion. $R_{s_k} = diag(\sigma_L^2, \sigma_R^2)$ covariance matrix of the read pulses.

ReadCompass()

Simulates the compass reading of the robot.

Returns
yaw and the covariance of its noise *R_yaw*

ReadCartesianFeature()

Simulates the reading of 2D cartesian features. The features are placed in the map in cartesian coordinates.

Returns

zsk: [[x1 y1],...,[xn yn]]
Cartesian position of the feature observations.

Rsk: block_diag(R_1,...,R_n), where R_i=[[r_xx r_xy],[r_xy r_yy]] is the
2x2 i-th feature observation covariance. Covariance of the Cartesian feature observations. Note the features are uncorrelated among them.

`ReadPolarFeature()`

Simulates the reading of 2D Polar features. The features are placed in the map in cartesian coordinates.

Returns

zsk: [[x1 y1 z1],...,[xn yn zn]]
Cartesian position of the feature observations.

Rsk: block_diag(R_1,...,R_n), where R_i=[[r_xx r_xy r_xz],[r_xy r_yy r_yz],[r_xz r_yz r_yy]] is the
2x2 i-th feature observation covariance. Covariance of the Polar feature observations. Note the features are uncorrelated among them.

`ReadRanges()`

Simulates the reading of distance towards 2D Cartessian features. Returns a vector of distances towards the features within the maximum range `Distance_max_range`. The functions works at a frequency of `Distance_feature_reading_frequency`.

Returns
vector of distances towards the features.

`PlotRobot()`

Updates the plot of the robot at the current pose

1.4.2 4 DOF AUV Robot Simulation

`class` AUV4DOFSimulatedRobot.**AUV4DOFSimulatedRobot**(*xs0*, *map=[]*, **args*)

Bases: *SimulatedRobot*

This class simulates an AUV equipped with the following sensors: Gyro, Compass, DVL, Depth, direct USBL and inverted USBL.

`dt = 0.1`

class attribute containing sample time of the simulation

`__init__`(*xs0*, *map=[]*, **args*)

Constructor.

Parameters

- **xs0** – initial simulated robot pose $x_{s_k} = [x_{s_k}, y_{s_k}, z_{s_k}, \psi_{s_k}]$ in the N-Frame
- **map** – map of the environment

`PlotRobot()`

Updates the plot of the robot at the current pose

`fs`(*xsk_1*, *usk*)

Motion model used to simulate the robot motion. Computes the current robot state x_k given the previous

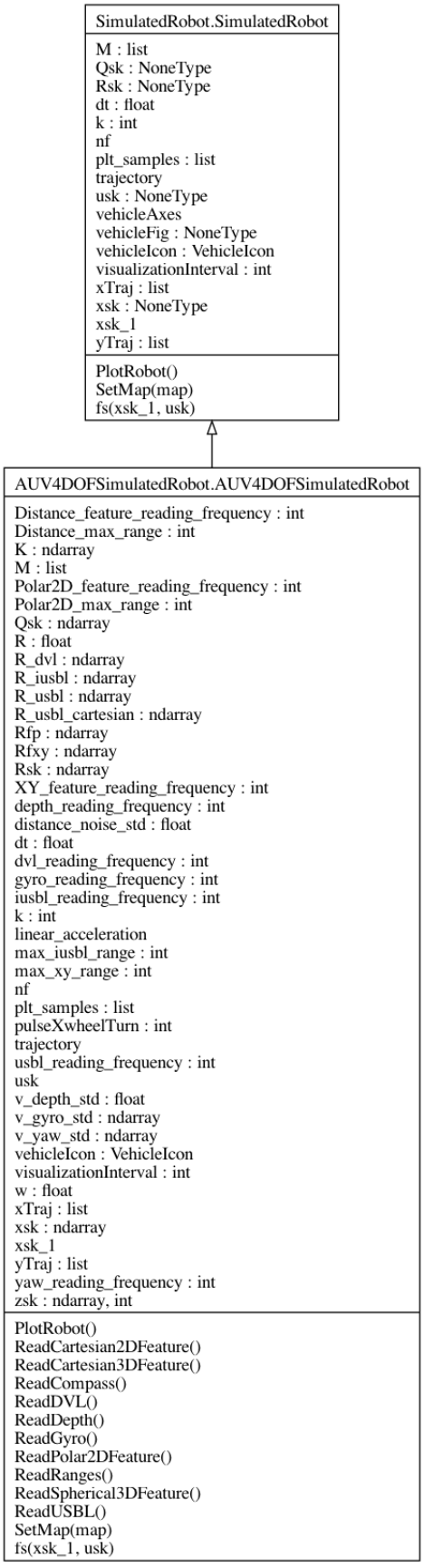


Fig. 3: AUV4DOFSimulatedRobot Class Diagram.

robot state x_{k-1} and the input u_k:

$$\begin{aligned}
\eta_{s_{k-1}} &= \begin{bmatrix} x_{s_{k-1}} & y_{s_{k-1}} & z_{s_{k-1}} & \psi_{s_{k-1}} \end{bmatrix}^T \\
\nu_{s_{k-1}} &= \begin{bmatrix} u_{s_{k-1}} & v_{s_{k-1}} & w_{s_{k-1}} & r_{s_{k-1}} \end{bmatrix}^T \\
x_{s_{k-1}} &= \begin{bmatrix} \eta_{s_{k-1}}^T & \nu_{s_{k-1}}^T \end{bmatrix}^T \\
u_{s_k} &= \nu_d = \begin{bmatrix} u_d & v_d & w_d & r_d \end{bmatrix}^T \\
w_{s_k} &= \dot{\nu}_{s_k} \\
x_{s_k} &= f_s(x_{s_{k-1}}, u_{s_k}, w_{s_k}) \\
&= \begin{bmatrix} \eta_{s_{k-1}} \oplus (\nu_{s_{k-1}} \Delta t + \frac{1}{2} w_{s_k}) \\ \nu_{s_{k-1}} + K(\nu_d - \nu_{s_{k-1}}) + w_{s_k} \Delta t \end{bmatrix} \quad ; \quad K = diag(k_1, k_2, k_3, k_4) \quad k_i > 0
\end{aligned} \tag{1.39}$$

Where $\eta_{s_{k-1}}$ is the previous 3 DOF robot pose (x,y,yaw) and $\nu_{s_{k-1}}$ is the previous robot velocity (velocity in the direction of x and y B-Frame axis of the robot and the angular velocity). u_{s_k} is the input to the motion model contaning the desired robot velocity in the x direction (u_d) and the desired angular velocity around the z axis (r_d). w_{s_k} is the motion model noise representing an acceleration perturbation in the robot axis. The w_{s_k} acceleration is the responsible for the slight velocity variation in the simulated robot motion. K is a diagonal matrix containing the gains used to drive the simulated velocity towards the desired input velocity.

Finally, the class updates the object attributes xsk, xsk_1 and usk to made them available for plotting purposes.

To be completed by the student.

Parameters

- **xsk_1** – previous robot state $x_{s_{k-1}} = \begin{bmatrix} \eta_{s_{k-1}}^T & \nu_{s_{k-1}}^T \end{bmatrix}^T$
- **usk** – model input $u_{s_k} = \nu_d = \begin{bmatrix} u_d & r_d \end{bmatrix}^T$

Returns
current robot state x_{s_k}

`SetMap`(*map*)

Initializes the map of the environment.

`ReadGyro`()

Simulates the gyro reading of the robot.

Returns
angular velocity [r] and the covariance of the noise [R_r]

`ReadCompass`()

Simulates the compass reading of the robot.

Returns
yaw and the covariance of its noise *R_yaw*

`ReadDVL`()

Simulates the DVL reading of the robot.

Returns
linear velocity [u v w] and the covariance of the noise [R_lin_vel]

`ReadDepth`()

Simulates the depth reading of the robot.

Returns
depth and the covariance of the noise [R_depth]

`ReadUSBL()`

Simulates the USBL reading of the robot. Assumes that the USBL transceiver is placed at the origin of the N_Frame and the transceiver is placed at the origin of the robot's B-Frame.

Returns
zsk: cartesian position [x y z] Rsk: Covariance of the noise [R_usbl_cartesian]

`ReadSphericalFeature()`

Simulates the reading of Spherical features. The features are placed in the map in cartesian coordinates.

Returns

zsk: [[x1 y1 z1],...,[xn yn zn]]
Cartesian position of the feature observations.

Rsk: block_diag(R_1,...,R_n), where R_i=[[r_xx r_xy r_xz],[r_xy r_yy r_yz],[r_xz r_yz r_yy]] is the
2x2 i-th feature observation covariance. Covariance of the Polar feature observations. Note the features are uncorrelated among them.

`ReadCartesianFeature()`

Simulates the reading of 3D cartesian features. The features are placed in the map in cartesian coordinates.

Returns

zsk: [[x1 y1 z1],...,[xn yn zn]]
Cartesian position of the feature observations.

Rsk: block_diag(R_1,...,R_n), where R_i=[[r_xx r_xy],[r_xy r_yy]] is the
2x2 i-th feature observation covariance. Covariance of the Cartesian feature observations. Note the features are uncorrelated among them.

`ReadPolarFeature()`

Simulates the reading of 2D Polar features. The features are placed in the map in cartesian coordinates.

Returns

zsk: [[x1 y1 z1],...,[xn yn zn]]
Cartesian position of the feature observations.

Rsk: block_diag(R_1,...,R_n), where R_i=[[r_xx r_xy r_xz],[r_xy r_yy r_yz],[r_xz r_yz r_yy]] is the
2x2 i-th feature observation covariance. Covariance of the Polar feature observations. Note the features are uncorrelated among them.

`ReadRanges()`

Simulates the reading of distance towards 2D Cartessian features. Returns a vector of distances towards the features within the maximum range `Distance_max_range`. The functions works at a frequency of `Distance_feature_reading_frequency`.

Returns
vector of distances towards the features.

`ReadCartesianObservedSphericalFeature()`

Simulates the reading of 3D cartesian features. The features are placed in the map in cartesian coordinates.

Returns

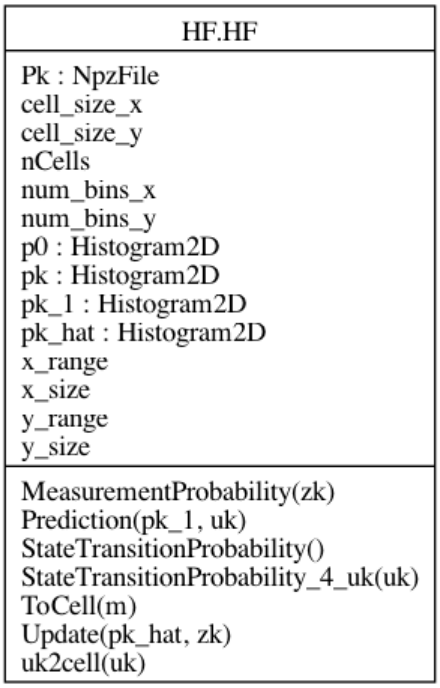
zsk: [[x1 y1 z1],...,[xn yn zn]]
Cartesian position of the feature observations.

Rsk: block_diag(R_1,...,R_n), where R_i=[[r_xx r_xy r_xz],[r_xy r_yy r_yz],[r_xz r_yz r_yy]] is the
3x3 i-th feature observation covariance. Covariance of the Cartesian feature observations. Note the features are uncorrelated among them.

1.5 Filters

1.5.1 Histogram Filter

Histogram Filter



class HF.**HF**(*p0*, **args*)

Bases: `object`

Histogram Filter base class. Implements the histogram filter algorithm using a discrete Bayes Filter.

__init__(*p0*, **args*)

The histogram filter is initialized with the initial probability histogram *p0*. It creates the following attributes:

- self.p0: the initial belief histogram
- self.pk_1: the previous belief histogram, initially initialized from *p0*
- self.pk_hat: the prior belief histogram, after applying the Total Probability theorem
- self.pk: the posterior belief histogram, after applying the Bayes Rule

that will be updated during the execution of the filter. This method also initializes the state transition probability matrix *self.Pk*. If the file *StateTransitionProbability.npy* exists, the matrix will be loaded from it. Otherwise, it is computed by the derived class through the pure virtual method *StateTransitionProbability(·)* and stored in the file for posterior uses. This is done to avoid recomputing the matrix every since this is a

time-consuming operation. The state transition probability matrix is used in the *Prediction()* step and the measurement probability matrix is used in the *Update()* one.

Parameters
p0 – initial probability histogram

`ToCell`(*m*)

Converts a metric value to a cell displacement.

Parameters
m – value in meters

Returns
value in cells

`StateTransitionProbability`()

Returns the state transition probability matrix. This is a pure virtual method that must be implemented by the derived class.

Returns
Pk state transition probability matrix

`StateTransitionProbability_4_uk`(*uk*)

Returns the state transition probability matrix for the given control input *uk*. This is a pure virtual method that must be implemented by the derived class.

Parameters
uk – control input. In localization, this is commonly the robot displacement. For example, in the case of a differential drive robot, this is the robot displacement in the robot frame commonly computed through the odometry.

Returns
Puk state transition probability matrix for a given uk

`MeasurementProbability`(*zk*)

Returns the measurement probability matrix for the given measurement *zk*. This is a pure virtual method that must be implemented by the derived class.

Parameters
zk – measurement.

Returns
pzk measurement probability histogram

`uk2cell`(*uk*)

Converts the control input *uk* to a cell displacement. :param uk: :return:

`Prediction`(*pk_1*, *uk*)

Computes the prediction step of the histogram filter. Given the previous probability histogram *pk_1* and the control input *uk*, it computes the predicted probability histogram *pk_hat* after the robot displacement *uk* according to the motion model described by the state transition probability.

Parameters

- **pk_1** – previous probability histogram
- **uk** – control input

Returns
pk_hat predicted probability histogram

Update(*pk_hat*, *zk*)

Computes the update step of the histogram filter. Given the predicted probability histogram *pk_hat* and the measurement *zk*, it computes first the measurement probability histogram *pzk* and then uses the Bayes Rule to compute the updated probability histogram *pk*. :param pk_hat: predicted probability histogram :param zk: measurement :return: pk: updated probability histogram

class Histogram.**Histogram2D**(*num_bins_x*, *num_bins_y*, *x_range*, *y_range*)

Bases: `object`

Class for creating and manipulating a 2D histogram.

__init__(*num_bins_x*, *num_bins_y*, *x_range*, *y_range*)

Initialize a new Histogram2D instance.

Param

num_bins_x (int): Number of bins in the X-direction. num_bins_y (int): Number of bins in the Y-direction. x_range (numpy.ndarray): Range of values for the X-axis. y_range (numpy.ndarray): Range of values for the Y-axis.

property histogram_2d

Get the 2D histogram data as a NumPy array.

Returns

numpy.ndarray: The 2D histogram data.

property histogram_1d

Get the histogram data as a 1D NumPy array.

Returns

numpy.ndarray: The 1D histogram data.

plot_histogram()

Plot the 2D histogram using Matplotlib.

property element

Property to access individual elements of the histogram using range values.

Returns

ElementAccessor: An instance of ElementAccessor for getting and setting individual elements by range.

1.5.2 Particle Filter

Particle Filter

class ParticleFilter.**ParticleFilter**(*index*, *kSteps*, *robot*, *particles*, **args*)

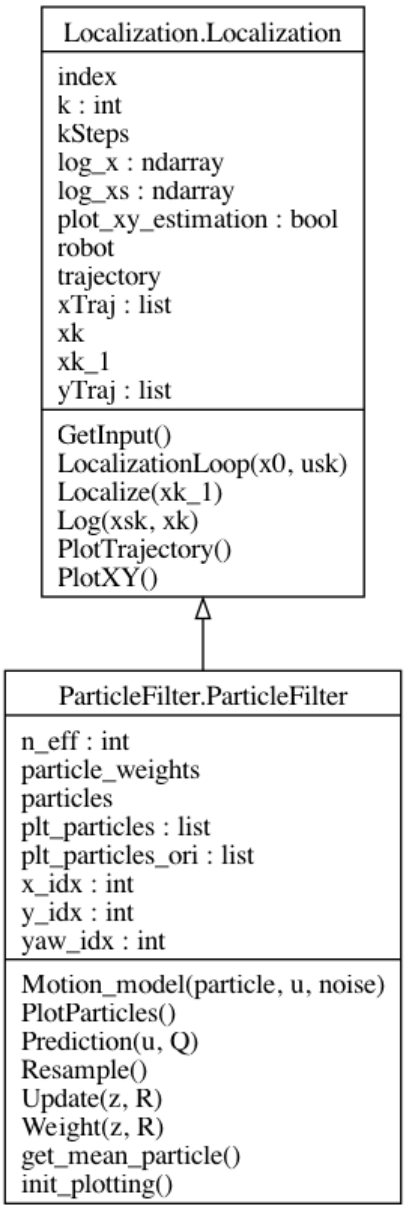
Bases: *Localization*

__init__(*index*, *kSteps*, *robot*, *particles*, **args*)

Constructor of the DRLocalization class.

Parameters

- **index** – Logging index structure (`Index`)
- **kSteps** – Number of time steps to simulate
- **robot** – Simulation robot object (`Robot`)


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
ParticleFilter.ParticleFilter
n_eff : int
particle_weights
particles
plt_particles : list
plt_particles_ori : list
x_idx : int
y_idx : int
yaw_idx : int
Motion_model(particle, u, noise)
PlotParticles()
Prediction(u, Q)
Resample()
Update(z, R)
Weight(z, R)
get_mean_particle()
init_plotting()

- **args** – Rest of arguments to be passed to the parent constructor
- **x0** – Initial Robot pose in the N-Frame

Motion_model(*particle*, *u*, *noise*)

To be implemented

Weight(*z*, *R*)

To be implemented

Resample()

To be implemented

Prediction(*u*, *Q*)

To be implemented

Update(*z*, *R*)

To be implemented

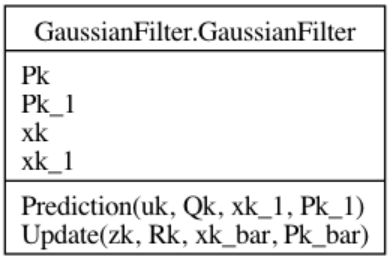
get_mean_particle()

init_plotting()

PlotParticles()

1.5.3 Gaussian Filter

Gaussian Filter



class GaussianFilter.**GaussianFilter**(*x0*, *P0*, **args*)

Bases: object

Gaussian Filter Interface

__init__(*x0*, *P0*, **args*)

Constructor of the GaussianFilter class.

Attributes:

- xk: mean of the state vector at time step k
- Pk: covariance of the state vector at time step k
- xk_1: mean of the state vector at time step k-1
- Pk_1: covariance of the state vector at time step k-1

Parameters

- **x0** – initial mean state vector
- **P0** – initial covariance matrix

Prediction(*uk*, *Qk*, *xk_1=None*, *Pk_1=None*)

Prediction step of the Gaussian Filter to be overwritten by the child class.

Parameters

- **uk** – input vector
- **Qk** – covariance matrix of the motion model noise
- **xk_1** – previous mean state vector
- **Pk_1** – previous covariance matrix

Return xk_bar, Pk_bar
current mean state vector and covariance matrix

Update(*zk*, *Rk*, *xk_bar=None*, *Pk_bar=None*)

Update step of the Gaussian Filter to be overwritten by the child class.

Parameters

- **zk** – observation vector
- **Rk** – covariance of the observation model noise
- **xk_bar** – mean of the predicted state
- **Pk_bar** – covariance of the predicted state

Returns
xk, Pk: current mean state vector and covariance matrix

Kalman Filter

Kalman Filter

class KF.**KF**(*Ak*, *Bk*, *Hk*, *Vk*, *x0*, *P0*, **args*)

Bases: *GaussianFilter*

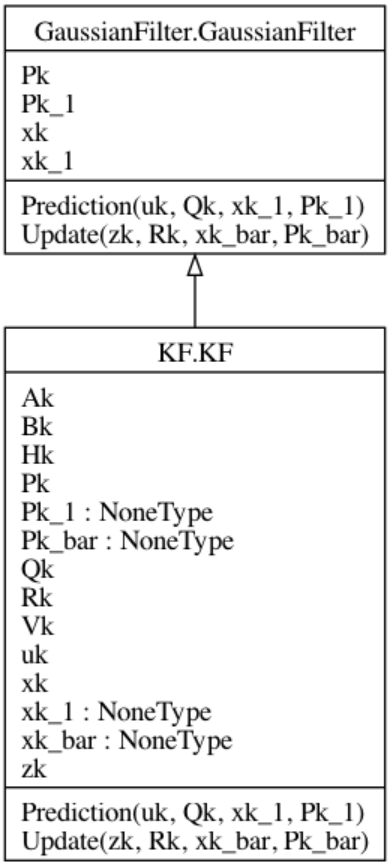
Kalman Filter class. Implements the `GaussianFilter` interface for the particular case of the Kalman Filter.

__init__(*Ak*, *Bk*, *Hk*, *Vk*, *x0*, *P0*, **args*)

Constructor of the KF class.

Parameters

- **Ak** – Transition matrix of the motion model
- **Bk** – Input matrix of the motion model
- **Hk** – Observation matrix of the observation model
- **Vk** – Noise projection matrix of the motion model
- **x0** – initial mean of the state vector
- **P0** – initial covariance matrix
- **args** – arguments to be passed to the parent class


GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
KF.KF
Ak
Bk
Hk
Pk
Pk_1 : NoneType
Pk_bar : NoneType
Qk
Rk
Vk
uk
xk
xk_1 : NoneType
xk_bar : NoneType
zk
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)

Prediction(*uk*, *Qk*, *xk_1=None*, *Pk_1=None*)

Prediction step of the Kalman Filter.

Parameters

- **uk** – input vector
- **Qk** – covariance matrix of the motion model noise
- **xk_1** – previous mean state vector
- **Pk_1** – previous covariance matrix

Return xk_bar, Pk_bar
current mean state vector and covariance matrix

Update(*zk*, *Rk*, *xk_bar=None*, *Pk_bar=None*)

Update step of the Kalman Filter.

Parameters

- **zk** – observation vector
- **Rk** – covariance of the observation model noise
- **xk_bar** – predicted mean state vector
- **Pk_bar** – predicted covariance matrix

Return xk.Pk
current mean state vector and covariance matrix

Extended Kalman Filter

Extended Kalman Filter

class EKF.**EKF**(*x0*, *P0*, **args*)

Bases: *GaussianFilter*

Extended Kalman Filter class. Implements the `GaussianFilter` interface for the particular case of the Extended Kalman Filter.

__init__(*x0*, *P0*, **args*)

Constructor of the EKF class.

Parameters

- **x0** – initial mean state vector
- **P0** – initial covariance matrix
- **args** – arguments to be passed to the parent class

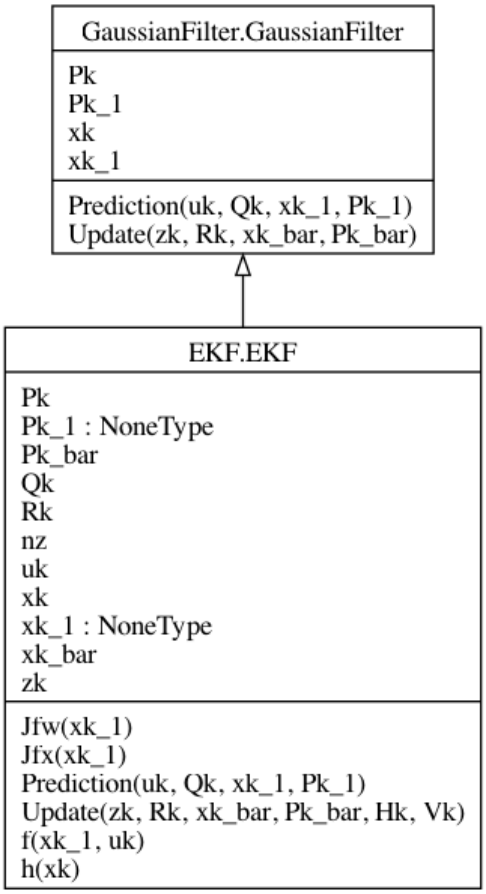
f(*xk_1*, *uk*)

Motion model of the EKF **to be overwritten by the child class**.

Parameters

- **xk_1** – previous mean state vector
- **uk** – input vector

Return xk_bar, Pk_bar
predicted mean state vector and its covariance matrix


GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)

`Jfx`(*xk_1*)

Jacobian of the motion model with respect to the state vector. **Method to be overwritten by the child class**.

Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

`Jfw`(*xk_1*)

Jacobian of the motion model with respect to the noise vector. **Method to be overwritten by the child class**.

Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

`h`(*xk*)

The observation model of the EKF is given by:

$$z_k = h(x_k, v_k) \tag{1.40}$$

This method computes the mean of this direct observation model. Therefore it does not depend on v_k since it is a zero mean Gaussian noise.

Parameters
xk – mean of the predicted state vector. By default it is taken from the class attribute.

Returns
expected observation vector

`Prediction`(*uk*, *Qk*, *xk_1=None*, *Pk_1=None*)

Prediction step of the EKF. It calls the motion model and its Jacobians to predict the state vector and its covariance matrix.

Parameters

- **uk** – input vector
- **Qk** – covariance matrix of the noise vector
- **xk_1** – previous mean state vector. By default it is taken from the class attribute. Otherwise it updates the class attribute.
- **Pk_1** – covariance matrix of the previous state vector. By default it is taken from the class attribute. Otherwise it updates the class attribute.

Return xk_bar, Pk_bar
predicted mean state vector and its covariance matrix. Also updated in the class attributes.

`Update`(*zk*, *Rk*, *xk_bar*, *Pk_bar*, *Hk*, *Vk*)

Update step of the EKF. It calls the observation model and its Jacobians to update the state vector and its covariance matrix.

Parameters

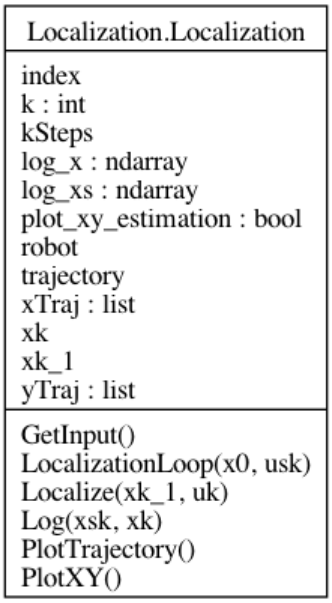
- **zk** – observation vector

- **Rk** – covariance matrix of the noise vector
- **xk_bar** – predicted mean state vector.
- **Pk_bar** – covariance matrix of the predicted state vector.
- **Hk** – Jacobian of the observation model with respect to the state vector.
- **Vk** – Jacobian of the observation model with respect to the noise vector.

Return xk,Pk
updated mean state vector and its covariance matrix. Also updated in the class attributes.

1.6 Localization

1.6.1 Robot Localization



class Localization.**Localization**(*index*, *kSteps*, *robot*, *x0*, **args*)

Bases: `object`

Localization base class. Implements the localization algorithm.

__init__(*index*, *kSteps*, *robot*, *x0*, **args*)

Constructor of the DRLocalization class.

Parameters

- **index** – Logging index structure (`Index`)
- **kSteps** – Number of time steps to simulate
- **robot** – Simulation robot object (`Robot`)
- **args** – Rest of arguments to be passed to the parent constructor
- **x0** – Initial Robot pose in the N-Frame

`GetInput()`

Gets the input from the robot. To be overidden by the child class.

Returns

- **uk:** input variable

`Localize(xk_1)`

Single Localization iteration invoked from `DRLocalization.Localization()`. Given the previous robot pose, the function reads the inout and computes the current pose.

Parameters
xk_1 – previous robot pose

Returns
xk current robot pose

`LocalizationLoop(x0, usk)`

Given an initial robot pose x_0 and the input to the `SimulatedRobot` this method calls iteratively `DRLocalization.Localize()` for k steps, solving the robot localization problem.

Parameters
x0 – initial robot pose

`Log(xsk, xk)`

Logs the results for later plotting.

Parameters

- **xsk** – ground truth robot pose from the simulation
- **xk** – estimated robot pose

`PlotXY()`

Plots, in a new figure, the ground truth (orange) and estimated (blue) trajectory of the robot at the end of the Localization Loop.

`PlotTrajectory()`

Plots the estimated trajectory (blue) of the robot during the localization process.

1.6.2 Dead Reckoning

4 DOF AUV Dead REckoning using DVL and Gyro

`class DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro(index, kSteps, robot, x0, *args)`

Bases: *Localization*

Dead Reckoning Localization for a 4DOF AUV with DVL and Gyro sensors

`__init__(index, kSteps, robot, x0, *args)`

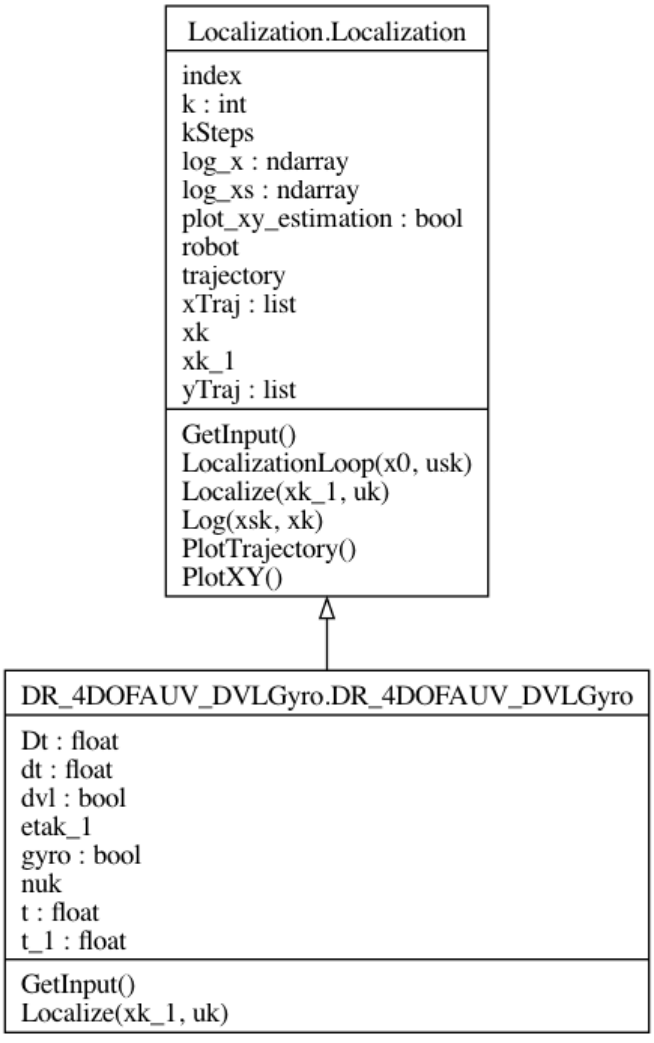
Constructor of the DR_4DOFAUV_DVLGyro class.

Parameters
args – Rest of arguments to be passed to the parent constructor

`Localize(xk_1)`

Motion model for the 4DOF ($[x_k\ y_k\ z_k\ \psi_k]^T$) AUV using as input the lineal velocity read from the DVL senosr and angular velocity read from the Gyro sensor

Parameters


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro
Dt : float
dt : float
dvl : bool
etak_1
gyro : bool
nuk
t : float
t_1 : float
GetInput()
Localize(xk_1, uk)

- **xk_1** – previous robot pose estimate ($[x_{k-1}\ y_{k-1}\ z_{k-1}\ \psi_{k-1}]^T$)
- **uk** – input vector ($[u_k\ v_k\ w_k\ r_k]^T$)

Return xk

current robot pose estimate ($[x_k\ y_k\ z_k\ \psi_k]^T$)

`GetInput()`

Gets the input vector and the input noise covariance matrix from the robot. The input vector contains the linear displacement computed as the integral of the velocity read from the DVL (linear) and the Gyro (angular) of the robot.

$$u_k = \begin{bmatrix}\nu_{DVL}^T & r_{Gyro}^T\end{bmatrix}^T = \begin{bmatrix}u_{DVL} & v_{DVL} & w_{DVL} & r_{Gyro}\end{bmatrix}^T \cdot \Delta t$$
$$Q_k = \begin{bmatrix}Q_{DVL} & 0 \\ 0 & \sigma_{Gyro}^2\end{bmatrix} \quad \text{where} \quad Q_{DVL} = \begin{bmatrix}\sigma_{u_{DVL}}^2 & \sigma_{uv_{DVL}} & \sigma_{uw_{DVL}} \\ \sigma_{vu_{DVL}} & \sigma_{v_{DVL}}^2 & \sigma_{vw_{DVL}} \\ \sigma_{wu_{DVL}} & \sigma_{wv_{DVL}} & \sigma_{w_{DVL}}^2\end{bmatrix} \cdot \Delta t^2 \tag{1.41}$$

Returns

input vector u_k and input noise covariance matrix Q_k defined in eq. (1.41).

3 DOF Differential Drive Mobile Robot Example

`class` DR_3DOFDifferentialDrive.**DR_3DOFDifferentialDrive**(*index*, *kSteps*, *robot*, *x0*, **args*)

Bases: *Localization*

Dead Reckoning Localization for a Differential Drive Mobile Robot.

__init__(*index*, *kSteps*, *robot*, *x0*, **args*)

Constructor of the *DR_3DOFDifferentialDrive* class.

Parameters

args – Rest of arguments to be passed to the parent constructor

`Localize`(*xk_1*)

Motion model for the 3DOF ($x_k = [x_k\ y_k\ \psi_k]^T$) Differential Drive Mobile robot using as input the readings of the wheel encoders ($u_k = [n_L\ n_R]^T$).

Parameters

- **xk_1** – previous robot pose estimate ($x_{k-1} = [x_{k-1}\ y_{k-1}\ \psi_{k-1}]^T$)
- **uk** – input vector ($u_k = [u_k\ v_k\ r_k]^T$)

Returns

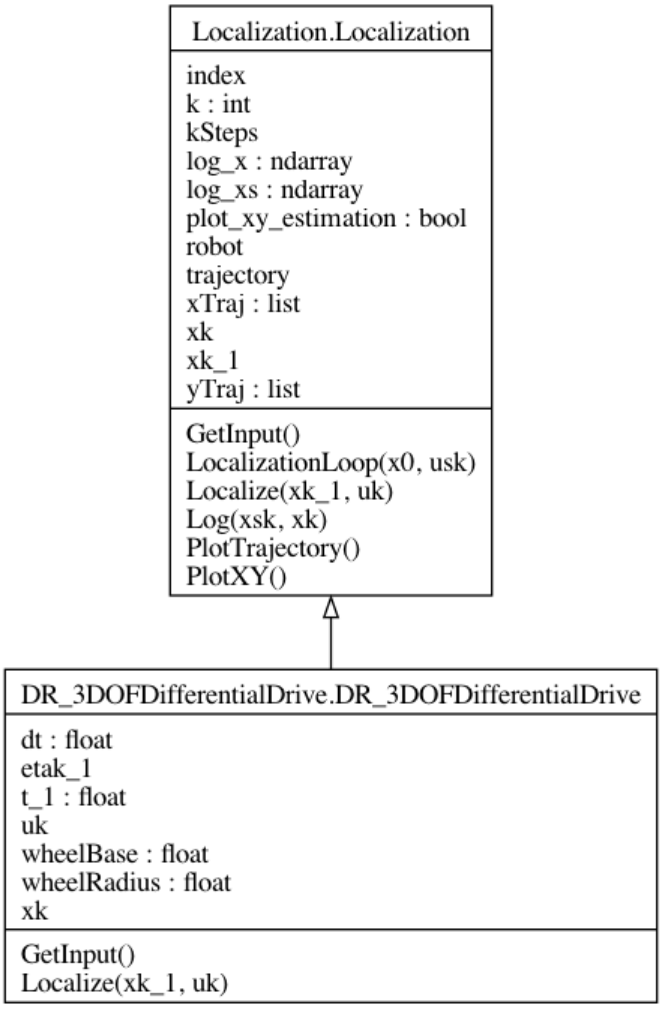
- **xk** current robot pose estimate ($[x_k\ y_k\ \psi_k]^T$)

`GetInput()`

Get the input for the motion model. In this case, the input is the robot displacement computed from the left and right wheel encoders pulses using.

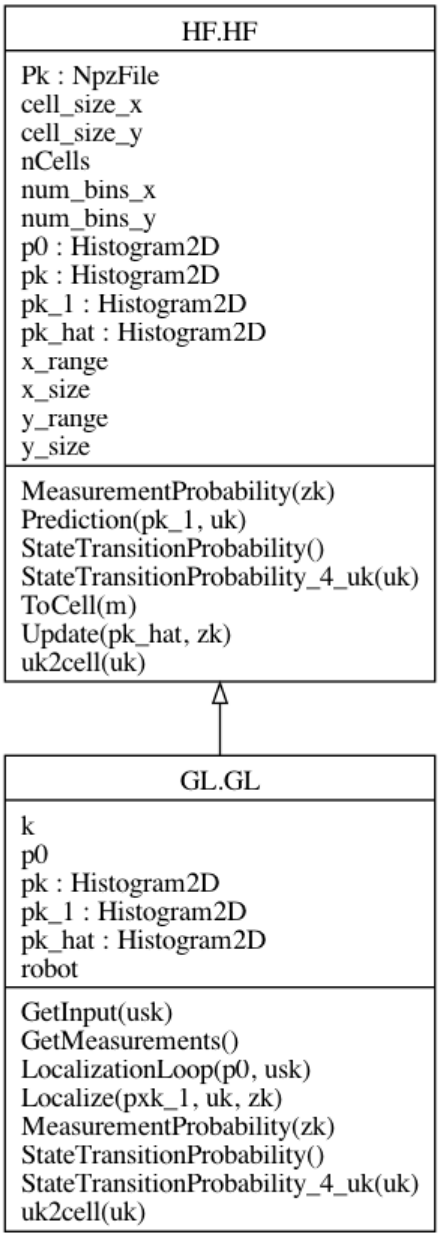
Returns

- **uk:** input vector ($u_k = {}^B[\Delta x\ \Delta y]^T$)


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)

1.6.3 Grid Localization

Grid Localization



class GL.**GL**(*p0*, *index*, *kSteps*, *robot*, *x0*, **args*)

Bases: *HF*

Grid Localization.

__init__(*p0*, *index*, *kSteps*, *robot*, *x0*, **args*)

Constructor of the GL_4DOFAUV class. Initializes the Dead reckoning localization algorithm as well as the histogram filter algorithm.

Parameters

- **dx_max** – maximum x displacement in meters
- **dy_max** – maximum y displacement in meters
- **range_dx** – range of x displacements in meters
- **range_dy** – range of y displacements in meters
- **p0** – initial probability histogram
- **index** – index struture containing plotting information
- **kSteps** – number of time steps to simulate the robot motion
- **robot** – robot object
- **x0** – initial robot pose
- **args** – additional arguments

GetMeasurements()

Read the measurements from the robot. This is a pure virtual method that must be implemented by the derived class.

StateTransitionProbability_4_uk(*uk*)

Returns the state transition probability matrix for the given control input *uk*. This is a pure virtual method that must be implemented by the derived class.

Parameters
uk – control input. In localization, this is commonly the robot displacement. For example, in the case of a differential drive robot, this is the robot displacement in the robot frame commonly computed through the odometry.

Returns
Puk state transition probability matrix for a given uk

StateTransitionProbability()

Computes the complete state transition probability matrix. The matrix is a $n_u imes m_u imes n^2$ matrix, where n_u and m_u are the number of possible displacements in the x and y axis, respectively, and n is the number of cells in the map. For each possible displacement u_k, each previous robot pose x_{k-1} and each current robot pose x_k, the probability $p(x_k|x_{k-1}, u_k)$ is computed. This is a pure virtual method that must be implemented by the derived class.

Returns
state transition probability matrix $P_k = px_k|x_{k-1}, uk$

MeasurementProbability(*zk*)

Computes the measurement probability histogram given the robot pose η_k and the measurement z_k. Method to be overriden by the child class.

Parameters
zk – vector of measurements

Returns
Measurement probability histogram $p_z = p(z_k|\eta_k)$

GetInput(*usk*)

Gets the number of cells the robot has displaced along its DOFs in the world N-Frame. Method to be overriden by the child class.

Parameters
usk – control input of the robot simulation

Returns
uk: vector containing the number of cells the robot has displaced in all the axis of the world N-Frame

uk2cell(*uk*)

Parameters
uk –

Returns

LocalizationLoop(*p0*, *usk*)

Given an initial robot pose x_0 and the input to the `AUV4DOFSimulatedRobot.SimulatedRobot` this method calls iteratively `GL_4DOFAUV.Localize()` for k steps, solving the robot localization problem.

Parameters

- **p0** – initial robot pose
- **usk** – control input of the robot simulation

Localize(*pxk_1*, *uk*, *zk*)

Overrides the parent method `DR_4DOFAUV_DVLGyro.Localize()`.

Parameters

- **pxk_1** – histogram of the previous robot position
- **uk** – robot displacement in number of cells in the world N-Frame
- **zk** – vector containing the measurements of the robot position in the world N-Frame

Returns
pk: histogram of the robot position after the prediction and the update steps

4 DOF AUV

AUV Grid Localization Example

class GL_4DOFAUV.**GL_4DOFAUV**(*dx_max*, *dy_max*, *range_dx*, *range_dy*, *p0*, *index*, *kSteps*, *robot*, *x0*, **args*)

Bases: *GL*, *DR_4DOFAUV_DVLGyro*

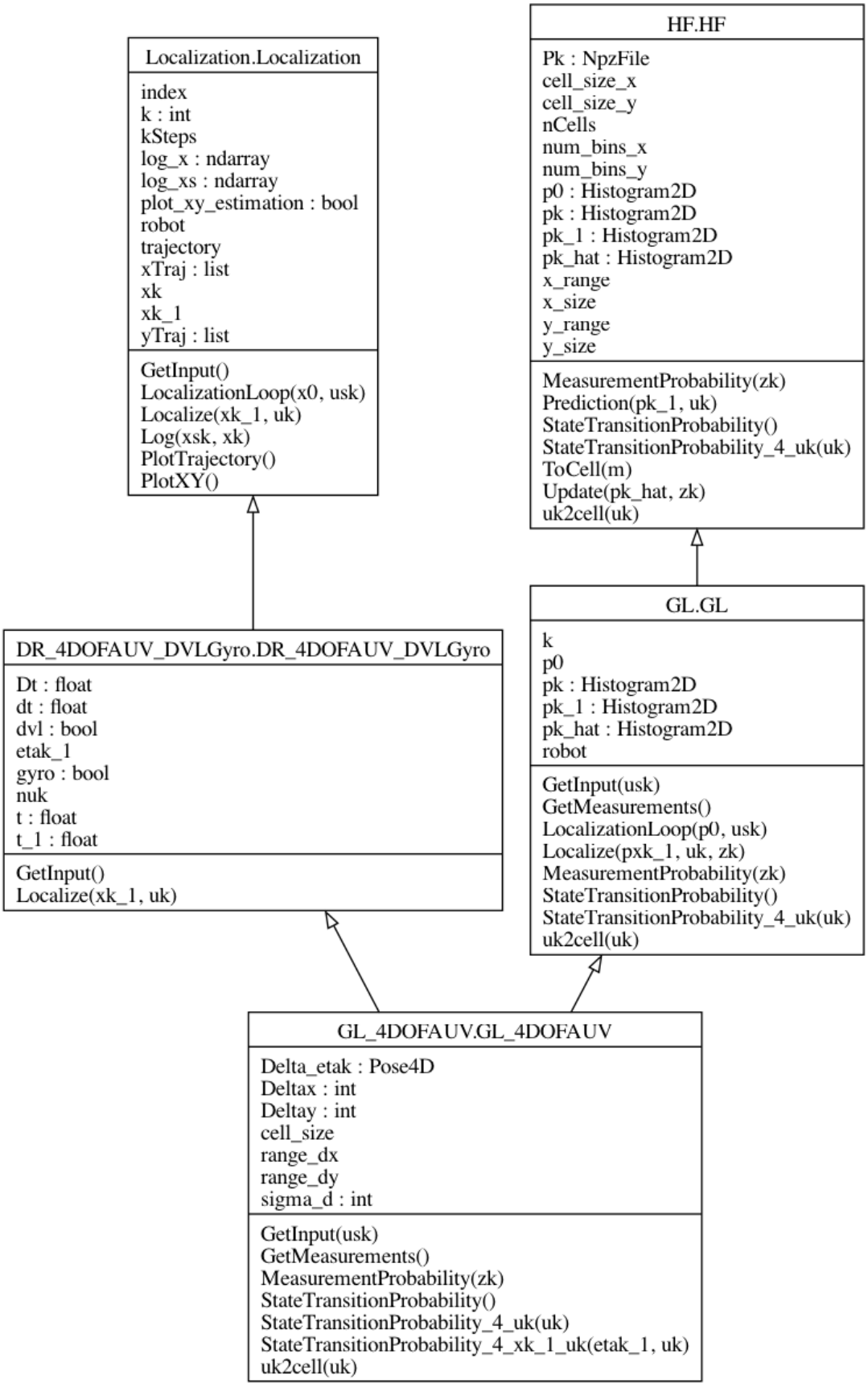
Grid Reckoning Localization for a 4 DOF AUV.

__init__(*dx_max*, *dy_max*, *range_dx*, *range_dy*, *p0*, *index*, *kSteps*, *robot*, *x0*, **args*)

Constructor of the *GL_4DOFAUV* class. Initializes the Dead reckoning localization algorithm as well as the histogram filter algorithm.

Parameters

- **dx_max** – maximum x displacement in meters
- **dy_max** – maximum y displacement in meters
- **range_dx** – range of x displacements in meters
- **range_dy** – range of y displacements in meters
- **p0** – initial probability histogram
- **index** – index struture containing plotting information
- **kSteps** – number of time steps to simulate the robot motion


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
HF.HF
Pk : NpzFile
cell_size_x
cell_size_y
nCells
num_bins_x
num_bins_y
p0 : Histogram2D
pk : Histogram2D
pk_1 : Histogram2D
pk_hat : Histogram2D
x_range
x_size
y_range
y_size
MeasurementProbability(zk)
Prediction(pk_1, uk)
StateTransitionProbability()
StateTransitionProbability_4_uk(uk)
ToCell(m)
Update(pk_hat, zk)
uk2cell(uk)
DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro
Dt : float
dt : float
dvl : bool
etak_1
gyro : bool
nuk
t : float
t_1 : float
GetInput()
Localize(xk_1, uk)
GL.GL
k
p0
pk : Histogram2D
pk_1 : Histogram2D
pk_hat : Histogram2D
robot
GetInput(usk)
GetMeasurements()
LocalizationLoop(p0, usk)
Localize(pxk_1, uk, zk)
MeasurementProbability(zk)
StateTransitionProbability()
StateTransitionProbability_4_uk(uk)
uk2cell(uk)
GL_4DOFAUV.GL_4DOFAUV
Delta_etak : Pose4D
Deltax : int
Deltay : int
cell_size
range_dx
range_dy
sigma_d : int
GetInput(usk)
GetMeasurements()
MeasurementProbability(zk)
StateTransitionProbability()
StateTransitionProbability_4_uk(uk)
StateTransitionProbability_4_xk_1_uk(etak_1, uk)
uk2cell(uk)

- **robot** – robot object
- **x0** – initial robot pose
- **args** – additional arguments

GetMeasurements()

Read the measurements from the robot. Returns a vector of range distances to the map features. Only those features that are within the SimulatedRobot.SimulatedRobot.Distance_max_range of the sensor are returned. The measurements arribe at a frequency defined in the SimulatedRobot.SimulatedRobot.Distance_feature_reading_frequency attribute.

Returns
vector of distances to the map features

StateTransitionProbability_4_uk(*uk*)

Returns the state transition probability matrix for the given control input *uk*. This is a pure virtual method that must be implemented by the derived class.

Parameters
uk – control input. In localization, this is commonly the robot displacement. For example, in the case of a differential drive robot, this is the robot displacement in the robot frame commonly computed through the odometry.

Returns
Puk state transition probability matrix for a given uk

StateTransitionProbability_4_xk_1_uk(*etak_1*, *uk*)

Computes the state transition probability histogram given the previous robot pose η_{k-1} and the input u_k:

$$p(\eta_k|\eta_{k-1}, u_k)$$

Parameters

- **etak_1** – previous robot pose in cells
- **uk** – input displacement in number of cells

Returns
state transition probability $p(\eta_k|\eta_{k-1}, u_k)$

StateTransitionProbability()

Computes the complete state transition probability matrix. The matrix is a $n_u imes m_u imes n^2$ matrix, where n_u and m_u are the number of possible displacements in the x and y axis, respectively, and n is the number of cells in the map. For each possible displacement u_k, each previous robot pose x_{k-1} and each current robot pose x_k, the probability $p(x_k|x_{k-1}, u_k)$ is computed.

Returns
state transition probability matrix $P_k = px_k|x_{k-1}, uk$

uk2cell(*uk*)

Parameters
uk –

Returns

MeasurementProbability(*zk*)

Computes the measurement probability histogram given the robot pose η_k and the measurement z_k. In this case the the measurement is the vector of the distances to the landmarks in the map.

Parameters
zk – $z_k = [r_0\ r_1\ ..r_k]$ where r_i is the distance to the i-th landmark in the map.

Returns
Measurement probability histogram $p_z = p(z_k|\eta_k)$

`GetInput`(*usk*)

Provides an implementation for the virtual method `GL.GetInput()`. Gets the number of cells the robot has displaced in the x and y directions in the world N-Frame. To do it, it calls several times the parent method `super().GetInput()`, corresponding to the Dead Reckoning Localization of the robot, until it has displaced at least one cell in any direction. Note that an iteration of the robot simulation `SimulatedRobot.fs()` is normally done in the `GL_4DOFAUV.LocalizationLoop()` method of the `GL_4DOFAUV.Localization` class, but in this case it is done here to simulate the robot motion between the consecutive calls to `super().GetInput()`.

Parameters
usk – control input of the robot simulation

Returns
uk: vector containing the number of cells the robot has displaced in the x and y directions in the world N-Frame

3 DOF Differential Drive Mobile Robot

Differential Drive Grid Localization Example

class GL_3DOFDifferentialDrive.**GL_3DOFDifferentialDrive**(*dx_max*, *dy_max*, *range_dx*, *range_dy*, *p0*, *index*, *kSteps*, *robot*, *x0*, **args*)

Bases: *GL*, *DR_3DOFDifferentialDrive*

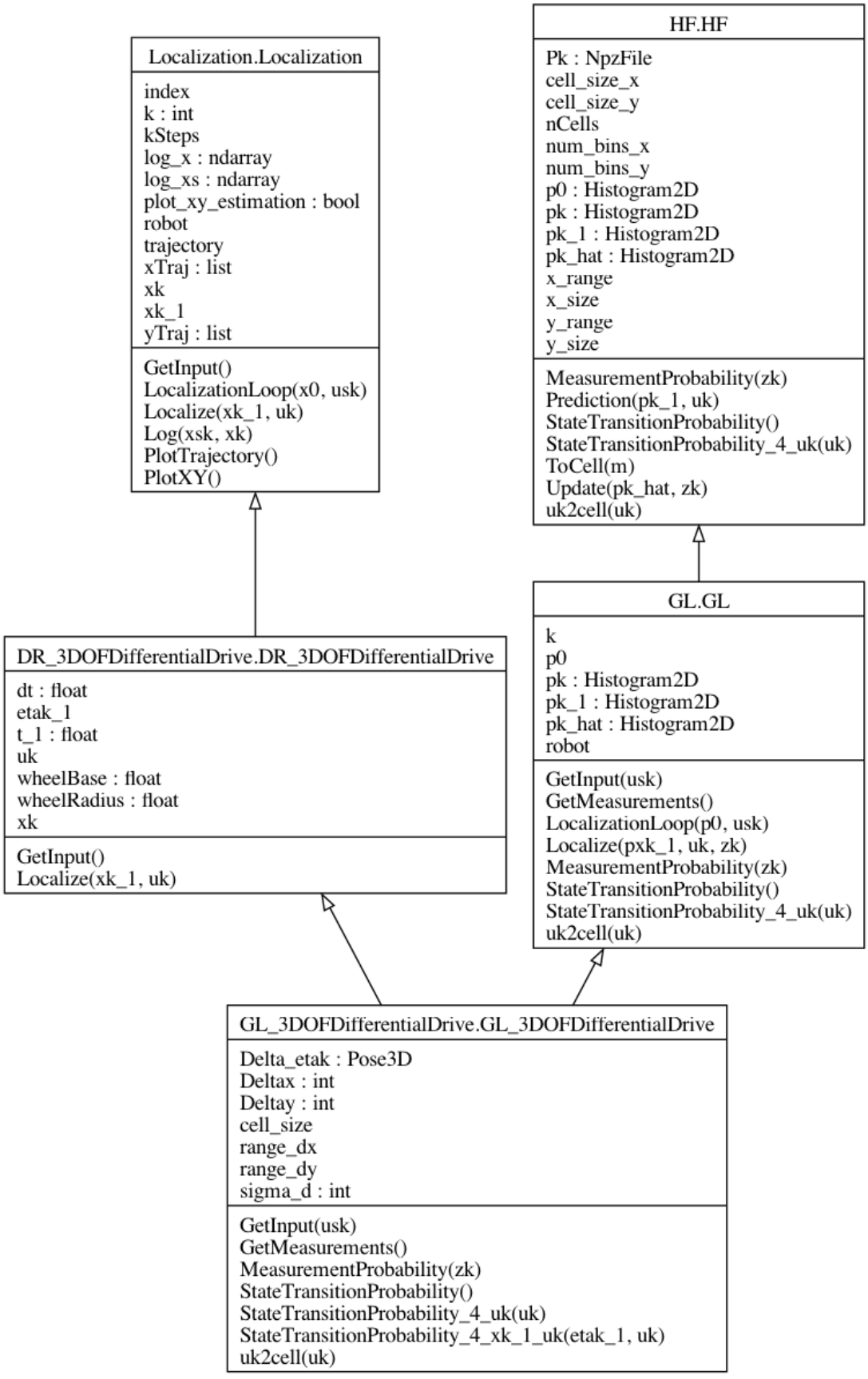
Grid Reckoning Localization for a 4 DOF AUV.

__init__(*dx_max*, *dy_max*, *range_dx*, *range_dy*, *p0*, *index*, *kSteps*, *robot*, *x0*, **args*)

Constructor of the `GL_4DOFAUV` class. Initializes the Dead reckoning localization algorithm as well as the histogram filter algorithm.

Parameters

- **dx_max** – maximum x displacement in meters
- **dy_max** – maximum y displacement in meters
- **range_dx** – range of x displacements in meters
- **range_dy** – range of y displacements in meters
- **p0** – initial probability histogram
- **index** – index struture containing plotting information
- **kSteps** – number of time steps to simulate the robot motion
- **robot** – robot object
- **x0** – initial robot pose
- **args** – additional arguments


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
HF.HF
Pk : NpzFile
cell_size_x
cell_size_y
nCells
num_bins_x
num_bins_y
p0 : Histogram2D
pk : Histogram2D
pk_1 : Histogram2D
pk_hat : Histogram2D
x_range
x_size
y_range
y_size
MeasurementProbability(zk)
Prediction(pk_1, uk)
StateTransitionProbability()
StateTransitionProbability_4_uk(uk)
ToCell(m)
Update(pk_hat, zk)
uk2cell(uk)
GL.GL
k
p0
pk : Histogram2D
pk_1 : Histogram2D
pk_hat : Histogram2D
robot
GetInput(usk)
GetMeasurements()
LocalizationLoop(p0, usk)
Localize(pxk_1, uk, zk)
MeasurementProbability(zk)
StateTransitionProbability()
StateTransitionProbability_4_uk(uk)
uk2cell(uk)
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)
GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive
Delta_etak : Pose3D
Deltax : int
Deltay : int
cell_size
range_dx
range_dy
sigma_d : int
GetInput(usk)
GetMeasurements()
MeasurementProbability(zk)
StateTransitionProbability()
StateTransitionProbability_4_uk(uk)
StateTransitionProbability_4_xk_1_uk(etak_1, uk)
uk2cell(uk)

`GetMeasurements()`

Read the measurements from the robot. Returns a vector of range distances to the map features. Only those features that are within the `SimulatedRobot.SimulatedRobot.Distance_max_range` of the sensor are returned. The measurements arribe at a frequency defined in the `SimulatedRobot.SimulatedRobot.Distance_feature_reading_frequency` attribute.

Returns
vector of distances to the map features

`StateTransitionProbability_4_uk(uk)`

Returns the state transition probability matrix for the given control input *uk*. This is a pure virtual method that must be implemented by the derived class.

Parameters
uk – control input. In localization, this is commonly the robot displacement. For example, in the case of a differential drive robot, this is the robot displacement in the robot frame commonly computed through the odometry.

Returns
Puk state transition probability matrix for a given uk

`StateTransitionProbability_4_xk_1_uk(etak_1, uk)`

Computes the state transition probability histogram given the previous robot pose η_{k-1} and the input u_k:

$$p(\eta_k|\eta_{k-1}, u_k)$$

Parameters

- **etak_1** – previous robot pose in cells
- **uk** – input displacement in number of cells

Returns
state transition probability $p(\eta_k|\eta_{k-1}, u_k)$

`StateTransitionProbability()`

Computes the complete state transition probability matrix. The matrix is a $n_u imes m_u imes n^2$ matrix, where n_u and m_u are the number of possible displacements in the x and y axis, respectively, and n is the number of cells in the map. For each possible displacement u_k, each previous robot pose x_{k-1} and each current robot pose x_k, the probability $p(x_k|x_{k-1}, u_k)$ is computed.

Returns
state transition probability matrix $P_k = px_k|x_{k-1}, uk$

`uk2cell(uk)`

Parameters
uk –

Returns

`MeasurementProbability(zk)`

Computes the measurement probability histogram given the robot pose η_k and the measurement z_k. In this case the the measurement is the vector of the distances to the landmarks in the map.

Parameters
zk – $z_k = [r_0\ r_1\ ..r_k]$ where r_i is the distance to the i-th landmark in the map.

Returns
Measurement probability histogram $p_z = p(z_k|\eta_k)$

GetInput(*usk*)

Provides an implementation for the virtual method `GL.GetInput()`. Gets the number of cells the robot has displaced in the x and y directions in the world N-Frame. To do it, it calls several times the parent method `super().GetInput()`, corresponding to the Dead Reckoning Localization of the robot, until it has displaced at least one cell in any direction. Note that an iteration of the robot simulation `SimulatedRobot.fs()` is normally done in the `GL_4DOFAUV.LocalizationLoop()` method of the `GL_4DOFAUV.Localization` class, but in this case it is done here to simulate the robot motion between the consecutive calls to `super().GetInput()`.

Parameters
usk – control input of the robot simulation

Returns
uk: vector containing the number of cells the robot has displaced in the x and y directions in the world N-Frame

1.6.4 Montecarlo Localization

Montecarlo Localization

class MCLocalization.**MCLocalization**(*index*, *kSteps*, *robot*, *particles*, **args*)

Bases: `ParticleFilter`

__init__(*index*, *kSteps*, *robot*, *particles*, **args*)

Constructor. :param index: Named tuple used to map the state vector, the simulation vector and the observation vector (`prpy.IndexStruct`) :param kSteps: simulation time steps :param robot: Simulated Robot object :param particles: initial particles :param args: arguments to be passed to the parent constructor

Prediction(*u*, *Q*)

To be implemented

Update(*z*, *R*)

To be implemented

Localize()

Single Localization iteration invoked from `DRLocalization.Localization()`. Given the previous robot pose, the function reads the inout and computes the current pose.

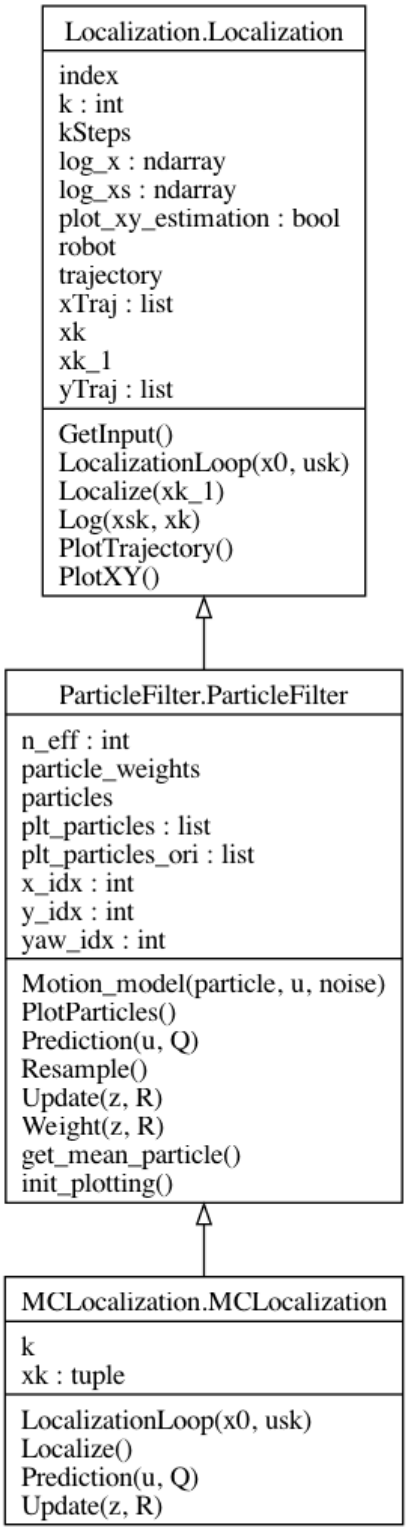
Parameters
xk_1 – previous robot pose

Returns
xk current robot pose

LocalizationLoop(*x0*, *usk*)

Given an initial robot pose x_0 and the input to the `SimulatedRobot` this method calls iteratively `DRLocalization.Localize()` for k steps, solving the robot localization problem.

Parameters
x0 – initial robot pose


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
ParticleFilter.ParticleFilter
n_eff : int
particle_weights
particles
plt_particles : list
plt_particles_ori : list
x_idx : int
y_idx : int
yaw_idx : int
Motion_model(particle, u, noise)
PlotParticles()
Prediction(u, Q)
Resample()
Update(z, R)
Weight(z, R)
get_mean_particle()
init_plotting()
MCLocalization.MCLocalization
k
xk : tuple
LocalizationLoop(x0, usk)
Localize()
Prediction(u, Q)
Update(z, R)

1.6.5 Gaussian Filter Localization

Gaussian Filter Localization

class GFLocalization.**GFLocalization**(*index*, *kSteps*, *robot*, *x0*, *P0*, **args*)

Bases: *Localization*, *GaussianFilter*

Map-less localization using a Gaussian filter.

__init__(*index*, *kSteps*, *robot*, *x0*, *P0*, **args*)

Constructor.

Parameters

- **x0** – initial state
- **P0** – initial covariance
- **index** – Named tuple used to relate the state vector, the simulation and the observation vectors (*IndexStruct.IndexStruct*)
- **kSteps** – simulation time steps
- **robot** – Simulated Robot object
- **args** – arguments to be passed to the parent constructor

GetInput()

Gets the input from the robot. Relates to the motion model as follows:

$$\begin{aligned} x_k &= f(x_{k-1}, u_k, w_k) \\ w_k &= N(0, Q_k) \end{aligned} \tag{1.42}$$

To be overidden by the child class .

Return uk, Qk
input and covariance of the motion model

GetMeasurements()

Get the measurements from the robot. Corresponds to the observation model:

$$\begin{aligned} z_k &= h(x_k, v_k) \\ v_k &= N(0, R_k) \end{aligned} \tag{1.43}$$

To be overidden by the child class .

Returns
zk, Rk, Hk, Vk: observation vector and covariance of the observation noise. Hk is the Observation matrix and Vk is the noise observation matrix.

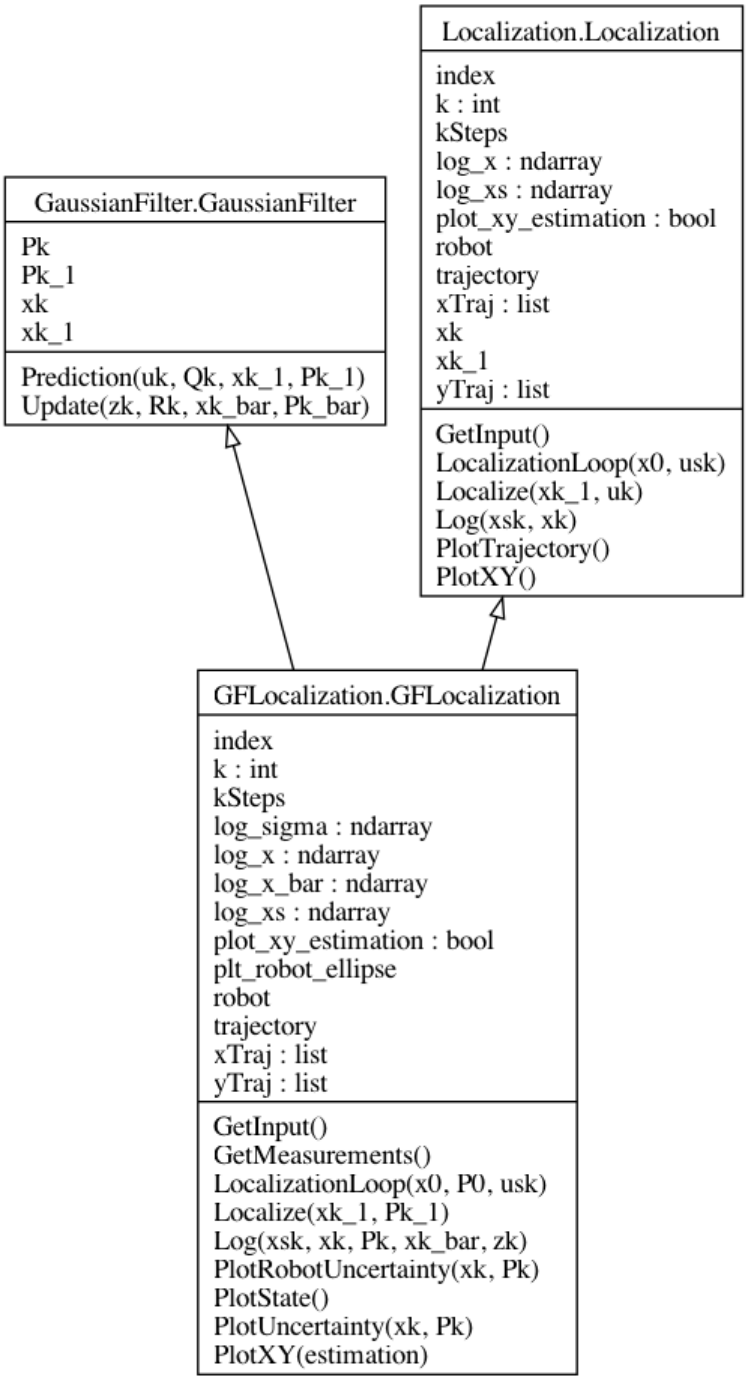
Localize(*xk_1*, *Pk_1*)

Localization iteration. Reads the input of the motion model, performs the prediction step, reads the measurements, performs the update step and logs the results. The method also plots the uncertainty ellipse of the robot pose.

Parameters

- **xk_1** – previous state vector
- **Pk_1** – previous covariance matrix

Return xk, Pk
updated state vector and covariance matrix


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)

LocalizationLoop(*x0*, *P0*, *usk*)

Localization loop. During *self.kSteps* it calls the *Localize()* method for each time step.

Parameters

- **x0** – initial state vector
- **P0** – initial covariance matrix

Log(*xsk*, *xk*, *Pk*, *xk_bar*, *zk*)

Logs the results for later plotting.

Parameters

- **xsk** – ground truth robot pose from the simulation
- **xk** – estimated robot pose

PlotState()

Plot the results of the localization For each state DOF s -si[s] is the corresponding simulated stated -x1[s] is the corresponding observation

PlotXY(*estimation=True*)

Plot the x-y trajectory of the robot simulation: True if the simulated XY robot trajectory is available

PlotRobotUncertainty(*xk*, *Pk*)

Plots the robot trajectory and its uncertainty ellipse. :param xk: state vector :param Pk: covariance matrix of the state vector

PlotUncertainty(*xk*, *Pk*)

Plots the uncertainty ellipse of the robot pose. :param xk: state vector :param Pk: covariance matrix of the state vector

IndexStruct

class IndexStruct.**IndexStruct**(*state: str*, *simulation: int*, *observation: int*)

Bases: NamedTuple

Named tuple used to map which dimensions of state vector, the simulation and the observation vectors are related. This is needed for logging and plotting, so it is possible to compute the estimation error by substracting the ground truth (simulation) from the estimated state.

Each Localization implementation deriving from *GFLocalization.GFLocalization* class must create a list of these structures, defining the localization variables being estimated and their relation to the simulation and observation vectors. The list must be passed to the *GFLocalization.GFLocalization* constructor.

Each element of the list has the following structure:

index[state_index]=("state_name", simulation_index, observation_index)

defining that the state vector dimension "state_name" is stored in rh state_index dimension of the state vector GaussianFilter.GaussianFilter.xk, being its ground truth value available in the simulation_index dimension of the simulation vector SimulatedRobot.SimulatedRobot.xsk and whose observation is contained in the observation_index dimension of the observation vector :math: *z_k*.

Example of usage:

index = [IndexStruct("x", 0, None), IndexStruct("y", 1, None), IndexStruct("z", 2, 0), IndexStruct("yaw", 3, 1),
IndexStruct("u", 4, 2), IndexStruct("v", 5, 3), IndexStruct("w", 6, 0), IndexStruct("yaw_dot", 7, None)]

- the x position of the robot, is stored in the 0 dimension of the state vector `GaussianFilter.GaussianFilter.xk` and is not being observed.
- the y position of the robot, is stored in the 1 dimension of the state vector `GaussianFilter.GaussianFilter.xk` and is not being observed.
- the z position of the robot, is stored in the 2 dimension of the state vector `GaussianFilter.GaussianFilter.xk` and is being observed in the 0 dimension of the observation vector :math: *z_k*.
- the yaw angle of the robot, is stored in the 3 dimension of the state vector `GaussianFilter.GaussianFilter.xk` and is being observed in the 1 dimension of the observation vector :math: *z_k*.
- the linear velocity of the robot along the x axis of the B-Frame, is stored in the 4 dimension of the state vector `GaussianFilter.GaussianFilter.xk`, and is being observed in the 2 dimension of the observation vector :math: *z_k*.
- the linear velocity of the robot along the y axis of the B-Frame, is stored in the 5 dimension of the state vector `GaussianFilter.GaussianFilter.xk`, and is being observed in the 3 dimension of the observation vector :math: *z_k*.
- the linear velocity of the robot along the z axis of the B-Frame, is stored in the 6 dimension of the state vector `GaussianFilter.GaussianFilter.xk`, and is being observed in the 0 dimension of the observation vector :math: *z_k*.
- the angular velocity of the robot, is stored in the 7 dimension of the state vector `GaussianFilter.GaussianFilter.xk` and is not being observed.

state: name of the estimated state *simulation*: index of the simulated state vector *observation*: index of the observation vector

state: str

Alias for field number 0

simulation: int

Alias for field number 1

observation: int

Alias for field number 2

4 DOF AUV

4 DOF AUV EKF Localization using an Input Velocity Motion Model with Depht, Yaw and Linear Velocity Measurements

4 DOF AUV EKF Localization using a Constant Velocity Motion Model with Depht, Yaw and Linear Velocity Measurements

class EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM.**EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM**(*kSteps*, *robot*, **args*)

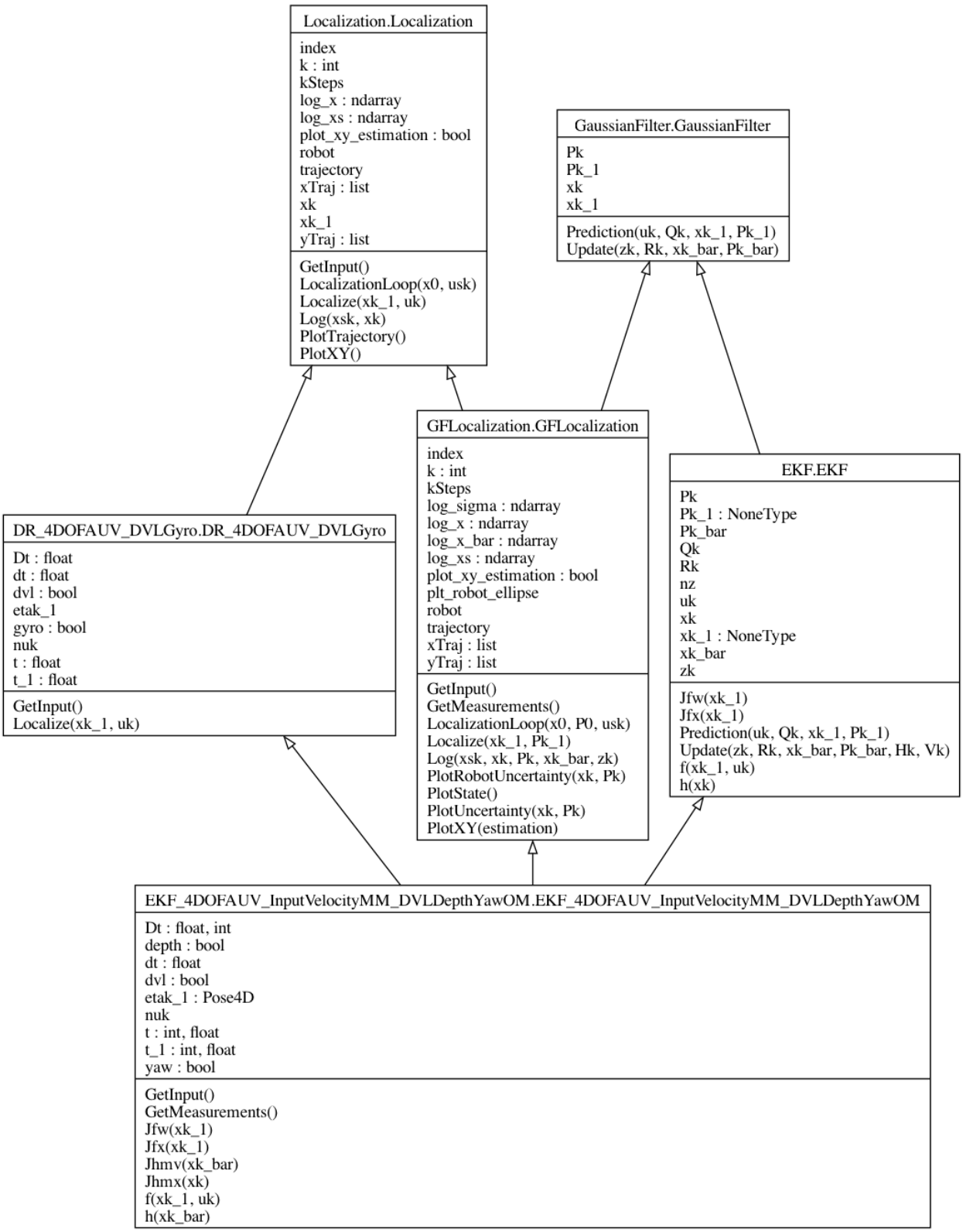
Bases: *GFLocalization*, *DR_4DOFAUV_DVLGyro*, *EKF*

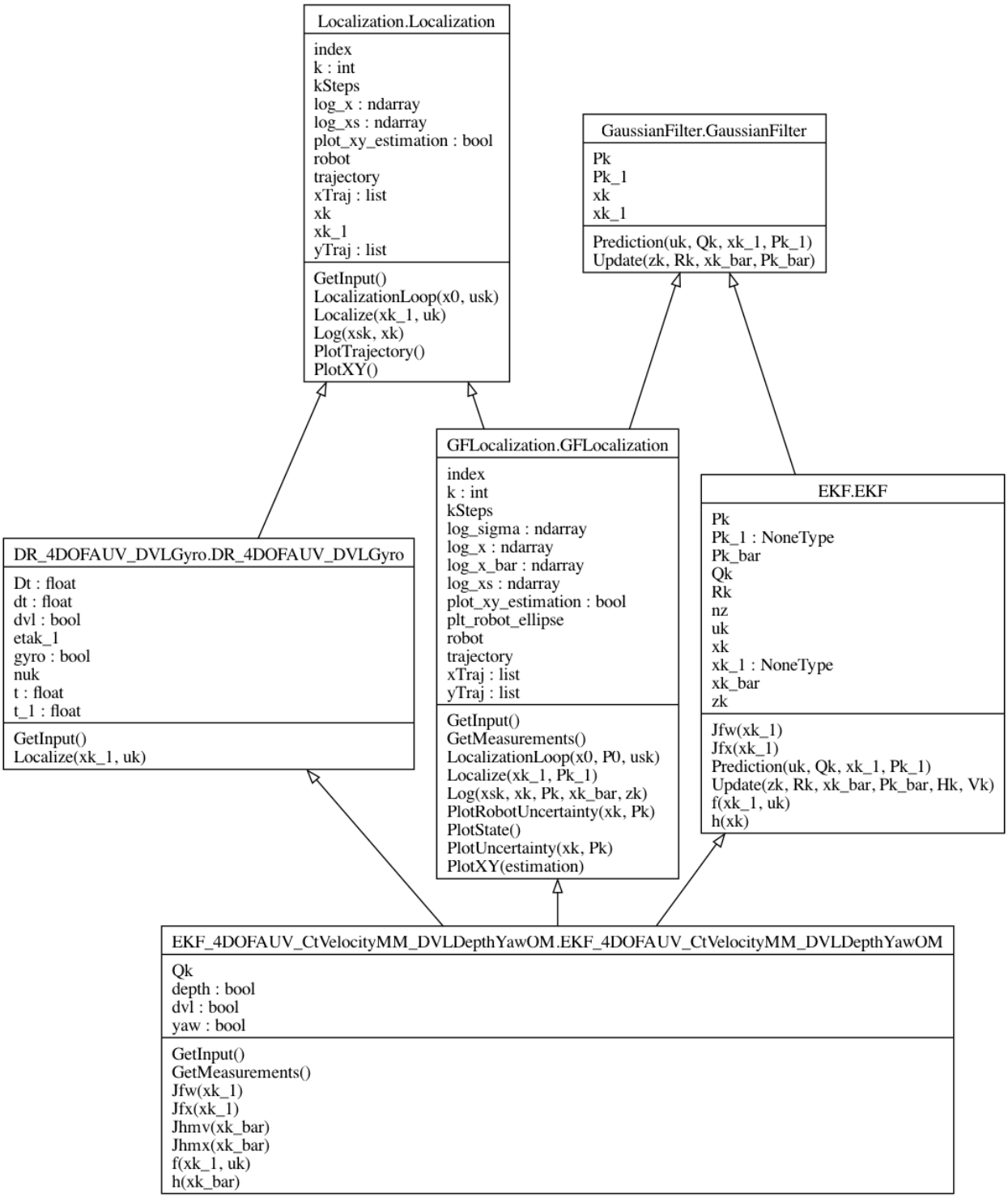
__init__(*kSteps*, *robot*, **args*)

Constructor.

Parameters

- **x0** – initial state


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro
Dt : float
dt : float
dvl : bool
etak_1
gyro : bool
nuk
t : float
t_1 : float
GetInput()
Localize(xk_1, uk)
EKF_4DOFAUV_InputVelocityMM_DVLDepthYawOM.EKF_4DOFAUV_InputVelocityMM_DVLDepthYawOM
Dt : float, int
depth : bool
dt : float
dvl : bool
etak_1 : Pose4D
nuk
t : int, float
t_1 : int, float
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
Jhmv(xk_bar)
Jhmx(xk)
f(xk_1, uk)
h(xk_bar)


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro
Dt : float
dt : float
dvl : bool
etak_1
gyro : bool
nuk
t : float
t_1 : float
GetInput()
Localize(xk_1, uk)
EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM.EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM
Qk
depth : bool
dvl : bool
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
Jhmv(xk_bar)
Jhmx(xk_bar)
f(xk_1, uk)
h(xk_bar)

- **P0** – initial covariance
- **index** – Named tuple used to relate the state vector, the simulation and the observation vectors (*IndexStruct.IndexStruct*)
- **kSteps** – simulation time steps
- **robot** – Simulated Robot object
- **args** – arguments to be passed to the parent constructor

f(*xk_1*, *uk*)

Motion model of the EKF **to be overwritten by the child class**.

Parameters

- **xk_1** – previous mean state vector
- **uk** – input vector

Return xk_bar, Pk_bar
predicted mean state vector and its covariance matrix

Jfx(*xk_1*)

Jacobian of the motion model with respect to the state vector. **Method to be overwritten by the child class**.

Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

Jfw(*xk_1*)

Jacobian of the motion model with respect to the noise vector. **Method to be overwritten by the child class**.

Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

h(*xk_bar*)

The observation model of the EKF is given by:

$$z_k = h(x_k, v_k) \tag{1.44}$$

This method computes the mean of this direct observation model. Therefore it does not depend on v_k since it is a zero mean Gaussian noise.

Parameters
xk – mean of the predicted state vector. By default it is taken from the class attribute.

Returns
expected observation vector

Jhmx(*xk_bar*)

Jhmv(*xk_bar*)

GetInput()

Returns

GetMeasurements()

Returns

3 DOF Differential Drive Mobile Robot

Differential Drive EKF Localization Using an Input displacement Motion Model

class EKF_3DOFDifferentialDriveInputDisplacement.**EKF_3DOFDifferentialDriveInputDisplacement**(*kSteps*, *robot*, *x0*, *P0*, **args*)

Bases: *GFLocalization*, *DR_3DOFDifferentialDrive*, *EKF*

This class implements an EKF localization filter for a 3 DOF Diffenteial Drive using an input displacement motion model incorporating yaw measurements from the compass sensor. It inherits from *GFLocalization.GFLocalization* to implement a localization filter, from the *DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive* class and, finally, it inherits from *EKF.EKF* to use the EKF Gaussian filter implementation for the localization.

__init__(*kSteps*, *robot*, *x0*, *P0*, **args*)

Constructor. Creates the list of *IndexStruct.IndexStruct* instances which is required for the automated plotting of the results. Then it defines the inital stawe vecto mean and covariance matrix and initializes the ancestor classes.

Parameters

- **kSteps** – number of iterations of the localization loop
- **robot** – simulated robot object
- **x0** – initial mean state vector
- **P0** – initial covariance matrix
- **args** – arguments to be passed to the base class constructor

GetInput()

Gets the input from the robot. Relates to the motion model as follows:

$$\begin{aligned} x_k &= f(x_{k-1}, u_k, w_k) \\ w_k &= N(0, Q_k) \end{aligned} \tag{1.45}$$

To be implemented by the student .

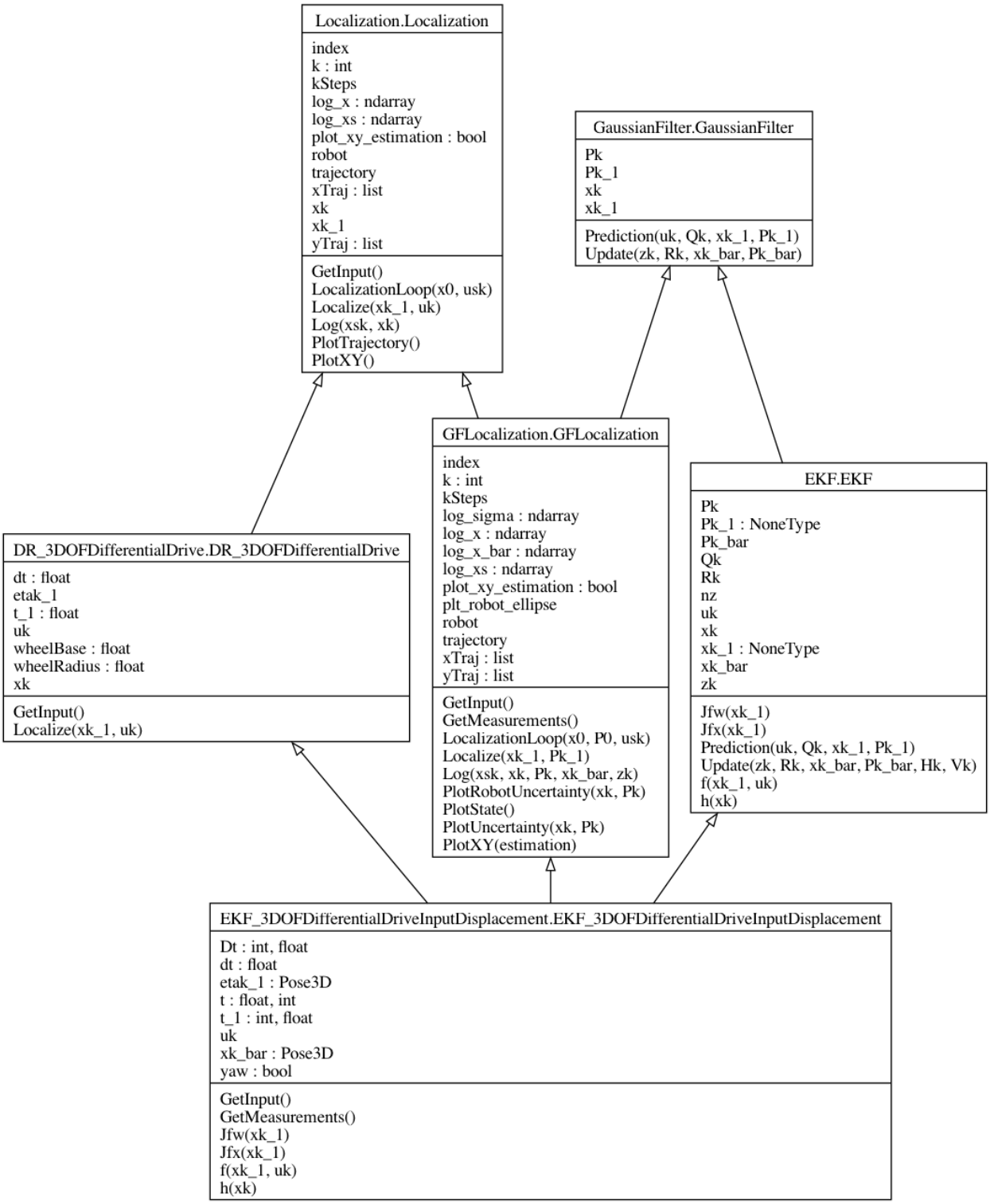
Return uk, Qk
input and covariance of the motion model

f(*xk_1*, *uk*)

Non-linear motion model.

$$^N\hat{\hat{x}}_k = f(^N\hat{x}_{k-1}, ^B u_k)$$

To be implemented by the student .


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)
EKF_3DOFDifferentialDriveInputDisplacement.EKF_3DOFDifferentialDriveInputDisplacement
Dt : int, float
dt : float
etak_1 : Pose3D
t : float, int
t_1 : int, float
uk
xk_bar : Pose3D
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
f(xk_1, uk)
h(xk)

Parameters

- **xk_1** – previous mean state vector.
- **uk** – Input vector.

Returns
xk_bar: predicted mean state vector.

Jfx(*xk_1*)

Jacobian of the motion model with respect to the state vector. **Method to be overwritten by the child class**.

To be implemented by the student .

Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

Jfw(*xk_1*)

Jacobian of the motion model with respect to the noise vector. **Method to be overwritten by the child class**.

To be implemented by the student .

Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

h(*xk*)

The observation model of the EKF is given by:

$$z_k = h(x_k, v_k) \tag{1.46}$$

This method computes the mean of this direct observation model. Therefore it does not depend on v_k since it is a zero mean Gaussian noise.

To be implemented by the student .

Parameters
xk – mean of the predicted state vector. By default it is taken from the class attribute.

Returns
expected observation vector

GetMeasurements()

Get the measurements from the robot. Corresponds to the observation model:

$$\begin{aligned} z_k &= h(x_k, v_k) \\ v_k &= N(0, R_k) \end{aligned} \tag{1.47}$$

To be implemented by the student .

Returns
zk, Rk, Hk, Vk: observation vector and covariance of the observation noise. Hk is the Observation matrix and Vk is the noise observation matrix.

Differential Drive EKF Localization Using a Constant Velocity Motion Model

class EKF_3DOFDifferentialDriveCtVelocity.**EKF_3DOFDifferentialDriveCtVelocity**(*kSteps*, *robot*, **args*)

Bases: *GFLocalization*, *DR_3DOFDifferentialDrive*, *EKF*

This class implements an EKF localization filter for a 3 DOF Differential Drive using a constant velocity motion model incorporating yaw measurements from the compass sensor and lineal velocity measurements computed from the odeometry. It inherits from *GFLocalization.GFLocalization* to implement a localization filter, from the *DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive* class and, finally, it inherits from *EKF.EKF* to use the EKF Gaussian filter implementation for the localization.

__init__(*kSteps*, *robot*, **args*)

Constructor. Creates the list of *IndexStruct.IndexStruct* instances which is required for the automated plotting of the results. Then it defines the inital state vector mean and covariance matrix and initializes the ancestor classes.

Parameters

- **kSteps** – number of iterations of the localization loop
- **robot** – simulated robot object
- **args** – arguments to be passed to the base class constructor

f(*xk_1*, *uk*)

Non-linear motion model.

$$^{N}\hat{\hat{x}}_k = f(^{N}\hat{x}_{k-1}, ^{B}u_k)$$

To be implemented by the student .

Parameters

- **xk_1** – previous mean state vector.
- **uk** – Input vector.

Returns
xk_bar: predicted mean state vector.

Jfx(*xk_1*)

Jacobian of the motion model with respect to the state vector. **Method to be overwritten by the child class**.

To be implemented by the student .

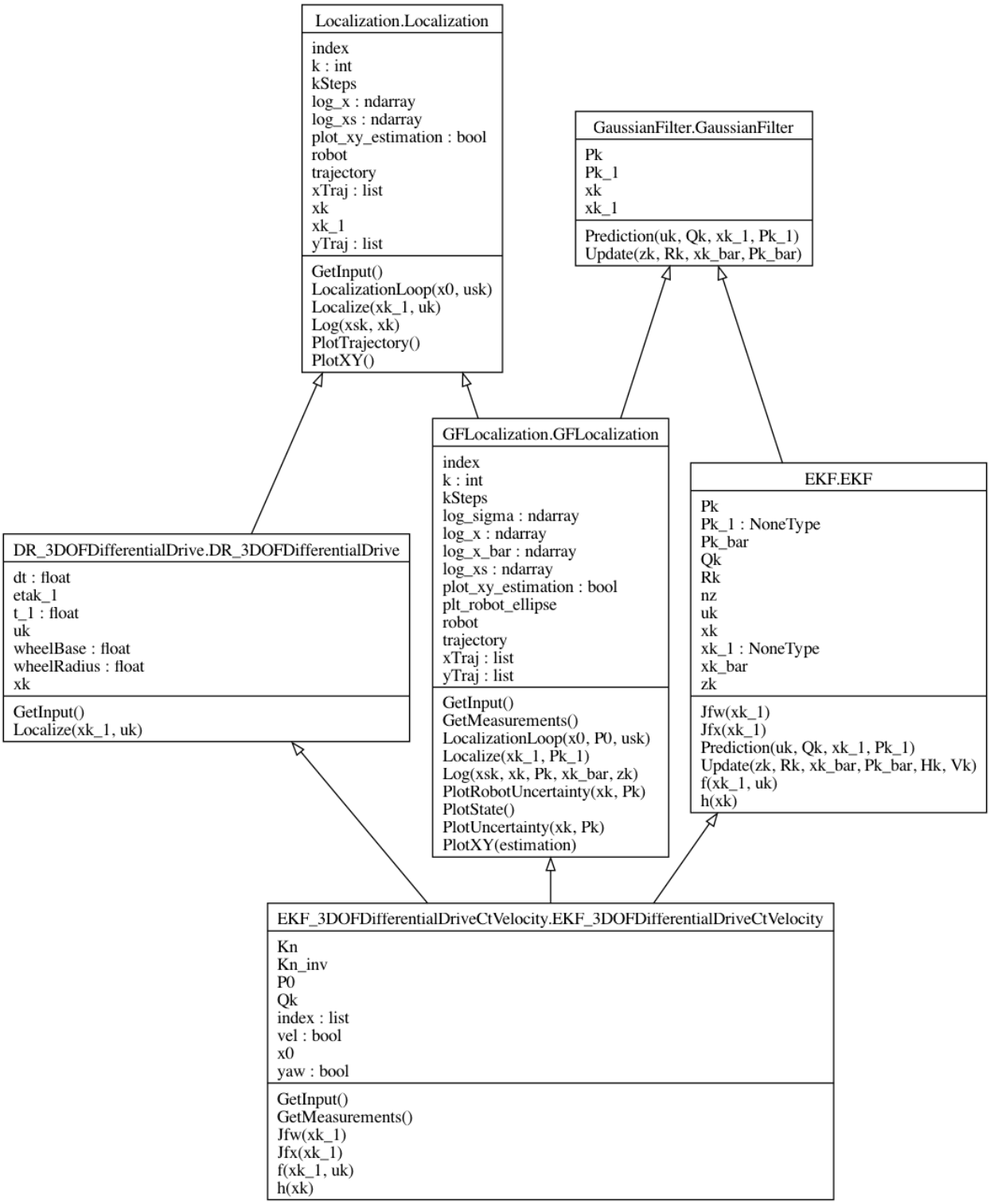
Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

Jfw(*xk_1*)

Jacobian of the motion model with respect to the noise vector. **Method to be overwritten by the child class**.

To be implemented by the student .


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)
EKF_3DOFDifferentialDriveCtVelocity.EKF_3DOFDifferentialDriveCtVelocity
Kn
Kn_inv
P0
Qk
index : list
vel : bool
x0
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
f(xk_1, uk)
h(xk)

Parameters
xk_1 – Linearization point. By default the linearization point is the previous state vector taken from a class attribute.

Returns
Jacobian matrix

h(*xk*)

The observation model of the EKF is given by:

$$z_k = h(x_k, v_k) \tag{1.48}$$

This method computes the mean of this direct observation model. Therefore it does not depend on v_k since it is a zero mean Gaussian noise.

To be implemented by the student .

Parameters
xk – mean of the predicted state vector. By default it is taken from the class attribute.

Returns
expected observation vector

GetInput()

Gets the input from the robot. Relates to the motion model as follows:

$$\begin{aligned} x_k &= f(x_{k-1}, u_k, w_k) \\ w_k &= N(0, Q_k) \end{aligned} \tag{1.49}$$

To be implemented by the student .

Return uk, Qk
input and covariance of the motion model

GetMeasurements()

Get the measurements from the robot. Corresponds to the observation model:

$$\begin{aligned} z_k &= h(x_k, v_k) \\ v_k &= N(0, R_k) \end{aligned} \tag{1.50}$$

To be implemented by the student .

Returns
zk, Rk, Hk, Vk: observation vector and covariance of the observation noise. Hk is the Observation matrix and Vk is the noise observation matrix.

1.6.6 Feature Map based EKF Localization

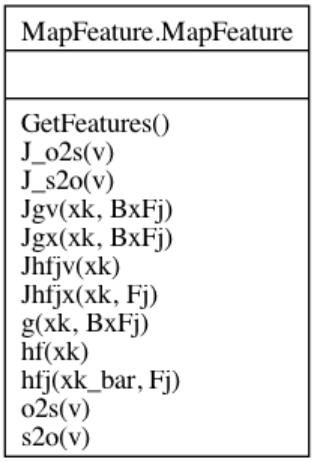
Feature Map based EKF Localization

Map Feature

class MapFeature.**MapFeature**(**args*)

Bases: object

This class provides the functionality required to use Map Features for Robot Localization. It has methods for reading the feature pose using the robot the sensors (*GetFeatures()*), as well as for computing its:



- observation model (*hf()*),
- inverse observation model (*g()*)
- and all the required Jacobians (`Jhfx()`, `Jhfv()`, *Jgx()* and *Jgv()*).

When mapped, a feature may involve 2 different representations:

- **The observation representation:** which is the representation used by the sensors to observe the feature.
- **The storage representation:** which is the representation used to store the feature in the map within the state vector.

For instance, a feature may be observed in polar coordinates but stored in Cartesian coordinates. In this case, the observation representation is the Polar coordinates and the storage representation is the Cartesian coordinates. The class provides methods to convert from one representation to the other (*s2o()* and *o2s()*) and their corresponding Jacobians (*J_s2o()* and *J_o2s()*). By default, the observation representation is the same as the storage representation, but this behaviour may be overriden in child classes.

__init__(**args*)

GetFeatures()

Reads the Feature observations from the sensors. For all features within the field of view of the sensor, the method returns the list of robot-related poses and the covariance of their corresponding observation noise. **This is a pure virtual method that must be overriden in child classes**.

Returns

vector of features observations in the B-Frame and the covariance of their corresponding noise.

- $z_k = [{}^B x_{F_i}^T \cdots {}^B x_{F_j}^T \cdots {}^B x_{F_k}^T]^T$
- $R_k = block_diag([R_{F_i} \cdots R_{F_j} \cdots R_{F_k}])$

hf(*xk*)

This is the direct observation model, implementing the feature observation equation for the data association hypothesis $\mathcal{H}$, the features observation vector $z_f, thestatevector : math : `x_k$, and the observation noise

v_k:

$$\begin{aligned} \mathcal{H} &= [F_a \ \cdots \ F_b \ \cdots \ F_c] \\ z_f &= [z_{f_1}^T \ \cdots \ z_{f_i}^T \ \cdots \ z_{f_{n_{zf}}}^T]^T \\ x_k &= [{}^N x_B^T \ x_{rest}^T]^T \\ v_k &= [v_{f1_k}^T \ \cdots \ v_{fi_k}^T \ \cdots \ v_{fn_{zf_k}}^T]^T \\ z_f &= h_f(x_k, v_k) \end{aligned} \tag{1.51}$$

which may be expanded as follows:

$$\begin{bmatrix} z_{f_1} \\ \vdots \\ z_{f_i} \\ \vdots \\ z_{n_{zf}} \end{bmatrix} = \begin{bmatrix} h_{F_a}(x_k, v_k) \\ \vdots \\ h_{F_b}(x_k, v_k) \\ \vdots \\ h_{Fc}(x_k, v_k) \end{bmatrix} = \begin{bmatrix} s2o(\ominus^N x_B \boxplus^N x_{F_a}) + v_{f1_k} \\ \vdots \\ s2o(\ominus^N x_B \boxplus^N x_{F_b}) + v_{fi_k} \\ \vdots \\ s2o(\ominus^N x_B \boxplus^N x_{F_c}) + v_{fn_{zf}} \end{bmatrix} \tag{1.52}$$

being $h_{F_j}(\cdot)$ (*hfj()*) the observation function (eq. (**??**)) for the data association hypothesis $\mathcal{H}$ and *s2o()* the conversion function from the storage representation to the observation one.

The method computes the expected observations h_f for the observed features contained within the z_f features observation vector. To do it, it iterates over each feature observation z_{f_i} calling the method *hfj()* for its corresponding associated feature $\mathcal{H}_i = F_j$ to compute the expected observation h_{F_j}, collecting all them in the returned vector.

Parameters
xk – state vector mean $\hat{x}_k$.

Returns
vector of expected features observations corresponding to the vector of observed features z_f.

hfj(*xk_bar*, *Fj*)

This is the direct observation model for a single feature observation z_{f_i} , so it implements its related observation function (see eq. (**??**)). For a single feature observation z_{f_i} of the feature ${}^N x_{F_{H_i}}$ the method computes its expected observation from the current robot pose ${}^N x_B$. This function uses a generic implementation through the following equation:

$$z_{f_i} = h_{F_j}(x_k, v_k) = s2o(\ominus^N x_B \boxplus^N x_{F_j}) + v_{fi_k} \tag{1.53}$$

Where ${}^N x_B$ is the robot pose included within the state vector ($x_k = [{}^N x_B^T \ x_{rest}^T]^T$) and *s2o()* is a conversion function from the store representation to the observation representation.

The method is called by *hf()* to compute the expected observation for each feature observation contained in the observation vector $z_f = [z_{f_1}^T \ \cdots \ z_{f_i}^T \ \cdots \ z_{f_{n_z f}}^T]^T$.

Parameters

- **xk_bar** – mean of the predicted state vector
- **Fj** – map index of the observed feature: ${}^N x_{F_j} = self.M[Fj]$

Returns
expected observation of the feature ${}^N x_{F_j}$

Jhfjx(*xk*, *Fj*)

Jacobian of the single feature direct observation model *hfj()* (eq. (**??**)) with respect to the state vector $\bar{x}_k$:

$$\begin{aligned} x_k &= [{}^N x_B^T \; x_{rest}^T]^T \\ {}^N x_B &= F \cdot x_k; F = \begin{bmatrix} I_{p\times p} & 0_{p\times np} \end{bmatrix} \\ J_{hfjx} &= \frac{\partial h_{F_j}(x_k, v_k)}{\partial x_k} = \frac{\partial s2o(\ominus F \cdot x_k \boxplus {}^N x_{F_j}) + v_{fi_k}}{\partial x_k} \\ &= J_{s2o}(\ominus {}^N x_B \boxplus {}^N x_{F_j}) J_{1\boxplus}(\ominus {}^N x_B, {}^N x_{F_j}) J_{\ominus}({}^N x_B) F \end{aligned} \tag{1.54}$$

where p is the dimension of the robot pose ${}^N x_B$ and np is the dimension of the rest of the state vector x_{rest}.

Parameters

- **xk** – state vector mean
- **Fj** – map index of the observed feature

Returns
Jacobian matrix defined in eq. (**??**)

Jhfjv(*xk*)

Jacobian of the single feature direct observation model *hfj()* (eq. (**??**)) with respect to the feature observation noise vector v_k:

$$\begin{aligned} x_k &= [{}^N x_B^T \; x_{rest}^T]^T \\ {}^N x_B &= F \cdot x_k; F = \begin{bmatrix} I_{p\times p} & 0_{p\times np} \end{bmatrix} \\ J_{hfjx} &= \frac{\partial h_{F_j}(x_k, v_k)}{\partial v_k} = \frac{\partial s2o(\ominus F \cdot x_k \boxplus {}^N x_{F_j}) + v_{fi_k}}{\partial v_k} \\ &= I_{zfi_dim \times zfi_dim} \end{aligned} \tag{1.55}$$

where p is the dimension of the robot pose ${}^N x_B$ and np is the dimension of the rest of the state vector x_{rest} and zfi_dim is the dimension of a single feature observation.

Parameters
xk – state vector mean $\hat{x}_k$.

Returns
Jacobian matrix defined in eq. (**??**)

g(*xk*, *BxFj*)

This method provides a generic implementation of the inverse observation model. It computes the feature pose in the N-Frame ${}^N x_{F_j}$ by compounding the robot pose ${}^N x_B$, from where the observation was taken, with the B-Frame referenced feature observation ${}^B x_{F_j}$:

$${}^N x_{F_j} = {}^N x_B \boxplus o2s({}^B x_{F_j} + v_k) \tag{1.56}$$

In this case, *o2s()* is the conversion function converting from the observation space to the representation one. It is worth noting that the robot pose ${}^N x_B$ is included within the state vector x_k but might not be the whole state vector. For instance, in some cases the state vector may include as well the robot velocity $x_k = [{}^N x_B^T \; {}^B \nu_k^T]^T$. Note that the *g()* works with a single feature observation, instead than with a vector of feature observations.

Parameters

- **xk** – mean state vector containing the robot pose ${}^N x_B$ from where the observation was taken
- **BxFj** – feature observation in the B-Frame ${}^B x_{F_j}$

Returns
mean feature pose in the N-Frame ${}^{N}x_{F_j}$

Jgx(*xk*, *BxFj*)

Jacobian of the inverse observation model `g()`, with respect to the state vector x_k. According to the generic implementation of the inverse observation model `g()` eq. (**??**), the Jacobian is computed as follows:

$$J_{gx} = \frac{\partial g({}^{N}x_B, {}^{B}x_{F_j}, v_k)}{\partial x_k} = \begin{bmatrix} J_{1\boxplus}({}^{N}x_B, o2s({}^{B}x_{F_j})) & 0 \end{bmatrix} \tag{1.57}$$

The zero submatrix, if present, corresponds to the derivate with respect to the non-positional elements of the state vector, for instance the robot velocity ${}^{B}\nu_k$, in case this was included within the state vector. If the state vector only containes the pose, then the 0 submatix vanishes.

Parameters
xk_bar – predicted state vector

Returns
Jacobian of the inverse observation model `g()` with respect to the state vector (eq. (**??**))

Jgv(*xk*, *BxFj*)

Jacobian of the inverse observation model `g()`, with respect to the observation noise v_k. According to the generic implementation of the inverse observation model `g()` eq. (**??**), the Jacobian is computed as follows:

$$J_{gv} = \frac{\partial g({}^{N}x_k, {}^{B}x_{F_j}, v_k)}{\partial v_k} = J_{2\boxplus}({}^{N}x_B, o2s({}^{B}x_{F_j}))J_{o2s}({}^{B}x_{F_j}) \tag{1.58}$$

Parameters

- **xk** – state vector containing the robot pose ${}^{N}x_B$ from where the observation was taken
- **BxFj** – feature observation in the B-Frame ${}^{B}x_{F_j}$

Returns
Jacobian of the inverse observation model `g()` with respect to the observation noise J_{gv} (see eq. (**??**)).

s2o(*v*)

Conversion function from the storage representation to the observation representation. Returns the same vector as the one provided as input, assuming that the observation representation and the storage representation are both of them in the same coordinates.

Parameters
v – vector in the storage representation

Returns
vector in the observation representation. Since it is assuming that the storage and observation representations

are in the same coordinates, it returns the same vector as the one provided as input.

o2s(*v*)

Conversion function from the observation representation to the storage representation. Returns the same vector as the one provided as input, assuming that the observation representation and the storage representation are both of them in the same coordinates.

Parameters
v – vector in the observation representation

Returns
vector in the storage representation. Since it is assuming that the storage and observation representations

are in the same coordinates, it returns the same vector as the one provided as input.

`J_s2o`(v)

Jacobian of the conversion function from the storage representation to the observation representation. Returns the identity matrix, assuming that the observation representation and the storage representation are both of them in the same coordinates.

Parameters
v – vector in the storage representation

Returns
Jacobian of the conversion function from the storage representation to the observation representation.

Since it is assuming that the storage and observation representations are in the same coordinates, it returns the identity matrix.

`J_o2s`(v)

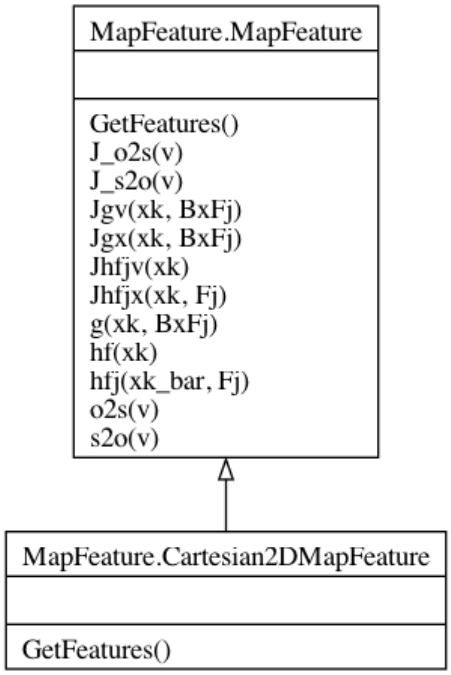
Jacobian of the conversion function from the observation representation to the storage representation. Returns the identity matrix, assuming that the observation representation and the storage representation are both of them in the same coordinates.

Parameters
v – vector in the observation representation

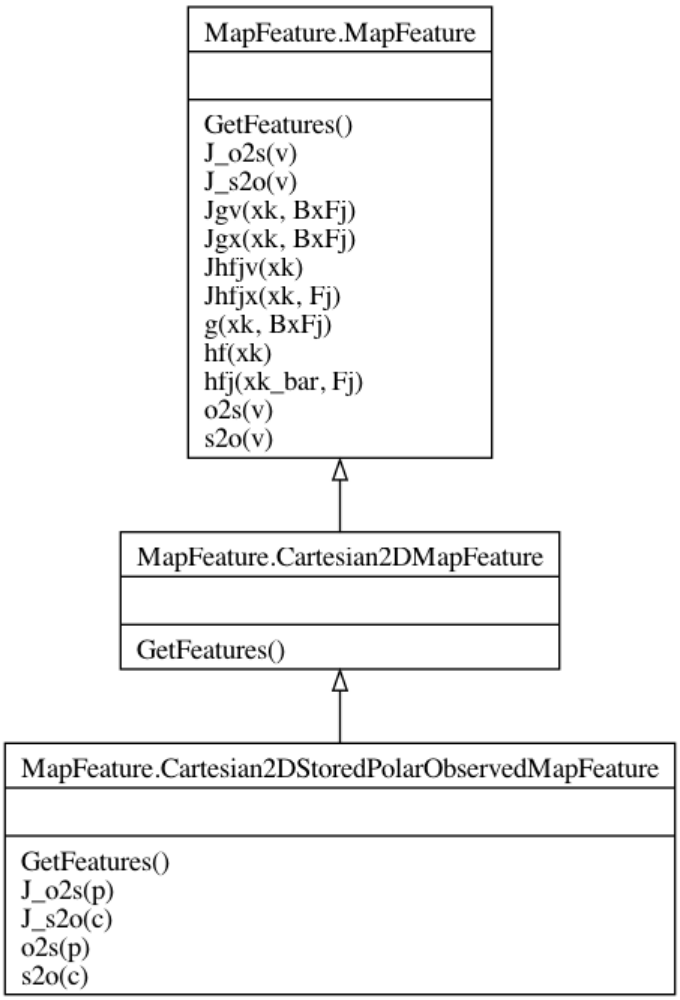
Returns
Jacobian of the conversion function from the observation representation to the storage representation.

Since it is assuming that the storage and observation representations are in the same coordinates, it returns the identity matrix.

Cartesian Map Feature



Cartesian Map Feature Observed in Polar Coordinates



Feature Map based EKF Localization

class FEKFMBL.**FEKFMBL**(*M*, *alpha*, **args*)

Bases: *GFLocalization*, *MapFeature*, *EKF*

Feature Extended Kalman Filter Map based Localization class. Inherits from *GFLocalization.GFLocalization* and *MapFeature.MapFeature*. The first one provides the basic functionality of a localization algorithm, while the second one provides the basic functionality required to use features. *FEKFMBL.FEKFMBL* extends those classes by adding functionality to use a map based on features.

xB = -2

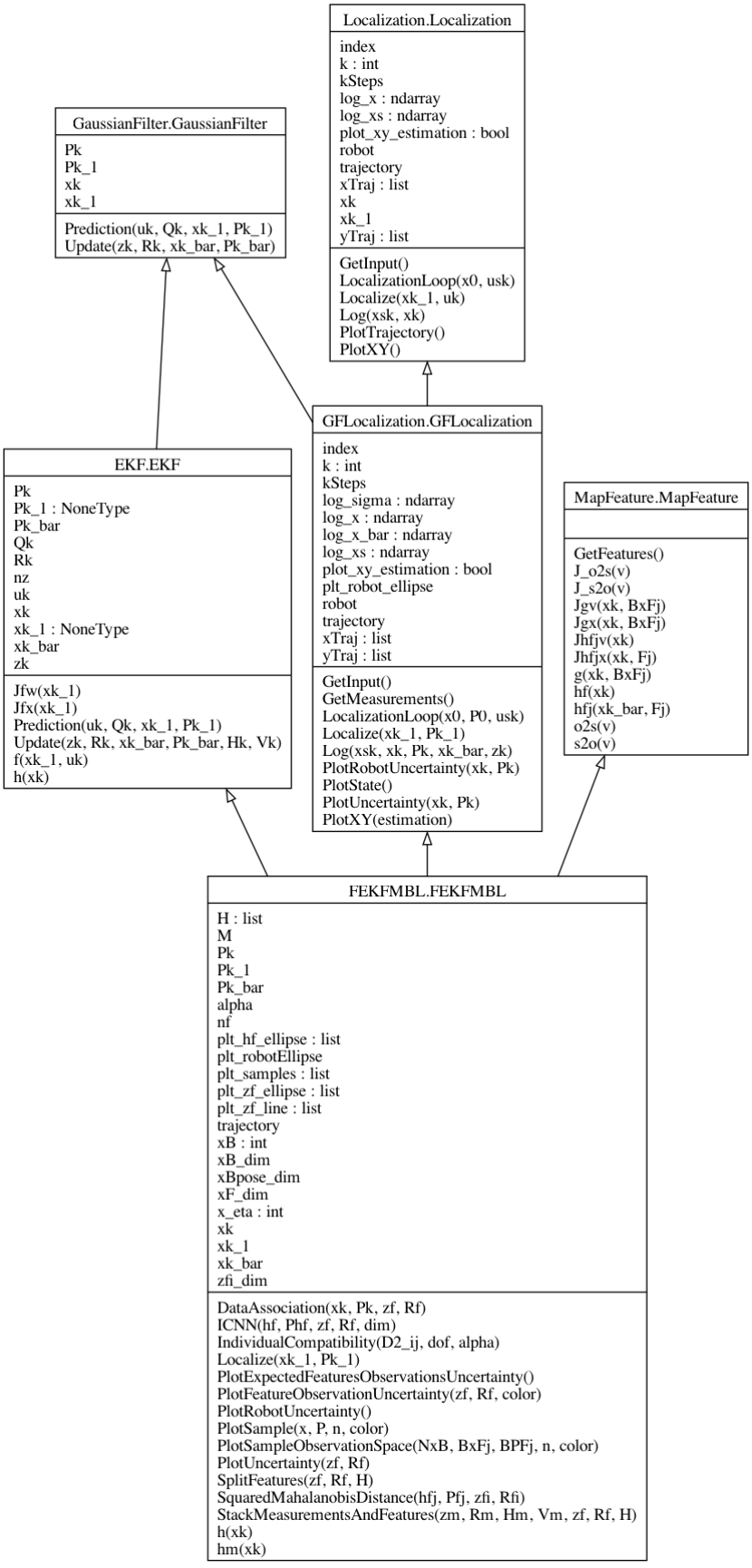
x_eta = -1

__init__(*M*, *alpha*, **args*)

Constructor of the FEKFMBL class.

Parameters

- **M** – Feature Based Map $M = [{}^{N}x_{F_1}^{T} \ ... \ {}^{N}x_{F_{n_f}}^{T}]^{T}$


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
MapFeature.MapFeature
GetFeatures()
J_o2s(v)
J_s2o(v)
Jgv(xk, BxFj)
Jgx(xk, BxFj)
Jhfjv(xk)
Jhfjx(xk, Fj)
g(xk, BxFj)
hf(xk)
hfj(xk_bar, Fj)
o2s(v)
s2o(v)
FEKFMBL.FEKFMBL
H : list
M
Pk
Pk_1
Pk_bar
alpha
nf
plt_hf_ellipse : list
plt_robotEllipse
plt_samples : list
plt_zf_ellipse : list
plt_zf_line : list
trajectory
xB : int
xB_dim
xBpose_dim
xF_dim
x_eta : int
xk
xk_1
xk_bar
zfi_dim
DataAssociation(xk, Pk, zf, Rf)
ICNN(hf, Phf, zf, Rf, dim)
IndividualCompatibility(D2_ij, dof, alpha)
Localize(xk_1, Pk_1)
PlotExpectedFeaturesObservationsUncertainty()
PlotFeatureObservationUncertainty(zf, Rf, color)
PlotRobotUncertainty()
PlotSample(x, P, n, color)
PlotSampleObservationSpace(NxB, BxFj, BPFj, n, color)
PlotUncertainty(zf, Rf)
SplitFeatures(zf, Rf, H)
SquaredMahalanobisDistance(hfj, Pfj, zfi, Rfi)
StackMeasurementsAndFeatures(zm, Rm, Hm, Vm, zf, Rf, H)
h(xk)
hm(xk)

- **alpha** – Chi2 tail probability. Confidence interaval of the individual compatibility test
- **args** – arguments to be passed to the EKFLocalization constructor

h(*xk*)

We do differenciate two types of observations:

- Measurements: :math:`z_m`corresponds to observations of the state vector (position, velocity, etc...)
- Feature Observations: z_f correspond to observations of the features (CartesianFeature, PolarFeature, SphericalFeature, etc...).

This method implements the full direct observation model including the measurements and feature observations:

$$z_k = h(x_k, v_k) \Rightarrow \begin{bmatrix} z_m \\ z_f \end{bmatrix} = \begin{bmatrix} h_m(x_k, v_m) \\ h_f(x_k, v_f) \end{bmatrix} \; ; \; v_k = [v_m^T \; v_f^T]^T \tag{1.59}$$

This method calls **h_m()** to compute the expected measurements and the *MapFeature.MapFeature.hf()* method to compute the expected feature observations. The method returns a stacked vector of expected measurements and feature observations.

Parameters
xk – mean state vector used as linearization point

Returns
Joint stacked vector of the expected measurement and feature observations

hm(*xk*)

Measurement observation model. This method computes the expected measurements $h_m(x_k, v_m)$ given the mean state vector x_k and the measurement noise v_m. It is implemented by calling to the ancestor class *EKF.EKF.h()* method.

Parameters
xk – mean state vector.

Returns
expected measruments.

SquaredMahalanobisDistance(*hfj*, *Pfj*, *zfi*, *Rfi*)

Computes the squared Mahalanobis distance between the expected feature observation hf_j and the feature observation z_{f_i}.

Parameters

- **hfj** – expected feature observation
- **Pfj** – expected feature observation covariance
- **zfi** – feature observation
- **Rfi** – feature observation covariance

Returns
Squared Mahalanobis distance between the expected feature observation hf_j and the feature observation z_{f_i}

IndividualCompatibility(*D2_ij*, *dof*, *alpha*)

Computes the individual compatibility test for the squared Mahalanobis distance D_{ij}^2. The test is performed using the Chi-Square distribution with dof degrees of freedom and a significance level α.

Parameters

- **D2_ij** – squared Mahalanobis distance

- **dof** – number of degrees of freedom
- **alpha** – confidence level

Returns
bolean value indicating if the Mahalanobis distance is smaller than the threshold defined by the confidence level

ICNN(*hf*, *Phf*, *zf*, *Rf*, *dim*)

Individual Compatibility Nearest Neighbor (ICNN) data association algorithm. Given a set of expected feature observations h_f and a set of feature observations z_f, the algorithm returns a pairing hypothesis H that associates each feature observation z_{f_i} with the expected feature observation h_{f_j} that minimizes the Mahalanobis distance D^2_{ij}.

Parameters

- **hf** – vector of expected feature observations
- **Phf** – Covariance matrix of the expected feature observations
- **zf** – vector of feature observations
- **Rf** – Covariance matrix of the feature observations
- **dim** – feature dimensionality

Returns
The vector of asociation hypothesiss

DataAssociation(*xk*, *Pk*, *zf*, *Rf*)

Data association algorithm. Given state vector (x_k and P_k) including the robot pose and a set of feature observations z_f and its covariance matrices R_f, the algorithm computes the expected feature observations h_f and its covariance matrices P_f. Then it calls an association algorithms like *ICNN()* (JCBB, etc.) to build a pairing hypothesis associating the observed features z_f with the expected features observations h_f.

The vector of association hypothesis H is stored in the H attribute and its dimension is the number of observed features within z_f. Given the jth feature observation z_{f_j}, *self.H[j]=i* means that z_{f_j} has been associated with the ith feature . If *self.H[j]=None* means that z_{f_j} has not been associated either because it is a new observed feature or because it is an outlier.

Parameters

- **xk** – mean state vector including the robot pose
- **Pk** – covariance matrix of the state vector
- **zf** – vector of feature observations
- **Rf** – Covariance matrix of the feature observations

Returns
The vector of asociation hypothesiss

Localize(*xk_1*, *Pk_1*)

Localization iteration. Reads the input of the motion model, performs the prediction step (*EKF.EKF.Prediction()*), reads the measurements and the features, solves the data association calling *DataAssociation()* and the performs the update step (*EKF.EKF.Update()*) and logs the results. The method also plots the uncertainty ellipse (*PlotUncertainty()*) of the robot pose, the feature observations and the expected feature observations.

Parameters

- **xk_1** – previous state vector

- **Pk_1** – previous covariance matrix

Return xk, Pk
updated state vector and covariance matrix

StackMeasurementsAndFeatures(*zm*, *Rm*, *Hm*, *Vm*, *zf*, *Rf*, *H*)

Given the vector of measurements observations z_m together with their covariance matrix R_m, the vector of feature observations z_f together with their covariance matrix R_f, The measurement observation matrix H_m, the measurement observation noise matrix V_m and the vector of feature associations H, this method returns the joint observation vector z_k, its related covariance matrix R_k, the stacked Observation matrix H_k, the stacked noise observation matrix V_k, the vector of non-paired features z_{np} and its noise covariance matrix R_{np}. It is assumed that the measurements and the features observations are independent, therefore the covariance matrix of the joint observation vector is a block diagonal matrix.

Parameters

- **zm** – measurement observations vector
- **Rm** – covariance matrix of the measurement observations
- **Hm** – measurement observation matrix
- **Vm** – measurement observation noise matrix
- **zf** – feature observations vector
- **Rf** – covariance matrix of the feature observations
- **H** – features associations vector

Returns
vector of joint measurement and feature observations z_k and its covariance matrix R_k

SplitFeatures(*zf*, *Rf*, *H*)

Given the vector of feature observations z_f and their covariance matrix R_f, and the vector of feature associations H, this function returns the vector of paired feature observations z_p together with its covariance matrix R_p, and the vector of non-paired feature observations z_{np} together with its covariance matrix R_{np}. The paired observations will be used to update the filter, while the non-paired ones will be considered as outliers. In the case of SLAM, they become new feature candidates.

Parameters

- **zf** – vector of feature observations
- **Rf** – covariance matrix of feature observations
- **H** – hypothesis of feature associations

Returns
vector of paired feature observations z_p, covariance matrix of paired feature observations R_p, vector of non-paired feature observations z_{np}, covariance matrix of non-paired feature observations R_{np}.

PlotFeatureObservationUncertainty(*zf*, *Rf*, *color*)

Plots the uncertainty ellipse of the feature observations. This method is called by *FEKFMBL.PlotUncertainty()*.

Parameters

- **zf** – vector of feature observations
- **Rf** – covariance matrix of the feature observations

PlotExpectedFeaturesObservationsUncertainty()

For all features in the map, this method plots the uncertainty ellipse of the expected feature observations. This method is called by *FEKFMBL.PlotUncertainty()*.

PlotSampleObservationSpace(*NxB*, *BxFj*, *BPFj*, *n*, *color='r.'*)

Plots n samples from a Gaussian distribution with mean x and covariance P. This method is called by *FEKFMBL.PlotUncertainty()*. This is a method for testing. It can be used to compare the uncertainty ellipse with the samples.

Parameters

- **x** – mean of the Gaussian distribution
- **P** – covariance of the Gaussian distribution
- **n** – number of samples
- **color** – color of the samples

PlotSample(*x*, *P*, *n*, *color='r.'*)

Plots n samples from a Gaussian distribution with mean x and covariance P. This method is called by *FEKFMBL.PlotUncertainty()*. This is a method for testing. It can be used to compare the uncertainty ellipse with the samples.

Parameters

- **x** – mean of the Gaussian distribution
- **P** – covariance of the Gaussian distribution
- **n** – number of samples
- **color** – color of the samples

PlotRobotUncertainty()

Plots the robot trajectory and its uncertainty ellipse. This method is called by *FEKFMBL.PlotUncertainty()*.

PlotUncertainty(*zf*, *Rf*)

Plots the uncertainty ellipses of the robot pose (*PlotRobotUncertainty()*), the feature observations (*PlotFeatureObservationUncertainty()*) and the expected feature observations (*PlotExpectedFeaturesObservationsUncertainty()*). This method is called by *FEKFMBL.Localize()* at the end of a localization iteration in order to update the online visualization.

Parameters

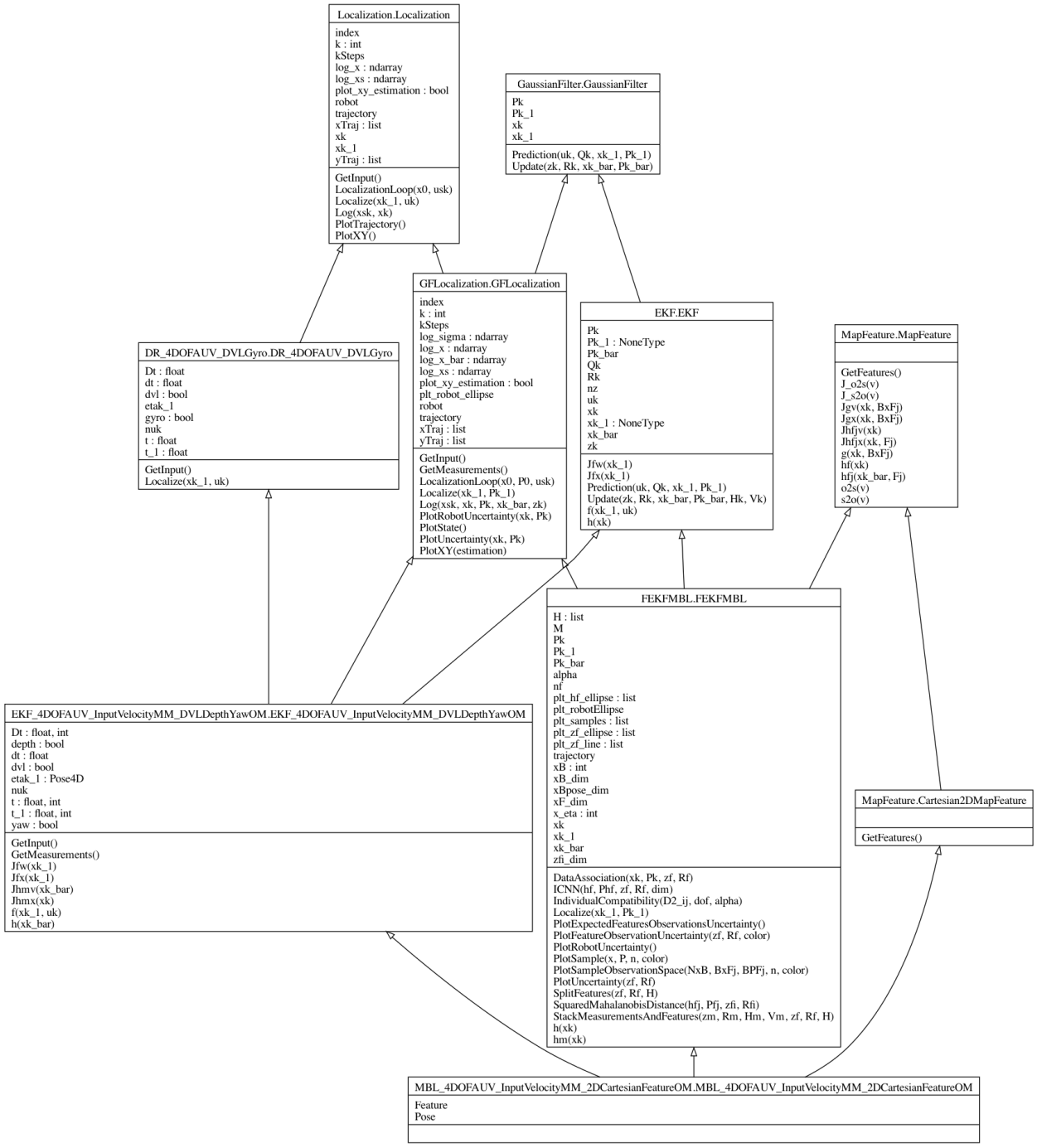
- **zf** – vector of feature observations
- **Rf** – covariance matrix of the feature observations

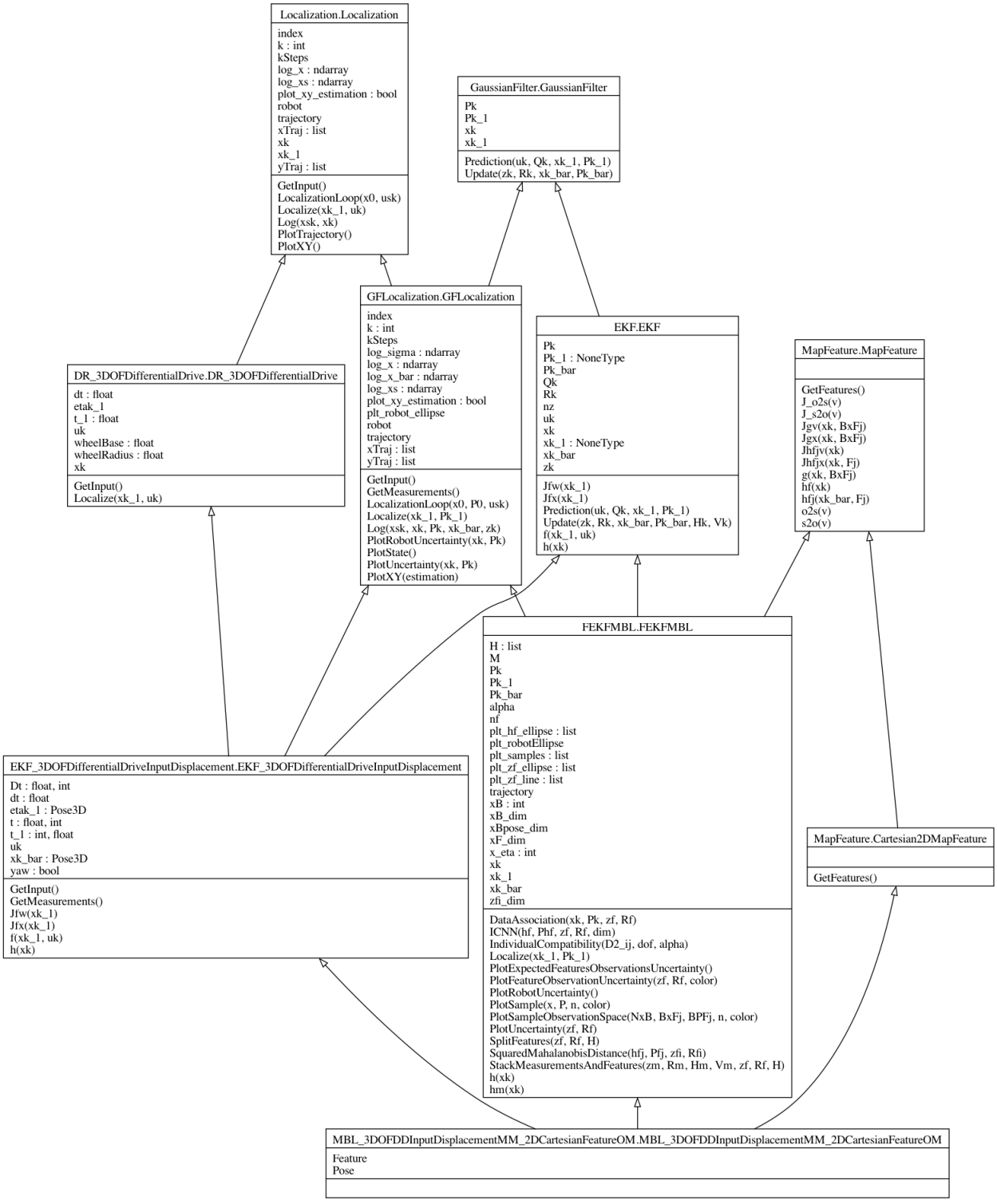
4 DOF AUV

4 DOF AUV Map based EKF Localization using an Input Velocity Motion Model with Depht, Yaw and Linear Velocity Measurements, and a 2D Cartesian Feature Observation Model

3 DOF Differential Drive Mobile Robot

Differential Drive EKF Map Based Localization EKF Using an Input displacement Motion Model and 2D Cartesian Feature Observation Model


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
MapFeature.MapFeature
GetFeatures()
J_o2s(v)
J_s2o(v)
Jgv(xk, BxFj)
Jgx(xk, BxFj)
Jhfjv(xk)
Jhfjx(xk, Fj)
g(xk, BxFj)
hf(xk)
hfj(xk_bar, Fj)
o2s(v)
s2o(v)
DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro
Dt : float
dt : float
dvl : bool
etak_1
gyro : bool
nuk
t : float
t_1 : float
GetInput()
Localize(xk_1, uk)
FEKFMBL.FEKFMBL
H : list
M
Pk
Pk_1
Pk_bar
alpha
nf
plt_hf_ellipse : list
plt_robotEllipse
plt_samples : list
plt_zf_ellipse : list
plt_zf_line : list
trajectory
xB : int
xB_dim
xBpose_dim
xF_dim
x_eta : int
xk
xk_1
xk_bar
zfi_dim
DataAssociation(xk, Pk, zf, Rf)
ICNN(hf, Phf, zf, Rf, dim)
IndividualCompatibility(D2_ij, dof, alpha)
Localize(xk_1, Pk_1)
PlotExpectedFeaturesObservationsUncertainty()
PlotFeatureObservationUncertainty(zf, Rf, color)
PlotRobotUncertainty()
PlotSample(x, P, n, color)
PlotSampleObservationSpace(NxB, BxFj, BPFj, n, color)
PlotUncertainty(zf, Rf)
SplitFeatures(zf, Rf, H)
SquaredMahalanobisDistance(hfj, Pfj, zfi, Rfi)
StackMeasurementsAndFeatures(zm, Rm, Hm, Vm, zf, Rf, H)
h(xk)
hm(xk)
EKF_4DOFAUV_InputVelocityMM_DVLDepthYawOM.EKF_4DOFAUV_InputVelocityMM_DVLDepthYawOM
Dt : float, int
depth : bool
dt : float
dvl : bool
etak_1 : Pose4D
nuk
t : float, int
t_1 : float, int
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
Jhmv(xk_bar)
Jhmx(xk)
f(xk_1, uk)
h(xk_bar)
MapFeature.Cartesian2DMapFeature
GetFeatures()
MBL_4DOFAUV_InputVelocityMM_2DCartesianFeatureOM.MBL_4DOFAUV_InputVelocityMM_2DCartesianFeatureOM
Feature
Pose


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
MapFeature.MapFeature
GetFeatures()
J_o2s(v)
J_s2o(v)
Jgv(xk, BxFj)
Jgx(xk, BxFj)
Jhfjv(xk)
Jhfjx(xk, Fj)
g(xk, BxFj)
hf(xk)
hfj(xk_bar, Fj)
o2s(v)
s2o(v)
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)
FEKFMBL.FEKFMBL
H : list
M
Pk
Pk_1
Pk_bar
alpha
nf
plt_hf_ellipse : list
plt_robotEllipse
plt_samples : list
plt_zf_ellipse : list
plt_zf_line : list
trajectory
xB : int
xB_dim
xBpose_dim
xF_dim
x_eta : int
xk
xk_1
xk_bar
zfi_dim
DataAssociation(xk, Pk, zf, Rf)
ICNN(hf, Phf, zf, Rf, dim)
IndividualCompatibility(D2_ij, dof, alpha)
Localize(xk_1, Pk_1)
PlotExpectedFeaturesObservationsUncertainty()
PlotFeatureObservationUncertainty(zf, Rf, color)
PlotRobotUncertainty()
PlotSample(x, P, n, color)
PlotSampleObservationSpace(NxB, BxFj, BPFj, n, color)
PlotUncertainty(zf, Rf)
SplitFeatures(zf, Rf, H)
SquaredMahalanobisDistance(hfj, Pfj, zfi, Rfi)
StackMeasurementsAndFeatures(zm, Rm, Hm, Vm, zf, Rf, H)
h(xk)
hm(xk)
EKF_3DOFDifferentialDriveInputDisplacement.EKF_3DOFDifferentialDriveInputDisplacement
Dt : float, int
dt : float
etak_1 : Pose3D
t : float, int
t_1 : int, float
uk
xk_bar : Pose3D
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
f(xk_1, uk)
h(xk)
MapFeature.Cartesian2DMapFeature
GetFeatures()
MBL_3DOFDDInputDisplacementMM_2DCartesianFeatureOM.MBL_3DOFDDInputDisplacementMM_2DCartesianFeatureOM
Feature
Pose

class MBL_3DOFDDInputDisplacementMM_2DCartesianFeatureOM.**MBL_3DOFDDInputDisplacementMM_2DCartesianFeatu**

Bases: CartesianMapFeature, *FEKFMBL*, *EKF_3DOFDifferentialDriveInputDisplacement*

Feature EKF Map based Localization of a 3 DOF Differential Drive Mobile Robot ($x_k = [{}^N x_{B_k} \ {}^N y_{B_k} \ {}^N \psi_{B_k}]^T$) using a 2D Cartesian feature map ($M = [[{}^N x_{F_1} \ {}^N y_{F_1}] \ [x_{F_2} \ {}^N y_{F_2}] \ ... \ [{}^N x_{F_n} \ {}^N y_{F_n}]]^T$), and an input displacement motion model ($u_k = [{}^B \Delta x_k \ {}^B \Delta y_k^B \Delta z_k \ {}^B \Delta \psi_k]^T$). The class inherits from the following classes: * Cartesian2DMapFeature: 2D Cartesian MapFeature using the Catesian coordinates for both, storage and landmark observations. * FEKFMBL: Feature EKF Map based Localization class. * EKF_3DOFDifferentialDriveInputDisplacement: EKF for 3 DOF Differential Drive Mobile Robot with input displacement motion model.

__init__(**args*)

Constructor of the FEKFMBL class.

Parameters

- **M** – Feature Based Map $M = [{}^N x_{F_1}^T \ ... \ {}^N x_{F_{n_f}}^T]^T$
- **alpha** – Chi2 tail probability. Confidence interaval of the individual compatibility test
- **args** – arguments to be passed to the EKFLocalization constructor

Differential Drive EKF Map Based Localization EKF Using an Input displacement Motion Model and 2D Feature Store in Cartesina and Observed in Polar Coordinates

class MBL_3DOFDDInputDisplacementMM_2DCartesianFeaturePolarObservedOM.**MBL_3DOFDDInputDisplacementMM_2DC**

Bases: CartesianStoredPolarObservedMapFeature, *FEKFMBL*, *EKF_3DOFDifferentialDriveInputDisplacement*

Feature EKF Map based Localization of a 3 DOF Differential Drive Mobile Robot ($x_k = [{}^N x_{B_k} \ {}^N y_{B_k} \ {}^N \psi_{B_k}]^T$) using a 2D Cartesian feature map ($M = [[{}^N x_{F_1} \ {}^N y_{F_1}] \ [x_{F_2} \ {}^N y_{F_2}] \ ... \ [{}^N x_{F_n} \ {}^N y_{F_n}]]^T$), and an input displacement motion model ($u_k = [{}^B \Delta x_k \ {}^B \Delta y_k \ {}^B \Delta z_k \ {}^B \Delta \psi_k]^T$). The class inherits from the following classes: * Cartesian2DStoredPolarObservedMapFeature: 2D Cartesian MapFeature using the Catesian coordinates for storage and polar coordinates for landmark observations. * FEKFMBL: Feature EKF Map based Localization class. * EKF_3DOFDifferentialDriveInputDisplacement: EKF for 3 DOF Differential Drive Mobile Robot with input displacement motion model.

__init__(**args*)

Constructor of the FEKFMBL class.

Parameters

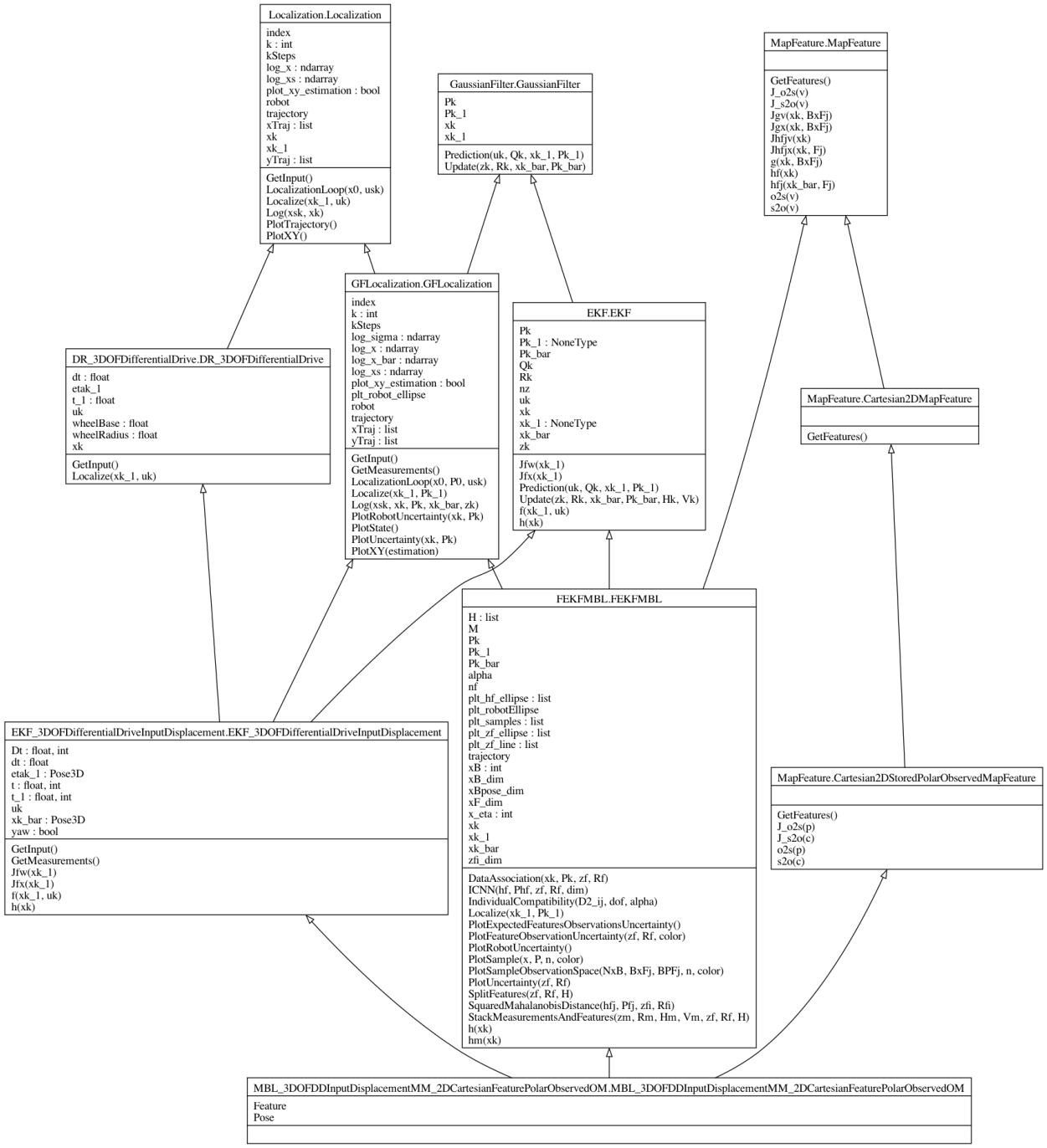
- **M** – Feature Based Map $M = [{}^N x_{F_1}^T \ ... \ {}^N x_{F_{n_f}}^T]^T$
- **alpha** – Chi2 tail probability. Confidence interaval of the individual compatibility test
- **args** – arguments to be passed to the EKFLocalization constructor

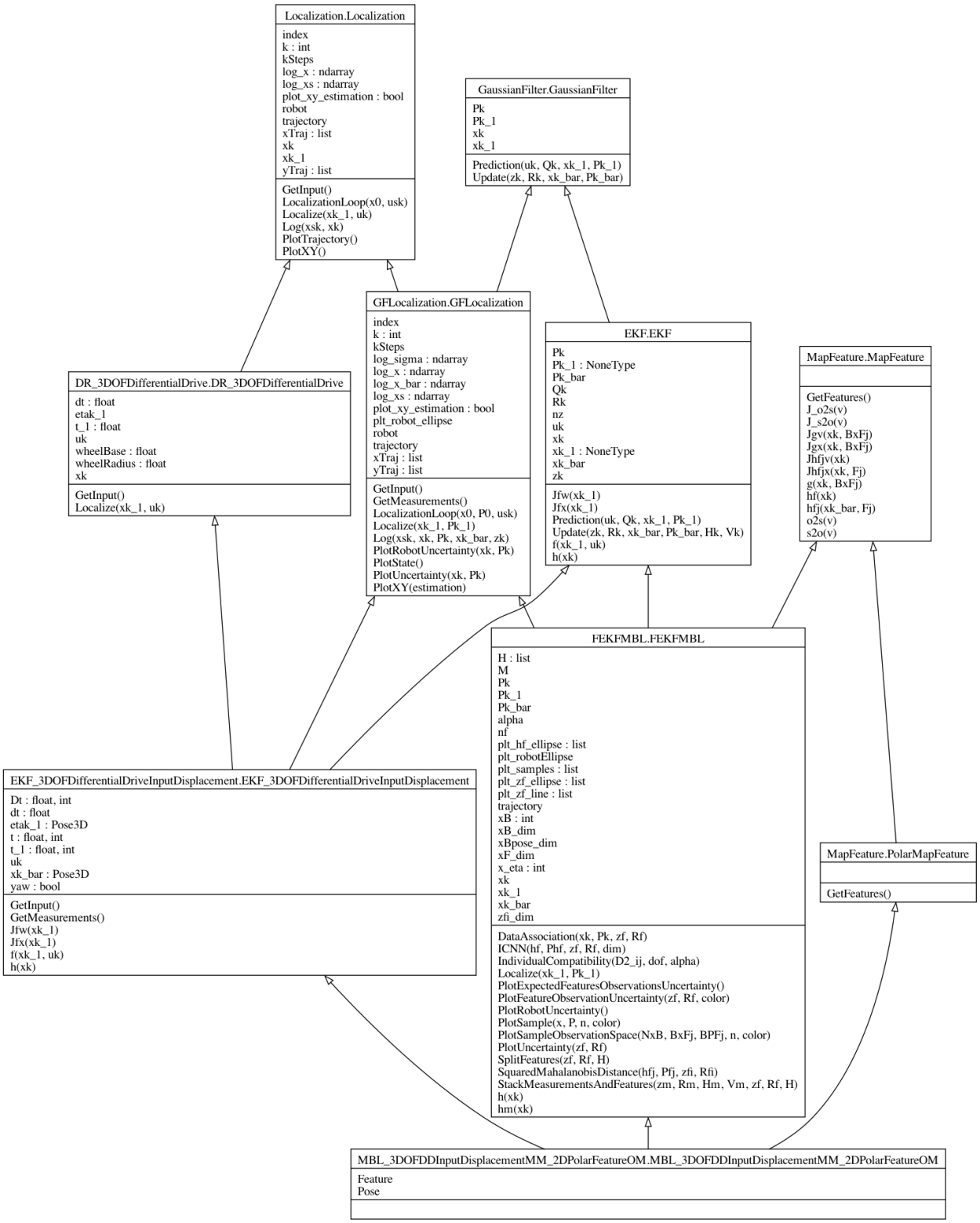
Differential Drive EKF Map Based Localization EKF Using an Input displacement Motion Model and 2D Polar Feature Observation Model

class MBL_3DOFDDInputDisplacementMM_2DPolarFeatureOM.**MBL_3DOFDDInputDisplacementMM_2DPolarFeatureOM**(**arg*

Bases: PolarMapFeature, *FEKFMBL*, *EKF_3DOFDifferentialDriveInputDisplacement*

Feature EKF Map based Localization of a 3 DOF Differential Drive Mobile Robot ($x_k = [{}^N x_{B_k} \ {}^N y_{B_k} \ {}^N \psi_{B_k}]^T$) using a 2D Cartesian feature map ($M = [[{}^N x_{F_1} \ {}^N y_{F_1}] \ [x_{F_2} \ {}^N y_{F_2}] \ ... \ [{}^N x_{F_n} \ {}^N y_{F_n}]]^T$), and an input displacement motion model ($u_k = [{}^B \Delta x_k \ {}^B \Delta y_k \ {}^B \Delta z_k \ {}^B \Delta \psi_k]^T$). The class inherits from the following classes: * Cartesian2DMapFeature: 2D Cartesian MapFeature using the Catesian coordinates


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
MapFeature.MapFeature
GetFeatures()
J_o2s(v)
J_s2o(v)
Jgv(xk, BxFj)
Jgx(xk, BxFj)
Jhfjv(xk)
Jhfjx(xk, Fj)
g(xk, BxFj)
hf(xk)
hfj(xk_bar, Fj)
o2s(v)
s2o(v)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)
MapFeature.Cartesian2DMapFeature
GetFeatures()
FEKFMBL.FEKFMBL
H : list
M
Pk
Pk_1
Pk_bar
alpha
nf
plt_hf_ellipse : list
plt_robotEllipse
plt_samples : list
plt_zf_ellipse : list
plt_zf_line : list
trajectory
xB : int
xB_dim
xBpose_dim
xF_dim
x_eta : int
xk
xk_1
xk_bar
zfi_dim
DataAssociation(xk, Pk, zf, Rf)
ICNN(hf, Phf, zf, Rf, dim)
IndividualCompatibility(D2_ij, dof, alpha)
Localize(xk_1, Pk_1)
PlotExpectedFeaturesObservationsUncertainty()
PlotFeatureObservationUncertainty(zf, Rf, color)
PlotRobotUncertainty()
PlotSample(x, P, n, color)
PlotSampleObservationSpace(NxB, BxFj, BPFj, n, color)
PlotUncertainty(zf, Rf)
SplitFeatures(zf, Rf, H)
SquaredMahalanobisDistance(hfj, Pfj, zfi, Rfi)
StackMeasurementsAndFeatures(zm, Rm, Hm, Vm, zf, Rf, H)
h(xk)
hm(xk)
EKF_3DOFDifferentialDriveInputDisplacement.EKF_3DOFDifferentialDriveInputDisplacement
Dt : float, int
dt : float
etak_1 : Pose3D
t : float, int
t_1 : float, int
uk
xk_bar : Pose3D
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
f(xk_1, uk)
h(xk)
MapFeature.Cartesian2DStoredPolarObservedMapFeature
GetFeatures()
J_o2s(p)
J_s2o(c)
o2s(p)
s2o(c)
MBL_3DOFDDInputDisplacementMM_2DCartesianFeaturePolarObservedOM.MBL_3DOFDDInputDisplacementMM_2DCartesianFeaturePolarObservedOM
Feature
Pose


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
MapFeature.MapFeature
GetFeatures()
J_o2s(v)
J_s2o(v)
Jgv(xk, BxFj)
Jgx(xk, BxFj)
Jhfjv(xk)
Jhfjx(xk, Fj)
g(xk, BxFj)
hf(xk)
hfj(xk_bar, Fj)
o2s(v)
s2o(v)
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)
FEKFMBL.FEKFMBL
H : list
M
Pk
Pk_1
Pk_bar
alpha
nf
plt_hf_ellipse : list
plt_robotEllipse
plt_samples : list
plt_zf_ellipse : list
plt_zf_line : list
trajectory
xB : int
xB_dim
xBpose_dim
xF_dim
x_eta : int
xk
xk_1
xk_bar
zfi_dim
DataAssociation(xk, Pk, zf, Rf)
ICNN(hf, Phf, zf, Rf, dim)
IndividualCompatibility(D2_ij, dof, alpha)
Localize(xk_1, Pk_1)
PlotExpectedFeaturesObservationsUncertainty()
PlotFeatureObservationUncertainty(zf, Rf, color)
PlotRobotUncertainty()
PlotSample(x, P, n, color)
PlotSampleObservationSpace(NxB, BxFj, BPFj, n, color)
PlotUncertainty(zf, Rf)
SplitFeatures(zf, Rf, H)
SquaredMahalanobisDistance(hfj, Pfj, zfi, Rfi)
StackMeasurementsAndFeatures(zm, Rm, Hm, Vm, zf, Rf, H)
h(xk)
hm(xk)
EKF_3DOFDifferentialDriveInputDisplacement.EKF_3DOFDifferentialDriveInputDisplacement
Dt : float, int
dt : float
etak_1 : Pose3D
t : float, int
t_1 : float, int
uk
xk_bar : Pose3D
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
f(xk_1, uk)
h(xk)
MapFeature.PolarMapFeature
GetFeatures()
MBL_3DOFDDInputDisplacementMM_2DPolarFeatureOM.MBL_3DOFDDInputDisplacementMM_2DPolarFeatureOM
Feature
Pose

for both, storage and landmark observations. * FEKFMBL: Feature EKF Map based Localization class. * EKF_3DOFDifferentialDriveInputDisplacement: EKF for 3 DOF Differential Drive Mobile Robot with input displacement motion model.

__init__(**args*)

Constructor of the FEKFMBL class.

Parameters

- **M** – Feature Based Map $M = [{}^N x_{F_1}^T \ ... \ {}^N x_{F_{n_f}}^T]^T$
- **alpha** – Chi2 tail probability. Confidence interaval of the individual compatibility test
- **args** – arguments to be passed to the EKFLocalization constructor

Differential Drive EKF Map Based Localization EKF Using a Ct Velocity Motion Model and 2D Cartesian Feature Observation Model

class MBL_3DOFDDCtVelocityMM_2DCartesianFeatureOM.**MBL_3DOFDDCtVelocityMM_2DCartesianFeatureOM**(**args*)

Bases: CartesianMapFeature, *FEKFMBL*, *EKF_3DOFDifferentialDriveCtVelocity*

Feature EKF Map based Localization of a 3 DOF Differential Drive Mobile Robot ($x_k = [{}^N x_{B_k} \ {}^N y_{B_k} \ {}^N \psi_{B_k}]^T$) using a 2D Cartesian feature map ($M = [[{}^N x_{F_1} \ {}^N y_{F_1}] \ [x_{F_2} \ {}^N y_{F_2}] \ ... \ [{}^N x_{F_n} \ {}^N y_{F_n}]]^T$), and a Constant Velocity Motion model with encoder readings. The class inherits from the following classes: * Cartesian2DMapFeature: 2D Cartesian MapFeature using the Catesian coordinates for both, storage and landmark observations. * FEKFMBL: Feature EKF Map based Localization class. * EKF_3DOFDifferentialDriveCtVelocity: EKF for 3 DOF Differential Drive Mobile Robot with Constant Velocity Motion Model and encoder readings.

__init__(**args*)

Constructor of the FEKFMBL class.

Parameters

- **M** – Feature Based Map $M = [{}^N x_{F_1}^T \ ... \ {}^N x_{F_{n_f}}^T]^T$
- **alpha** – Chi2 tail probability. Confidence interaval of the individual compatibility test
- **args** – arguments to be passed to the EKFLocalization constructor

1.7 Simulatenous Localization And Mapping

1.7.1 Feature based EKF Simultaneous Localization And Mapping

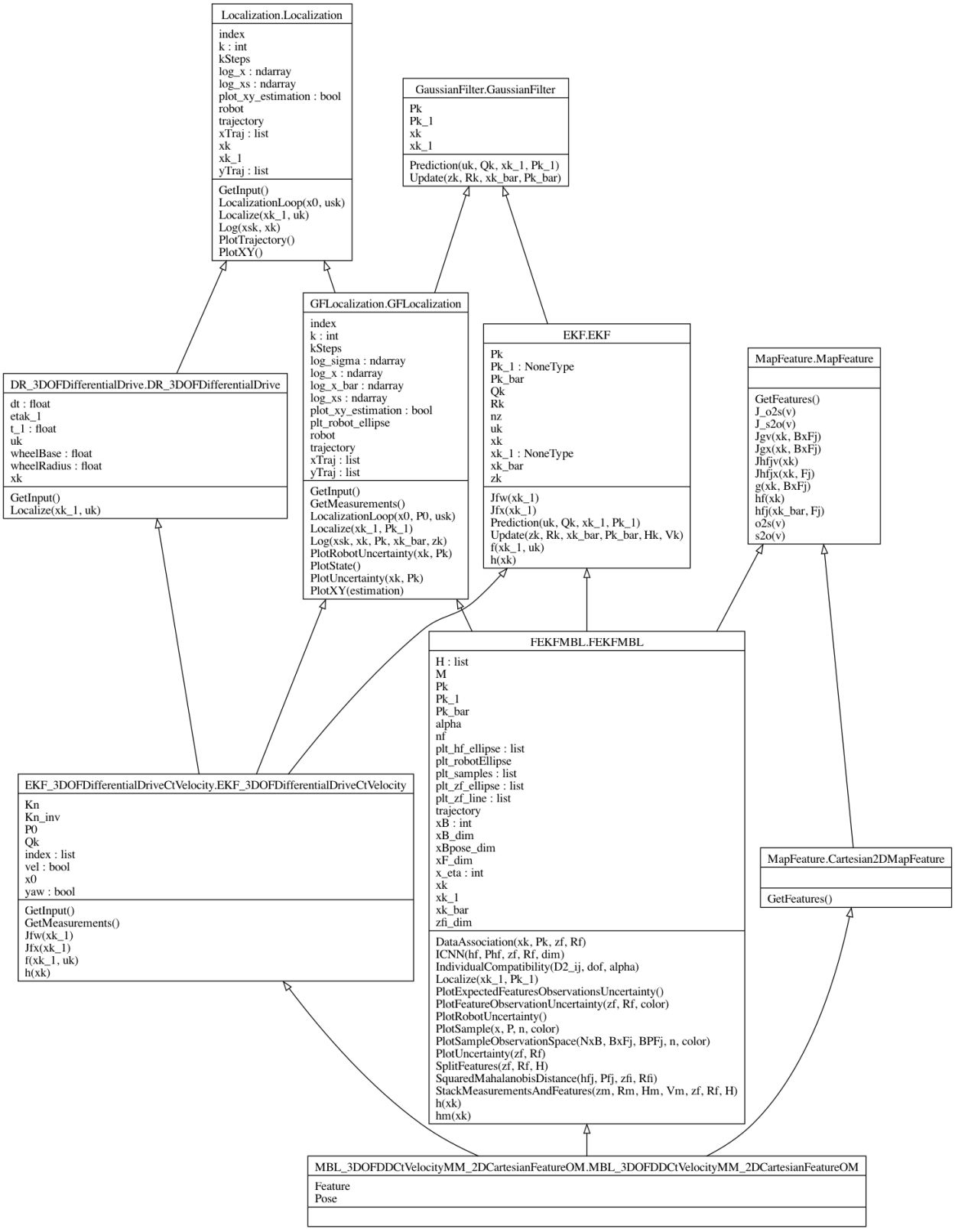
Feature based EKF Simultaneous Localization And Mapping

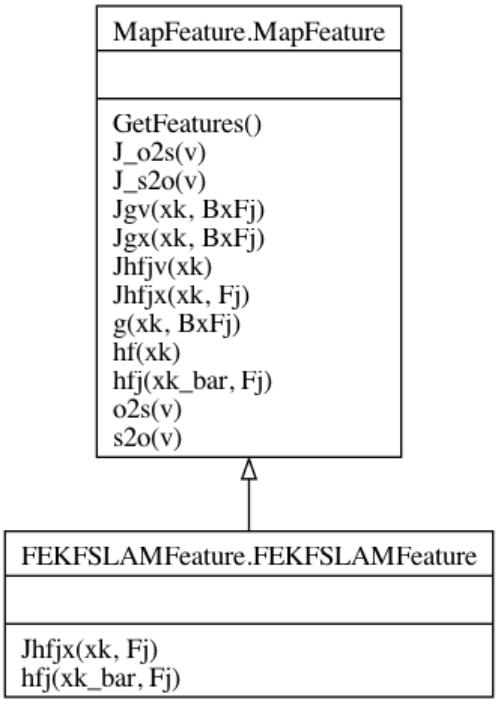
FEKFSLAM Feature

class FEKFSLAMFeature.**FEKFSLAMFeature**(**args*)

Bases: *MapFeature*

This class extends the MapFeature class to implement the Feature EKF SLAM algorithm. The :class:MapFeature class is a base class providing support to localize the robot using a map of point features. The main difference between FEKMBL and FEAKFSLAM is that the former uses the robot pose as a state variable, while the latter uses the robot pose and the feature map as state variables. This means that few methods provided


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
MapFeature.MapFeature
GetFeatures()
J_o2s(v)
J_s2o(v)
Jgv(xk, BxFj)
Jgx(xk, BxFj)
Jhfjv(xk)
Jhfjx(xk, Fj)
g(xk, BxFj)
hf(xk)
hfj(xk_bar, Fj)
o2s(v)
s2o(v)
DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive
dt : float
etak_1
t_1 : float
uk
wheelBase : float
wheelRadius : float
xk
GetInput()
Localize(xk_1, uk)
FEKFMBL.FEKFMBL
H : list
M
Pk
Pk_1
Pk_bar
alpha
nf
plt_hf_ellipse : list
plt_robotEllipse
plt_samples : list
plt_zf_ellipse : list
plt_zf_line : list
trajectory
xB : int
xB_dim
xBpose_dim
xF_dim
x_eta : int
xk
xk_1
xk_bar
zfi_dim
DataAssociation(xk, Pk, zf, Rf)
ICNN(hf, Phf, zf, Rf, dim)
IndividualCompatibility(D2_ij, dof, alpha)
Localize(xk_1, Pk_1)
PlotExpectedFeaturesObservationsUncertainty()
PlotFeatureObservationUncertainty(zf, Rf, color)
PlotRobotUncertainty()
PlotSample(x, P, n, color)
PlotSampleObservationSpace(NxB, BxFj, BPFj, n, color)
PlotUncertainty(zf, Rf)
SplitFeatures(zf, Rf, H)
SquaredMahalanobisDistance(hfj, Pfj, zfi, Rfi)
StackMeasurementsAndFeatures(zm, Rm, Hm, Vm, zf, Rf, H)
h(xk)
hm(xk)
EKF_3DOFDifferentialDriveCtVelocity.EKF_3DOFDifferentialDriveCtVelocity
Kn
Kn_inv
P0
Qk
index : list
vel : bool
x0
yaw : bool
GetInput()
GetMeasurements()
Jfw(xk_1)
Jfx(xk_1)
f(xk_1, uk)
h(xk)
MapFeature.Cartesian2DMapFeature
GetFeatures()
MBL_3DOFDDCtVelocityMM_2DCartesianFeatureOM.MBL_3DOFDDCtVelocityMM_2DCartesianFeatureOM
Feature
Pose



by class need to be overridden to gather the information from state vector instead that from the deterministic map.

hfj(*xk_bar*, *Fj*)

This method implements the direct observation model for a single feature observation z_{f_i} , so it implements its related observation function (see eq. (**??**)). For a single feature observation z_{f_i} of the feature ${}^N x_{F_j}$ the method computes its expected observation from the current robot pose ${}^N x_B$. This function uses a generic implementation through the following equation:

$$z_{f_i} = h_{Fj}(x_k, v_k) = s2o(\ominus {}^N x_B \boxplus {}^N x_{F_j}) + v_{fi_k} \tag{1.60}$$

Where ${}^N x_B$ is the robot pose and ${}^N x_{F_j}$ are both included within the state vector:

$$x_k = [{}^N x_B^T \ \cdots \ {}^N x_{F_j} \ \cdots \ {}^N x_{F_{nf}}]^T \tag{1.61}$$

and `s2o()` is a conversion function from the store representation to the observation representation.

The method is called by `FEKFSLAM.hf()` to compute the expected observation for each feature observation contained in the observation vector $z_f = [z_{f_1}^T \ \cdots \ z_{f_i}^T \ \cdots \ z_{f_{nzf}}^T]^T$.

Parameters

- **xk_bar** – mean of the predicted state vector
- **Fj** – map index of the observed feature.

Returns

expected observation of the feature ${}^N x_{F_j}$

Jhfjx(*xk*, *Fj*)

Jacobian of the single feature direct observation model ***hfj()*** (eq. (**??**)) with respect to the state vector $\bar{x}_k$:

$$\begin{aligned} x_k &= [{}^N x_B^T \ \cdots \ {}^N x_{F_j} \ \cdots \ {}^N x_{F_{nf}}]^T \\ J_{hfjx} &= \frac{\partial h_{f_{zfi}}(x_k, v_k)}{\partial x_k} = \frac{\partial s2o(\ominus^N x_B \boxplus^N x_{F_j}) + v_{fi_k}}{\partial x_k} \\ &= \left[\frac{\partial h_{F_j}(x_k,v_k)}{\partial {}^N x_{B_k}} \quad \frac{\partial h_{F_j}(x_k,v_k)}{\partial {}^N x_{F_1}} \quad \cdots \quad \frac{\partial h_{F_j}(x_k,v_k)}{\partial {}^N x_{F_j}} \quad \cdots \quad \frac{\partial h_{F_j}(x_k,v_k)}{\partial {}^N x_{F_n}}\right] \\ &= \left[J_{s2o} J_{1\boxplus} J_{\ominus} \quad 0 \quad \cdots \quad J_{s2o} J_{2\boxplus} \quad \cdots \quad 0\right] \end{aligned} \tag{1.62}$$

where we have used the abreviature:

$$\begin{aligned} J_{s2o} &\equiv J_{s2o}(\ominus^N x_B \boxplus^N x_{F_j}) \\ J_{1\boxplus} &\equiv J_{1\boxplus}(\ominus^N x_B, {}^N x_{F_j}) \\ J_{2\boxplus} &\equiv J_{2\boxplus}(\ominus^N x_B, {}^N x_{F_j}) \end{aligned}$$

Parameters

- **xk** – state vector mean
- **Fj** – map index of the observed feature

Returns

Jacobian matrix defined in eq. (**??**)

FEKF SLAM Localization

class FEKFSLAM.**FEKFSLAM**(**args*)

Bases: *FEKFMBL*

Class implementing the Feature-based Extended Kalman Filter for Simultaneous Localization and Mapping (FEKFSLAM). It inherits from the FEKFMBL class, which implements the Feature Map based Localization (MBL) algorithm using an EKF. *FEKFSLAM* extends FEKFMBL by adding the capability to map features previously unknown to the robot.

xB = -2

x_eta = -1

__init__(**args*)

Constructor of the FEKFMBL class.

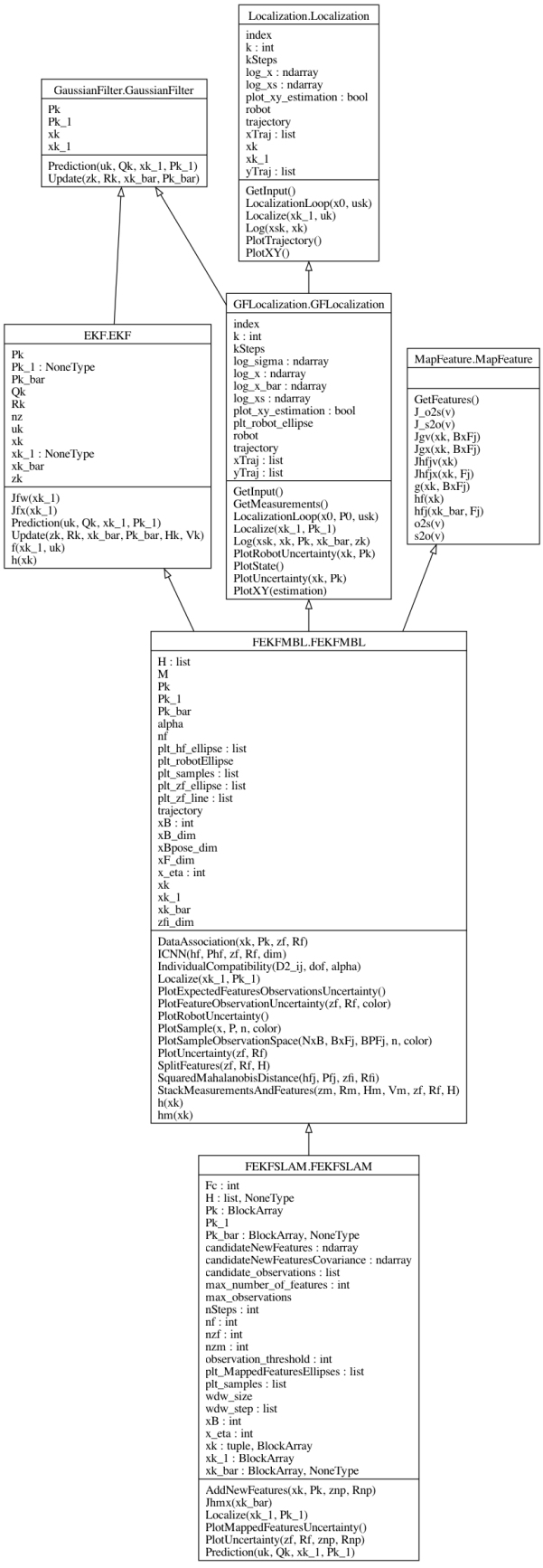
Parameters

- **M** – Feature Based Map $M = [{}^N x_{F_1}^T \ ... \ {}^N x_{F_{n_f}}^T]^T$
- **alpha** – Chi2 tail probability. Confidence interaval of the individual compatibility test
- **args** – arguments to be passed to the EKFLocalization constructor

AddNewFeatures(*xk*, *Pk*, *znp*, *Rnp*)

This method adds new features to the map. Given:

- The SLAM state vector mean and covariance:


Localization.Localization
index
k : int
kSteps
log_x : ndarray
log_xs : ndarray
plot_xy_estimation : bool
robot
trajectory
xTraj : list
xk
xk_1
yTraj : list
GetInput()
LocalizationLoop(x0, usk)
Localize(xk_1, uk)
Log(xsk, xk)
PlotTrajectory()
PlotXY()
GaussianFilter.GaussianFilter
Pk
Pk_1
xk
xk_1
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar)
GFLocalization.GFLocalization
index
k : int
kSteps
log_sigma : ndarray
log_x : ndarray
log_x_bar : ndarray
log_xs : ndarray
plot_xy_estimation : bool
plt_robot_ellipse
robot
trajectory
xTraj : list
yTraj : list
GetInput()
GetMeasurements()
LocalizationLoop(x0, P0, usk)
Localize(xk_1, Pk_1)
Log(xsk, xk, Pk, xk_bar, zk)
PlotRobotUncertainty(xk, Pk)
PlotState()
PlotUncertainty(xk, Pk)
PlotXY(estimation)
EKF.EKF
Pk
Pk_1 : NoneType
Pk_bar
Qk
Rk
nz
uk
xk
xk_1 : NoneType
xk_bar
zk
Jfw(xk_1)
Jfx(xk_1)
Prediction(uk, Qk, xk_1, Pk_1)
Update(zk, Rk, xk_bar, Pk_bar, Hk, Vk)
f(xk_1, uk)
h(xk)
MapFeature.MapFeature
GetFeatures()
J_o2s(v)
J_s2o(v)
Jgv(xk, BxFj)
Jgx(xk, BxFj)
Jhfjv(xk)
Jhfjx(xk, Fj)
g(xk, BxFj)
hf(xk)
hfj(xk_bar, Fj)
o2s(v)
s2o(v)
FEKFMBL.FEKFMBL
H : list
M
Pk
Pk_1
Pk_bar
alpha
nf
plt_hf_ellipse : list
plt_robotEllipse
plt_samples : list
plt_zf_ellipse : list
plt_zf_line : list
trajectory
xB : int
xB_dim
xBpose_dim
xF_dim
x_eta : int
xk
xk_1
xk_bar
zfi_dim
DataAssociation(xk, Pk, zf, Rf)
ICNN(hf, Phf, zf, Rf, dim)
IndividualCompatibility(D2_ij, dof, alpha)
Localize(xk_1, Pk_1)
PlotExpectedFeaturesObservationsUncertainty()
PlotFeatureObservationUncertainty(zf, Rf, color)
PlotRobotUncertainty()
PlotSample(x, P, n, color)
PlotSampleObservationSpace(NxB, BxFj, BPFj, n, color)
PlotUncertainty(zf, Rf)
SplitFeatures(zf, Rf, H)
SquaredMahalanobisDistance(hfj, Pfj, zfi, Rfi)
StackMeasurementsAndFeatures(zm, Rm, Hm, Vm, zf, Rf, H)
h(xk)
hm(xk)
FEKFSLAM.FEKFSLAM
Fc : int
H : list, NoneType
Pk : BlockArray
Pk_1
Pk_bar : BlockArray, NoneType
candidateNewFeatures : ndarray
candidateNewFeaturesCovariance : ndarray
candidate_observations : list
max_number_of_features : int
max_observations
nSteps : int
nf : int
nzf : int
nzm : int
observation_threshold : int
plt_MappedFeaturesEllipses : list
plt_samples : list
wdw_size
wdw_step : list
xB : int
x_eta : int
xk : tuple, BlockArray
xk_1 : BlockArray
xk_bar : BlockArray, NoneType
AddNewFeatures(xk, Pk, znp, Rnp)
Jhmx(xk_bar)
Localize(xk_1, Pk_1)
PlotMappedFeaturesUncertainty()
PlotUncertainty(zf, Rf, znp, Rnp)
Prediction(uk, Qk, xk_1, Pk_1)

$$
\begin{aligned}
{}^{N}x_k &\approx \mathcal{N}({}^{N}\hat{x}_k, {}^{N}P_k) \\
{}^{N}\hat{x}_k &= \begin{bmatrix} {}^{N}\hat{x}_{B_k}^T & {}^{N}\hat{x}_{F_1}^T & \cdots & {}^{N}\hat{x}_{F_{nf}}^T \end{bmatrix}^T \\
{}^{N}P_k &= \begin{bmatrix} {}^{N}P_B & {}^{N}P_{BF_1} & \cdots & {}^{N}P_{BF_{nf}} \\ {}^{N}P_{F_1B} & {}^{N}P_{F_1} & \cdots & {}^{N}P_{F_1F_{nf}} \\ \vdots & \vdots & \ddots & \vdots \\ {}^{N}P_{F_{nf}B} & {}^{N}P_{F_{nf}F_1} & \cdots & {}^{N}P_{nf} \end{bmatrix}
\end{aligned}
\tag{1.63}
$$

- And the vector of non-paired feature observations (feature which have not been associated with any feature in the map), and their covariance matrix:

$$
\begin{aligned}
z_{np} &= \begin{bmatrix} {}^{B}z_{F_1} & \cdots & {}^{B}z_{F_{n_{zf}}} \end{bmatrix}^T \\
R_{np} &= \begin{bmatrix} {}^{B}R_{F_1} & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & {}^{B}R_{F_{n_{zf}}} \end{bmatrix}
\end{aligned}
\tag{1.64}
$$

this method creates a grown state vector ([xk_plus, Pk_plus]) by adding the new features to the state vector. Therefore, for each new feature ${}^{B}z_{F_i}$, included in the vector z_{np}, and its corresponding feature observation noise ${}^{B}R_{F_i}$, the state vector mean and covariance are updated as follows:

$$
\begin{aligned}
{}^{N}x_k^+ &\approx \mathcal{N}({}^{N}\hat{x}_k^+, {}^{N}P_k^+) \\
{}^{N}x_k^+ &= \begin{bmatrix} {}^{N}x_{B_k}^T & {}^{N}x_{F_1}^T & \cdots & {}^{N}x_{F_n}^T \mid \left({}^{N}x_{B_k} \boxplus ({}^{B}z_{F_i} + v_k)\right)^T \end{bmatrix}^T \\
{}^{N}\hat{x}_k^+ &= \begin{bmatrix} {}^{N}\hat{x}_{B_k}^T & {}^{N}\hat{x}_{F_1}^T & \cdots & {}^{N}\hat{x}_{F_n}^T \mid {}^{N}\hat{x}_{B_k} \boxplus {}^{B}z_{F_i}^T \end{bmatrix}^T \\
P_k^+ &= \left[\begin{array}{cccc|c} {}^{N}P_{B_k} & {}^{N}P_{B_kF_1} & \cdots & {}^{N}P_{B_kF_n} & {}^{N}P_{B_k}J_{1\boxplus}^T \\ {}^{N}P_{F_1B_k} & {}^{N}P_{F_1} & \cdots & {}^{N}P_{F_1F_n} & {}^{N}P_{F_1B_k}J_{1\boxplus}^T \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ {}^{N}P_{F_nB_k} & {}^{N}P_{F_nF_1} & \cdots & {}^{N}P_{F_n} & {}^{N}P_{F_nB_k}J_{1\boxplus}^T \\ \hline J_{1\boxplus}{}^{N}P_{B_k} & J_{1\boxplus}{}^{N}P_{B_kF_1} & \cdots & J_{1\boxplus}{}^{N}P_{B_kF_n} & J_{1\boxplus}{}^{N}P_RJ_{1\boxplus}^T + J_{2\boxplus}{}^{B}R_{F_i}J_{2\boxplus}^T \end{array}\right]
\end{aligned}
\tag{1.65}
$$

Parameters

- **xk** – state vector mean
- **Pk** – state vector covariance
- **znp** – vector of non-paired feature observations (they have not been associated with any feature in the map)
- **Rnp** – Matrix of non-paired feature observation covariances

Returns

[xk_plus, Pk_plus] state vector mean and covariance after adding the new features

Prediction(*uk*, *Qk*, *xk_1*, *Pk_1*)

This method implements the prediction step of the FEKFSLAM algorithm. It predicts the state vector mean

and covariance at the next time step. Given state vector mean and covariance at time step k-1:

$$
\begin{aligned}
{}^{N}x_{k-1} &\approx \mathcal{N}({}^{N}\hat{x}_{k-1}, {}^{N}P_{k-1}) \\
{}^{N}\hat{x}_{k-1} &= \begin{bmatrix} {}^{N}\hat{x}_{B_{k-1}}^{T} & {}^{N}\hat{x}_{F_1}^{T} & \cdots & {}^{N}\hat{x}_{F_{nf}}^{T} \end{bmatrix}^{T} \\
{}^{N}P_{k-1} &= \begin{bmatrix} {}^{N}P_{B_{k-1}} & {}^{N}P_{BF_1} & \cdots & {}^{N}P_{BF_{nf}} \\ {}^{N}P_{F_1B} & {}^{N}P_{F_1} & \cdots & {}^{N}P_{F_1F_{nf}} \\ \vdots & \vdots & \ddots & \vdots \\ {}^{N}P_{F_{nf}B} & {}^{N}P_{F_{nf}F_1} & \cdots & {}^{N}P_{nf} \end{bmatrix}
\end{aligned}
\tag{1.66}
$$

the control input and its covariance u_k and Q_k, the method computes the state vector mean and covariance at time step k:

$$
\begin{aligned}
{}^{N}\hat{\bar{x}}_k &= \begin{bmatrix} f\left({}^{N}\hat{x}_{B_{k-1}}, u_k\right) & {}^{N}\hat{x}_{F_1}^{T} \cdots {}^{N}\hat{x}_{F_n}^{T} \end{bmatrix}^{T} \\
{}^{N}\bar{P}_k &= F_{1_k}\, {}^{N}P_{k-1} F_{1_k}^{T} + F_{2_k} Q_k F_{2_k}^{T}
\end{aligned}
\tag{1.67}
$$

where

$$
\begin{aligned}
F_{1_k} &= \left.\frac{\partial f_S({}^{N}x_{k-1}, u_k, w_k)}{\partial {}^{N}x_{k-1}}\right|_{\substack{{}^{N}x_{k-1}={}^{N}\hat{x}_{k-1} \\ w_k=0}} \\
&= \begin{bmatrix} \frac{\partial f\left({}^{N}x_{B_{k-1}}, u_k, w_k\right)}{\partial {}^{N}x_{B_{k-1}}} & \frac{\partial f\left({}^{N}x_{B_{k-1}}, u_k, w_k\right)}{\partial {}^{N}x_{F1}} & \cdots & \frac{\partial f\left({}^{N}x_{B_{k-1}}, u_k, w_k\right)}{\partial {}^{N}x_{Fn}} \\ \frac{\partial {}^{N}x_{F1}}{\partial {}^{N}x_{k-1}} & \frac{\partial {}^{N}x_{F1}}{\partial {}^{N}x_{F1}} & \cdots & \frac{\partial {}^{N}x_{Fn}}{\partial {}^{N}x_{Fn}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial {}^{N}x_{Fn}}{\partial {}^{N}x_{k-1}} & \frac{\partial {}^{N}x_{Fn}}{\partial {}^{N}x_{F1}} & \cdots & \frac{\partial {}^{N}x_{Fn}}{\partial {}^{N}x_{Fn}} \end{bmatrix} = \begin{bmatrix} J_{f_x} & 0 & \cdots & 0 \\ 0 & I & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & I \end{bmatrix} \\
F_{2_k} &= \left.\frac{\partial f({}^{N}x_{k-1}, u_k, w_k)}{\partial w_k}\right|_{\substack{{}^{N}x_{k-1}={}^{N}\hat{x}_{k-1} \\ w_k=0}} = \begin{bmatrix} \frac{\partial f\left({}^{N}x_{B_{k-1}}, u_k, w_k\right)}{\partial w_k} \\ \frac{\partial {}^{N}x_{F1}}{\partial w_k} \\ \vdots \\ \frac{\partial {}^{N}x_{Fn}}{\partial w_k} \end{bmatrix} = \begin{bmatrix} J_{f_w} \\ 0 \\ \vdots \\ 0 \end{bmatrix}
\end{aligned}
\tag{1.68}
$$

obtaining the following covariance matrix:

$$
\begin{aligned}
{}^{N}\bar{P}_k &= F_{1_k}\, {}^{N}P_{k-1} F_{1_k}^{T} + F_{2_k} Q_k F_{2_k}^{T}\, {}^{N}\bar{P}_k \\
&= \left[\begin{array}{c|ccc} J_{f_x} P_{B_{k-1}} J_{f_x}^{T} + J_{f_w} Q J_{f_w}^{T} & J_{f_x} P_{B_k F_1} & \cdots & J_{f_x} P_{B_k F_n} \\ \hline {}^{N}P_{F_1 B_k} J_{f_x}^{T} & {}^{N}P_{F_1} & \cdots & {}^{N}P_{F_1 F_n} \\ \vdots & \vdots & \ddots & \vdots \\ {}^{N}P_{F_n B_k} J_{f_x}^{T} & {}^{N}P_{F_n F_1} & \cdots & {}^{N}P_{F_n} \end{array}\right]
\end{aligned}
\tag{1.69}
$$

The method returns the predicted state vector mean (${}^{N}\hat{\bar{x}}_k$) and covariance (${}^{N}\bar{P}_k$).

Parameters

- **uk** – Control input
- **Qk** – Covariance of the Motion Model noise
- **xk_1** – State vector mean at time step k-1
- **Pk_1** – Covariance of the state vector at time step k-1

Returns

[xk_bar, Pk_bar] predicted state vector mean and covariance at time step k

`Localize`(*xk_1*, *Pk_1*)

This method implements the FEKFSLAM algorithm. It localizes the robot and maps the features in the environment. It implements a single interation of the SLAM algorithm, given the current state vector mean and covariance. The unique difference it has with respect to its ancestor `FEKFMBL.Localize()` is that it calls the method *`AddNewFeatures()`* to add new non-paired features to the map.

Parameters

- **xk_1** – state vector mean at time step k-1
- **Pk_1** – covariance of the state vector at time step k-1

Returns

[xk, Pk] state vector mean and covariance at time step k

`PlotMappedFeaturesUncertainty`()

This method plots the uncertainty of the mapped features. It plots the uncertainty ellipses of the mapped features in the environment. It is called at each Localization iteration.

`PlotUncertainty`(*zf*, *Rf*, *znp*, *Rnp*)

This method plots the uncertainty of the robot (blue), the mapped features (red), the expected feature observations (black) and the feature observations.

Parameters

- **zf** – vector of feature observations
- **Rf** – covariance matrix of feature observations
- **znp** – vector of non-paired feature observations
- **Rnp** – covariance matrix of non-paired feature observations

Returns

CHAPTER

TWO

INDICES AND TABLES

- genindex
- modindex
- search

INDEX

Symbols

__init__() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 25
__init__() (*DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive method*), 40
__init__() (*DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro method*), 38
__init__() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 22
__init__() (*EKF.EKF method*), 34
__init__() (*EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM.EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM method*), 54
__init__() (*Feature.Feature method*), 11
__init__() (*GFLocalization.GFLocalization method*), 50
__init__() (*GL.GL method*), 42
__init__() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 47
__init__() (*GL_4DOFAUV.GL_4DOFAUV method*), 44
__init__() (*GaussianFilter.GaussianFilter method*), 32
__init__() (*HF.HF method*), 29
__init__() (*Histogram.Histogram2D method*), 31
__init__() (*KF.KF method*), 33
__init__() (*Localization.Localization method*), 37
__init__() (*Pose.Pose3D method*), 6
__init__() (*Pose.Pose4D method*), 8
__init__() (*SimulatedRobot.SimulatedRobot method*), 20

A

AUV4DOFSimulatedRobot (*class in AUV4DOFSimulatedRobot*), 25

B

boxplus() (*Feature.CartesianFeature method*), 12
boxplus() (*Feature.Feature method*), 11
boxplus() (*Feature.PolarFeature method*), 14
boxplus() (*Pose.Pose method*), 5

C

c2p() (*in module conversions*), 17
c2s() (*in module conversions*), 19
CartesianFeature (*class in Feature*), 12

D

DifferentialDriveSimulatedRobot (*class in DifferentialDriveSimulatedRobot*), 22
DR_3DOFDifferentialDrive (*class in DR_3DOFDifferentialDrive*), 40
DR_4DOFAUV_DVLGyro (*class in DR_4DOFAUV_DVLGyro*), 38
dt (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot attribute*), 25
dt (*SimulatedRobot.SimulatedRobot attribute*), 20

E

EKF (*class in EKF*), 34
EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM (*class in EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM*), 54
element (*Histogram.Histogram2D property*), 31

F

f() (*EKF.EKF method*), 34
f() (*EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM.EKF_4DOFAUV method*), 54
Feature (*class in Feature*), 11
fs() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 25
fs() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 22
fs() (*SimulatedRobot.SimulatedRobot method*), 21

G

GaussianFilter (*class in GaussianFilter*), 32
GetInput() (*DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive method*), 40
GetInput() (*DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro method*), 40

GetInput() (*GFLocalization.GFLocalization method*), 50
GetInput() (*GL.GL method*), 43
GetInput() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 50
GetInput() (*GL_4DOFAUV.GL_4DOFAUV method*), 47
GetInput() (*Localization.Localization method*), 38
GetMeasurements() (*GFLocalization.GFLocalization method*), 52
GetMeasurements() (*GL.GL method*), 43
GetMeasurements() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 49
GetMeasurements() (*GL_4DOFAUV.GL_4DOFAUV method*), 46
GFLocalization (*class in GFLocalization*), 50
GL (*class in GL*), 42
GL_3DOFDifferentialDrive (*class in GL_3DOFDifferentialDrive*), 47
GL_4DOFAUV (*class in GL_4DOFAUV*), 44

H

h() (*EKF.EKF method*), 36
HF (*class in HF*), 29
Histogram2D (*class in Histogram*), 31
histogram_1d (*Histogram.Histogram2D property*), 31
histogram_2d (*Histogram.Histogram2D property*), 31

I

IndexStruct (*class in IndexStruct*), 53

J

J_1boxplus() (*Feature.CartesianFeature method*), 14
J_1boxplus() (*Feature.Feature method*), 11
J_1boxplus() (*Feature.PolarFeature method*), 16
J_1boxplus() (*Pose.Pose method*), 5
J_1oplus() (*Pose.Pose method*), 3
J_1oplus() (*Pose.Pose3D method*), 6
J_1oplus() (*Pose.Pose4D method*), 10
J_2boxplus() (*Feature.CartesianFeature method*), 14
J_2boxplus() (*Feature.Feature method*), 12
J_2boxplus() (*Feature.PolarFeature method*), 16
J_2boxplus() (*Pose.Pose method*), 5
J_2c() (*Feature.CartesianFeature method*), 14
J_2c() (*Feature.Feature method*), 12
J_2c() (*Feature.PolarFeature method*), 16
J_2oplus() (*Pose.Pose method*), 3
J_2oplus() (*Pose.Pose3D method*), 6
J_2oplus() (*Pose.Pose4D method*), 10
J_c2p() (*in module conversions*), 17
J_c2s() (*in module conversions*), 19
J_ominus() (*Pose.Pose method*), 5
J_ominus() (*Pose.Pose3D method*), 8
J_ominus() (*Pose.Pose4D method*), 10
J_p2c() (*in module conversions*), 17
J_s2c() (*in module conversions*), 18
J_v2v() (*in module conversions*), 19
Jfw() (*EKF.EKF method*), 36
Jfw() (*EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM.EKF_4DOFAUV method*), 54
Jfx() (*EKF.EKF method*), 36
Jfx() (*EKF_4DOFAUV_CtVelocityMM_DVLDepthYawOM.EKF_4DOFAUV method*), 54

K

KF (*class in KF*), 33

L

Localization (*class in Localization*), 37
LocalizationLoop() (*GFLocalization.GFLocalization method*), 52
LocalizationLoop() (*GL.GL method*), 44
LocalizationLoop() (*Localization.Localization method*), 38
Localize() (*DR_3DOFDifferentialDrive.DR_3DOFDifferentialDrive method*), 40
Localize() (*DR_4DOFAUV_DVLGyro.DR_4DOFAUV_DVLGyro method*), 40
Localize() (*GFLocalization.GFLocalization method*), 52
Localize() (*GL.GL method*), 44
Localize() (*Localization.Localization method*), 38
Log() (*GFLocalization.GFLocalization method*), 52
Log() (*Localization.Localization method*), 38

M

MeasurementProbability() (*GL.GL method*), 43
MeasurementProbability() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 49
MeasurementProbability() (*GL_4DOFAUV.GL_4DOFAUV method*), 47
MeasurementProbability() (*HF.HF method*), 30

O

observation (*IndexStruct.IndexStruct attribute*), 54
ominus() (*Pose.Pose method*), 5
ominus() (*Pose.Pose3D method*), 8
ominus() (*Pose.Pose4D method*), 10
oplus() (*Pose.Pose method*), 3
oplus() (*Pose.Pose3D method*), 6
oplus() (*Pose.Pose4D method*), 8

P

p2c() (*in module conversions*), 17
plot_histogram() (*Histogram.Histogram2D method*), 31

PlotRobot() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 25
PlotRobot() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 25
PlotRobot() (*SimulatedRobot.SimulatedRobot method*), 21
PlotRobotUncertainty() (*GFLocalization.GFLocalization method*), 52
PlotState() (*GFLocalization.GFLocalization method*), 52
PlotTrajectory() (*Localization.Localization method*), 38
PlotUncertainty() (*GFLocalization.GFLocalization method*), 52
PlotXY() (*GFLocalization.GFLocalization method*), 52
PlotXY() (*Localization.Localization method*), 38
PolarFeature (*class in Feature*), 14
Pose (*class in Pose*), 3
Pose3D (*class in Pose*), 6
Pose4D (*class in Pose*), 8
Prediction() (*EKF.EKF method*), 36
Prediction() (*GaussianFilter.GaussianFilter method*), 32
Prediction() (*HF.HF method*), 30
Prediction() (*KF.KF method*), 34

R

ReadCartesianFeature() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 28
ReadCartesianFeature() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 24
ReadCartesianObservedSphericalFeature() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 28
ReadCompass() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 27
ReadCompass() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 24
ReadDepth() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 27
ReadDVL() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 27
ReadEncoders() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 24
ReadGyro() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 27
ReadPolarFeature() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 28
ReadPolarFeature() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 25
ReadRanges() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 28
ReadRanges() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 25
ReadSphericalFeature() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 28
ReadUSBL() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 28
ReadWheelAngularVelocity() (*DifferentialDriveSimulatedRobot.DifferentialDriveSimulatedRobot method*), 24

S

s2c() (*in module conversions*), 18
SetMap() (*AUV4DOFSimulatedRobot.AUV4DOFSimulatedRobot method*), 27
SetMap() (*SimulatedRobot.SimulatedRobot method*), 21
SimulatedRobot (*class in SimulatedRobot*), 20
simulation (*IndexStruct.IndexStruct attribute*), 53
state (*IndexStruct.IndexStruct attribute*), 53
StateTransitionProbability() (*GL.GL method*), 43
StateTransitionProbability() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 49
StateTransitionProbability() (*GL_4DOFAUV.GL_4DOFAUV method*), 46
StateTransitionProbability() (*HF.HF method*), 30
StateTransitionProbability_4_uk() (*GL.GL method*), 43
StateTransitionProbability_4_uk() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 49
StateTransitionProbability_4_uk() (*GL_4DOFAUV.GL_4DOFAUV method*), 46
StateTransitionProbability_4_uk() (*HF.HF method*), 30
StateTransitionProbability_4_xk_1_uk() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 49
StateTransitionProbability_4_xk_1_uk() (*GL_4DOFAUV.GL_4DOFAUV method*), 46

T

ToCartesian() (*Feature.CartesianFeature method*), 14
ToCartesian() (*Feature.Feature method*), 12
ToCartesian() (*Feature.PolarFeature method*), 16

ToCell() (*HF.HF method*), 30

U

uk2cell() (*GL.GL method*), 44
uk2cell() (*GL_3DOFDifferentialDrive.GL_3DOFDifferentialDrive method*), 49
uk2cell() (*GL_4DOFAUV.GL_4DOFAUV method*), 46
uk2cell() (*HF.HF method*), 30
Update() (*EKF.EKF method*), 36
Update() (*GaussianFilter.GaussianFilter method*), 32
Update() (*HF.HF method*), 30
Update() (*KF.KF method*), 34

V

v2v() (*in module conversions*), 19